PDF Digital Signatures — A Technical Reference

PDFalyzer Studio Contributors

2026-03-06

Contents

0.1 Preface . . . 1
0.1.1 Who This Book Is For . . . 1
0.1.2 What This Book Is Not . . . 1
0.1.3 How This Book Is Organized . . . 1
0.1.4 Conventions Used in This Book . . . 2
0.1.5 A Note on the PDF Specification . . . 3
0.1.6 Three Decades of PDF: A Timeline . . . 3
0.1.7 Acknowledgments . . . 10

1 PDF Digital Signatures **13**
1.1 Signature Architecture . . . 13
1.1.1 Overview: What a PDF Signature Actually Is . . . 13
1.1.2 Signature Fields in the AcroForm Tree . . . 14
1.1.3 The Signature Dictionary (`/Type /Sig`) . . . 18
1.1.4 The `/Contents` Value: CMS SignedData . . . 23
1.1.5 The Signing Process: Step by Step . . . 27
1.1.6 Incremental Save: The Foundation of Multi-Signature . . . 29
1.1.7 Widget Annotations and Page Placement . . . 30
1.1.8 AcroForm `/SigFlags`: Document-Level Signature Metadata . . . 31
1.1.9 PDFBox API Mapping . . . 31
1.1.10 Common Mistakes and How to Avoid Them . . . 33
1.1.11 Key Points . . . 34
1.1.12 Specification References . . . 35

2 Byte Range Deep Dive **37**
2.1 The Byte Range Model . . . 37
2.1.1 What the Spec Says . . . 37
2.1.2 Visualizing the Byte Range . . . 38
2.1.3 Why the Gap Exists: The Chicken-and-Egg Problem . . . 38
2.2 The Gap in Detail . . . 39
2.2.1 What's Actually in the Gap . . . 39
2.2.2 The Zero-Padding Question . . . 40
2.3 Incremental Saves and the Byte Range . . . 40
2.3.1 How Signing Creates an Incremental Update . . . 40
2.3.2 What Happens After Signing: The Trailing Bytes Problem . . . 41
2.3.3 Multi-Signature Scenarios . . . 41
2.3.4 Permission-Based Restrictions: What Can Change After Signing? . . . 42
2.4 Size Estimation: How Much Space to Reserve . . . 43
2.4.1 The Problem . . . 43
2.4.2 Estimation Strategies . . . 44
2.4.3 What Happens When the Estimate Is Wrong . . . 44
2.5 Byte Range Verification: How to Check It . . . 45
2.5.1 The Verification Algorithm . . . 45
2.5.2 Implementation in Java . . . 45

2.6 Hex Dump Walkthrough: A Real Signed PDF . . . 47
2.6.1 Finding the Signature Dictionary . . . 47
2.6.2 Examining the Gap Boundaries . . . 47
2.6.3 Verifying No Trailing Bytes . . . 48
2.7 Common Byte Range Problems . . . 48
2.7.1 Problem 1: Trailing Bytes After Signed Region . . . 48
2.7.2 Problem 2: `/Contents` Size Exceeded . . . 48
2.7.3 Problem 3: ByteRange Doesn't Start at Zero . . . 49
2.7.4 Problem 4: Gap Contains More Than Just `/Contents` . . . 49
2.7.5 Problem 5: Hash Algorithm Mismatch . . . 49
2.8 Key Takeaways . . . 49

3 Approval vs. Certification Signatures 51
3.1 The Two Signature Types . . . 51
3.1.1 Approval Signatures . . . 51
3.1.2 Certification Signatures . . . 52
3.2 The DocMDP Permission Model . . . 53
3.2.1 Permission Level 1: No Changes Allowed . . . 53
3.2.2 Permission Level 2: Form Filling and Signing Only . . . 53
3.2.3 Permission Level 3: Form Filling, Signing, and Annotations . . . 53
3.2.4 What "Invalidates the Certification" Actually Means . . . 54
3.3 Lock Dictionaries: Field-Level Protection . . . 54
3.3.1 What Lock Dictionaries Do . . . 54
3.3.2 Lock Actions . . . 54
3.3.3 The `/Fields` Array . . . 55
3.4 FieldMDP: Fine-Grained Modification Detection . . . 55
3.4.1 FieldMDP vs. DocMDP . . . 55
3.5 Interaction Rules: When Multiple Signatures Coexist . . . 56
3.5.1 The Certification + Approval Pattern . . . 56
3.5.2 What Happens When Permissions Conflict . . . 57
3.6 Implementation: Certification Signatures with PDFBox . . . 57
3.6.1 Creating a Certification Signature . . . 57
3.6.2 Adding Lock Dictionaries . . . 58
3.7 Seed Value Dictionaries: Constraining How a Signature Field Is Signed . . . 58
3.7.1 Purpose of Seed Values . . . 58
3.7.2 Key Seed Value Entries . . . 59
3.7.3 Timestamp Seed Values . . . 60
3.8 Real-World Patterns and Mistakes . . . 60
3.8.1 Mistake 1: Using Certification When Approval Is Needed . . . 60
3.8.2 Mistake 2: Not Setting MDP When It's Needed . . . 61
3.8.3 Mistake 3: Lock Dictionary with Wrong Field Names . . . 61
3.8.4 Mistake 4: Assuming All Viewers Enforce MDP . . . 61
3.9 Key Takeaways . . . 61

4 Visual Appearance of Signatures 63
4.1 Appearance Streams: The Basics . . . 63
4.1.1 What an Appearance Stream Is . . . 63
4.1.2 Visible vs. Invisible Signatures . . . 64
4.2 The Layer Model: n0, n2, and n4 . . . 65

4.2.1 Adobe's Legacy Layer System . . . 65
4.2.2 What This Means for Your Implementation . . . 65
4.3 Creating Signature Appearances with PDFBox . . . 65
4.3.1 Basic Text Appearance . . . 65
4.3.2 Appearance with Signature Image . . . 66
4.3.3 Attaching the Appearance to the Signature . . . 67
4.4 Appearance Rendering Across Viewers . . . 68
4.4.1 How Different Viewers Handle Appearances . . . 68
4.4.2 The `/NeedAppearances` Problem . . . 69
4.5 Positioning: Getting the Rectangle Right . . . 69
4.5.1 The Coordinate System . . . 69
4.5.2 Handling Page Rotation . . . 70
4.6 Security Implications of Visual Appearances . . . 70
4.6.1 The Appearance/Content Disconnect . . . 70
4.6.2 Shadow Attacks and Visual Appearances . . . 71
4.7 Common Appearance Problems . . . 71
4.7.1 Problem: Blank Signature Rectangle . . . 71
4.7.2 Problem: Text Appears Clipped or Off-Screen . . . 71
4.7.3 Problem: Font Not Rendering . . . 72
4.8 Key Takeaways . . . 72

5 Signature Validation **73**
5.1 The Five Steps of Signature Validation . . . 73
5.1.1 Step 1: Structural Integrity (Byte Range) . . . 73
5.1.2 Step 2: Hash Verification (Digest Match) . . . 74
5.1.3 Step 3: Cryptographic Verification (CMS Signature) . . . 74
5.1.4 Step 4: Certificate Chain Validation . . . 75
5.1.5 Step 5: Trust Evaluation . . . 76
5.2 Validation Differences Across Viewers . . . 77
5.3 Debugging Validation Failures . . . 77
5.3.1 The Systematic Approach . . . 77
5.3.2 Common Error Messages and Their Meaning . . . 78
5.4 Timestamp Verification . . . 79
5.4.1 Why Timestamps Matter for Validation . . . 79
5.4.2 Verifying the Timestamp . . . 79
5.4.3 Timestamp Chain of Trust . . . 80
5.5 Building a Complete Validation Pipeline . . . 80
5.5.1 Architecture . . . 80
5.5.2 When to Use EU DSS Instead . . . 80
5.6 Key Takeaways . . . 81

6 PAdES Profiles (ETSI EN 319 142) **83**
6.1 The Four PAdES Baseline Profiles . . . 83
6.1.1 The Progression: Why Four Levels? . . . 84
6.2 PAdES-B-B: The Baseline Signature . . . 84
6.2.1 Requirements . . . 84
6.2.2 The ESS-signing-certificate-v2 Attribute . . . 85
6.2.3 SubFilter: `ETSI.CAdES.detached` vs. `adbe.pkcs7.detached` . . . 85
6.3 PAdES-B-T: Adding a Timestamp . . . 86

6.3.1 What B-T Adds . . . 86
6.3.2 How Timestamping Works . . . 86
6.3.3 Choosing a TSA . . . 87
6.4 PAdES-B-LT: Long-Term Data . . . 87
6.4.1 What B-LT Adds . . . 87
6.4.2 The DSS Dictionary . . . 87
6.4.3 Why B-LT Matters . . . 88
6.5 PAdES-B-LTA: Archival Timestamps . . . 88
6.5.1 What B-LTA Adds . . . 88
6.5.2 The Archival Chain . . . 89
6.5.3 Re-Timestamping . . . 89
6.6 Profile Comparison Matrix . . . 89
6.7 Migrating Between Profiles . . . 90
6.8 Key Takeaways . . . 90

7 Timestamps & Long-Term Validation 91
7.1 RFC 3161: The Timestamp Protocol . . . 91
7.1.1 What a Timestamp Proves . . . 91
7.1.2 The Timestamp Request/Response Flow . . . 91
7.1.3 Implementation with BouncyCastle . . . 92
7.1.4 TSA Selection Criteria . . . 93
7.2 Signature Timestamps vs. Document Timestamps . . . 94
7.2.1 Signature Timestamps . . . 94
7.2.2 Document Timestamps . . . 94
7.3 The Document Security Store (DSS) . . . 95
7.3.1 Purpose . . . 95
7.3.2 Building the DSS . . . 95
7.3.3 VRI: Validation Related Information . . . 96
7.4 Fetching Revocation Data . . . 97
7.4.1 OCSP (Online Certificate Status Protocol) . . . 97
7.4.2 CRL (Certificate Revocation List) . . . 98
7.5 The Complete B-LTA Workflow . . . 99
7.6 Key Takeaways . . . 100

8 Security, Incidents & Outlook 101
8.1 PDF Signature Attacks: A Complete Technical Analysis . . . 101
8.1.1 The Researchers . . . 101
8.1.2 Before Bochum: The State of PDF Signature Validation . . . 102
8.1.3 Attack Class 1: Universal Signature Forgery (USF) . . . 103
8.1.4 Attack Class 2: Incremental Saving Attack (ISA) . . . 107
8.1.5 Attack Class 3: Signature Wrapping Attack (SWA) . . . 112
8.1.6 Attack Class 4: Shadow Attacks (2021) . . . 115
8.1.7 Pre-Bochum Vulnerabilities: Earlier PDF Signature Issues . . . 119
8.1.8 Exploits in the Wild . . . 122
8.1.9 The Patch Response: What Changed . . . 123
8.1.10 Building a Robust Validator: Lessons from the Attacks . . . 124
8.1.11 A Note on Responsible Disclosure . . . 125
8.1.12 Summary . . . 126

9 Code, Tools & Implementation **127**

10 The Java PDF Library Landscape **129**
10.1 The iText Saga: From PhD Project to Corporate Licensing Drama 129
10.1.1 Bruno Lowagie and the Birth of iText (2000–2009) 129
10.1.2 The LGPL-to-AGPL Switch: The Day the Rules Changed (2009) 130
10.1.3 The CMS Structure Problem: Why Old iText Signatures Look Different . 131
10.1.4 iText 5.x, 7.x, and 8.x: The Commercial Evolution 132
10.1.5 The IBM Connection 132
10.2 OpenPDF: The Community Fork That Refuses to Die 133
10.2.1 The Fork (2016) 133
10.2.2 What OpenPDF Is (and Isn't) 133
10.2.3 Should You Use OpenPDF? 134
10.3 Apache PDFBox: The Open-Source Standard 134
10.3.1 Origin Story: Ben Litchfield and the Apache Incubation 134
10.3.2 PDFBox 2.x: The Workhorse (2016–2023) 134
10.3.3 PDFBox 3.x: The Breaking Change (2024) 135
10.3.4 The PDFBox Signing Infrastructure: What's Under the Hood 136
10.4 The Legion of the Bouncy Castle 137
10.4.1 Why BouncyCastle Exists 137
10.4.2 The BouncyCastle Ecosystem 138
10.4.3 BouncyCastle Version History and the `-jdk18on` Suffix 138
10.5 EU DSS: The European Commission's Signing Framework 139
10.5.1 What EU DSS Is 139
10.5.2 Why EU DSS Matters for PDF Signatures 139
10.5.3 EU DSS Architecture 140
10.5.4 EU DSS vs. Rolling Your Own 140
10.5.5 EU DSS Version History 141
10.6 Analysis and Debugging Tools 142
10.6.1 RUPS: The PDF Object Explorer 142
10.6.2 veraPDF: The Compliance Validator 142
10.6.3 QPDF: The Swiss Army Knife 143
10.6.4 MuTool (from MuPDF) 143
10.6.5 OpenSSL: CMS Verification Outside the PDF Context 143
10.6.6 ASN.1 Dump Tools 144
10.7 Choosing the Right Library for Your Project 144
10.7.1 Decision Matrix 144
10.7.2 Our Recommendation 145
10.8 The Ecosystem at a Glance 146
10.9 Key Takeaways 147

0.1 Preface

PDF digital signatures are one of those technologies that seem straightforward until you actually have to implement them. You read the spec, write some code, run a test — and the signature validates in your tool but fails in Adobe Acrobat. Or it works in Acrobat but fails the EU's validation service. Or it works everywhere on Monday and breaks on Friday because a timestamp authority renewed its certificate.

If you've been there, this book is for you.

This is a technical reference for engineers, developers, and security researchers who need to understand PDF digital signatures at the deepest level — from the raw bytes in the file to the legal framework that gives those bytes legal standing across 27 EU member states. Every claim is backed by precise references to ISO 32000-2:2020, the ETSI PAdES standards, relevant RFCs, and the eIDAS regulation. Code examples use Apache PDFBox 3.0.0 and BouncyCastle 1.80 on Java 21, because that's what most of us actually use in production.

0.1.1 Who This Book Is For

You are a software engineer, a security researcher, or a technically curious person who needs to *understand* PDF signatures, not just use them. Maybe you're building a signing service. Maybe you're debugging why a customer's signature shows "invalid" in one viewer but "valid" in another. Maybe you're the person who got handed a government compliance checklist that says "PAdES-B-LTA" and nobody in your organization knows what that means.

We assume you know how to program, that you've encountered public-key cryptography before (even if the details are fuzzy), and that you're comfortable reading data structures. We do *not* assume you know anything about PDF internals, CMS, or the European regulatory landscape. Parts I and II build that foundation from scratch.

0.1.2 What This Book Is Not

This is not a tutorial. There are no "Hello World" signing examples in Chapter 1. If you need to sign a PDF in the next 20 minutes, copy the code from Chapter 28 and come back later to understand why it works.

This is not a product manual. We use PDFBox and BouncyCastle because they're open-source, widely deployed, and well-documented. The concepts apply equally to iText, OpenPDF, or any other library — the PDF specification doesn't care what language you write in.

This is not a legal guide. We explain the eIDAS regulation and its technical requirements in detail, but we are engineers, not lawyers. When legal precision matters — and with qualified electronic signatures, it always does — consult someone with a law degree.

0.1.3 How This Book Is Organized

The book is structured in six parts, each building on the previous:

Part I: Cryptographic Foundations (Chapters 1–6) starts from first principles. If you can explain the difference between RSA and ECDSA, between a certificate and a key pair, between CMS and PKCS#7 — skip ahead. If any of those distinctions are hazy, start here. We cover just

enough cryptography to make the rest of the book make sense, with a focus on the specific algorithms and structures you'll encounter in PDF signatures.

Part II: PDF Internals (Chapters 7–11) dissects the PDF file format itself. You'll learn how objects, cross-reference tables, and incremental updates work — because you cannot understand PDF signatures without understanding incremental saves. You'll also learn about AcroForm fields, annotations, and coordinate systems, because signature fields are form fields, signature appearances are annotations, and the coordinate system *will* trip you up.

Part III: PDF Digital Signatures (Chapters 12–18) is the heart of the book. This is where we take apart the `/Sig` dictionary key by key, trace the byte range byte by byte, and walk through the CMS structure field by field. We cover approval and certification signatures, visual appearances, validation logic, PAdES profiles, and long-term validation. If you only read one part of this book, read this one.

Part IV: eIDAS & Qualified Signatures (Chapters 19–23) enters the regulatory world. The European eIDAS regulation created a legal framework where a digital signature can have the same legal standing as a handwritten one — but only if it meets very specific technical requirements. We explain what those requirements are, how the EU Trusted List works, what a Qualified Trust Service Provider does, and how remote signing via the Cloud Signature Consortium API fits into the picture.

Part V: Security, Incidents & Outlook (Chapters 24–27) covers what can go wrong — and what has gone wrong. The Ruhr University Bochum research team published devastating attacks against PDF signature validation in 2019–2021. We analyze every attack technically, explain why it worked, and show how to defend against it. We also cover certificate authority breaches, industry oddities, and the emerging EUDI wallet.

Part VI: Code & Implementation (Chapters 28–31) is where theory meets production. Complete, tested Java code for signing, CMS construction, verification, and analysis — not snippets, but full classes you can drop into a project. We also provide test cases and reference PDFs for every scenario discussed in the book.

0.1.4 Conventions Used in This Book

PDF dictionary keys are written in monospace with a leading slash: `/SubFilter`, `/ByteRange`, `/Contents`. When we show a full dictionary, it's presented as a code listing with callout annotations explaining each key.

Spec references follow this pattern: "ISO 32000-2:2020 §12.8.1" means ISO 32000-2:2020, section 12.8.1. RFC references: "RFC 5652 — Cryptographic Message Syntax (CMS) §5" means RFC 5652, section 5. We use these abbreviations consistently throughout.

Code examples are Java 21 using PDFBox 3.0.0 and BouncyCastle 1.80. Every code listing compiles and runs — the source is in the book's repository alongside test cases that verify the examples.

Admonition blocks carry specific meaning:

Additional context, background, or alternative approaches.

Practical advice from experience — the kind of thing a senior colleague would tell you over coffee.

Something you must understand to avoid a specific class of bugs.

A common mistake or misunderstanding that leads to broken signatures.

A security-relevant issue that could lead to signature bypasses or compliance failures.

0.1.5 A Note on the PDF Specification

The PDF specification has a complicated history. Adobe published it as a proprietary format in 1993, released it as ISO 32000-1 in 2008, and then ISO 32000-2:2020 superseded it in 2020 with significant changes to the digital signature sections.

Throughout this book, we reference ISO 32000-2:2020 as the authoritative source. Where the behavior of older PDF versions matters (and for digital signatures, it often does — plenty of documents in the wild still use PDF 1.7 features), we note the differences explicitly.

The PAdES standards (ETSI EN 319 142) add an additional layer of requirements on top of ISO 32000-2. They don't replace the PDF spec — they constrain it. A PAdES-compliant signature is always a valid ISO 32000-2 signature, but not every valid ISO 32000-2 signature is PAdES-compliant. This distinction matters enormously in the EU, and increasingly, everywhere else.

0.1.6 Three Decades of PDF: A Timeline

PDF was born as a page description language for printing. Digital signatures were an afterthought — bolted on six years later in PDF 1.3, using a cryptographic framework (PKCS#7) that predated PDF itself. Understanding why PDF signatures work the way they do — and why they break the way they break — requires understanding the history that shaped them.

The following table traces every major milestone in the evolution of PDF and, specifically, the digital signature infrastructure built on top of it. Some of these are format changes. Some are regulatory events. Some are security incidents that forced the entire industry to rethink assumptions it had held for years. All of them left marks on the code you will write or debug.

Table 1: The evolution of PDF and PDF digital signatures, 1993–2026

Year	Event	What happened and why it matters
1993	**PDF 1.0** released (Acrobat 1.0)	Adobe published the Portable Document Format as a proprietary specification for reliable cross-platform document rendering. No forms, no interactivity, no signatures — just pages, text, images, and fonts. The file structure established here (header, body, cross-reference table, trailer) survives unchanged to this day. Every PDF you sign in 2026 still starts with `%PDF-1.x` or `%PDF-2.0` and ends with `%%EOF`.

Year	Event	What happened and why it matters
1996	**PDF 1.2** (Acrobat 3.0)	Introduced **AcroForm** — interactive form fields that users could fill in. This matters for signatures because signature fields *are* form fields. The `/AcroForm` dictionary, the field tree, widget annotations — all of the infrastructure that Chapter 9 dissects — was born here. PDF 1.2 also added Unicode support and Flate compression, both of which are ubiquitous in modern PDFs.
1999	**PDF 1.3** — digital signatures introduced (Acrobat 4.0)	The moment PDF gained the ability to carry cryptographic signatures. Adobe defined the `/Sig` dictionary type, the `/ByteRange` array, the `/Contents` hex string for the signature value, and the `/SubFilter` key with its initial value `adbe.pkcs7.detached`. The design decision to embed the CMS signature *inside* the PDF file (rather than as a detached sidecar) and to define the signed region as a byte range with a gap for the signature value itself — these were architectural choices made in 1999 that we still live with. Every quirk in Chapter 12 traces back to this release.
2001	**PDF 1.4** (Acrobat 5.0)	Added **transparency** (the imaging model's biggest change ever), 128-bit RC4 encryption, and **tagged PDF** for accessibility. For signatures: PDF 1.4 formalized the `adbe.pkcs7.sha1` SubFilter (where the SHA-1 hash of the entire document is embedded as the CMS content, rather than using a detached signature). This SubFilter is now deprecated but still appears in legacy documents. Acrobat 5 also introduced **certification signatures** (`/DocMDP` transform) — the distinction between "I approve this document" and "I certify this is the authoritative version" starts here.
2003	**PDF 1.5** (Acrobat 6.0)	Introduced **object streams** and **cross-reference streams** — a complete reworking of how PDF files store their internal structure. Instead of the classic ASCII cross-reference table, PDF 1.5 allows binary cross-reference data compressed inside stream objects. This matters for signatures because the byte range must cover the *entire* file structure, and cross-reference streams change where objects physically live in the file. PDF 1.5 also added **XFA forms** (Adobe's XML-based form format), which would become a persistent source of interoperability problems for signatures on XFA documents — a problem so annoying that ISO 32000-2:2020 deprecated XFA entirely.

Year	Event	What happened and why it matters
2005	**PDF 1.6** (Acrobat 7.0)	**AES-128 encryption** replaced the weak RC4-based encryption from earlier versions. Embedded file streams (attachments) and 3D content were added. For signatures: PDF 1.6 refined the interaction between encryption and signing — a signed, encrypted PDF requires careful handling to ensure the signature covers the encrypted content correctly. The `/NeedAppearances` flag behavior was also clarified, which matters because a missing appearance stream on a signature field causes some viewers to show a blank signature block.
2006	**PDF 1.7** (Acrobat 8.0)	The last version of PDF that Adobe defined unilaterally before handing the specification to ISO. PDF 1.7 solidified the **AES-256 encryption** scheme and finalized the AcroForm/XFA coexistence model. The vast majority of signed PDFs in the wild today are PDF 1.7 documents — not because of any technical limitation of PDF 2.0, but because template authors, form generators, and enterprise software have not yet migrated. When you debug a signature failure in production, you are almost certainly looking at a PDF 1.7 file.
2008	**ISO 32000-1:2008** — PDF becomes an ISO standard	Adobe donated the PDF 1.7 specification to ISO, making it an open international standard. The technical content was essentially identical to Adobe's PDF Reference 1.7, but the governance changed: future changes would go through the ISO committee process, not Adobe's product team. For signatures, this was the moment PDF digital signatures gained the credibility of an ISO-backed specification — a prerequisite for government and regulated-industry adoption.

Year	Event	What happened and why it matters
2009	**PAdES** — ETSI TS 102 778 (Parts 1–5)	The European Telecommunications Standards Institute published the first **PDF Advanced Electronic Signatures** standards. PAdES defined profiles for how CMS signatures should be embedded in PDF to meet European legal requirements: which signed attributes are mandatory (ESS signing-certificate-v2), which SubFilter values to use (`ETSI.CAdES.detached`), and how to embed timestamps, revocation data, and long-term validation information. PAdES didn't replace ISO 32000 — it constrained it. A PAdES-compliant signature is always a valid ISO 32000 signature, but the reverse is not true. This distinction — and the interoperability problems it causes when a PDF viewer validates against ISO 32000 rules but not PAdES rules — is a recurring theme in this book (Chapters 17 and 26).
2011	**DigiNotar breach** — CA infrastructure crisis	A Dutch certificate authority, DigiNotar, was completely compromised by an attacker who issued over 500 fraudulent certificates — including certificates for google.com, mozilla.org, and intelligence agencies. The certificates were used in man-in-the-middle attacks against Iranian dissidents. Every major browser vendor revoked trust in DigiNotar within weeks; the company went bankrupt within months. For PDF signatures: DigiNotar was a wake-up call about the fragility of the certificate trust model that all digital signatures depend on. If a CA can be compromised, any signature issued by that CA's certificates becomes suspect. This incident accelerated the adoption of Certificate Transparency logs and tighter CA audit requirements — both of which affected the trust infrastructure behind PDF signatures. Chapter 25 covers this and other CA breaches in detail.

Year	Event	What happened and why it matters
2012	**Adobe Approved Trust List (AATL)** formalized	Adobe formalized its **AATL** program — a curated list of root certificates that Acrobat trusts for signature validation. If your signing certificate chains up to an AATL root, Acrobat shows a green checkmark. If it doesn't — even if the certificate is perfectly valid, issued by a reputable CA, and chains to a trust anchor in your OS trust store — Acrobat shows a warning. This created a de facto monopoly on "valid" PDF signatures: to get a green checkmark in the world's dominant PDF viewer, you need an AATL-listed certificate, which costs significantly more than a standard code-signing certificate. The AATL remains one of the most contentious aspects of the PDF signature ecosystem (Chapter 26).
2014	**eIDAS Regulation** (EU 910/2014) enacted	The European Union passed the **Electronic Identification, Authentication and Trust Services** regulation, creating a legal framework where digital signatures can have the same legal standing as handwritten signatures — but only if they meet strict technical requirements. eIDAS defined three tiers: **electronic signatures** (any electronic indication of signing intent), **advanced electronic signatures** (uniquely linked to the signatory, capable of identifying them, under their sole control, linked to the data so any change is detectable), and **qualified electronic signatures** (advanced + created by a qualified device + based on a qualified certificate issued by a qualified trust service provider). Only QES has automatic legal equivalence to a handwritten signature across all EU member states. PAdES became the primary technical mechanism for creating eIDAS-compliant signatures in PDF. Chapters 19–23 cover the entire eIDAS ecosystem.

Year	Event	What happened and why it matters
2016	**eIDAS fully effective** (July 1, 2016)	The eIDAS regulation became fully effective across all EU member states, with the **EU Trusted List (EUTL)** operational. The EUTL is a machine-readable, cryptographically signed list of every Qualified Trust Service Provider in the EU — the authoritative source for determining whether a signing certificate qualifies for the highest legal tier. For developers: if your application needs to validate whether a PDF signature is a qualified electronic signature, you must check the signing certificate against the EUTL, not just against a generic trust store. The EUTL XML schema, its signature validation, and the LOT (List of Trusted Lists) hierarchy are covered in Chapter 20.
2017	**ISO 32000-2:2017** — PDF 2.0, first edition	The first major revision of the PDF specification under ISO governance. PDF 2.0 deprecated XFA forms, removed several legacy features, and made significant changes to the digital signature sections. Notably: the `adbe.pkcs7.sha1` SubFilter was deprecated (SHA-1 is cryptographically broken for collision resistance since the SHAttered attack of February 2017), and the specification added the `ETSI.RFC3161` SubFilter for document-level timestamps. PDF 2.0 also clarified the byte range coverage rules and tightened the requirements around incremental updates after signing — changes that were partly motivated by early reports of the signature validation problems that Bochum would systematically expose two years later.
2019	**"1 Trillion Dollar Refund"** — Bochum attacks (CCS 2019)	The Ruhr University Bochum team published their landmark paper at ACM CCS 2019, systematically breaking PDF signature validation in every major viewer. They identified three attack classes: **Universal Signature Forgery (USF)** — crafting a signed PDF with a garbage or empty signature value that viewers accepted as valid; **Incremental Saving Attack (ISA)** — appending unsigned changes to a signed PDF that viewers failed to flag; and **Signature Wrapping Attack (SWA)** — relocating the signature's byte range to point at attacker-controlled data. Of 22 desktop viewers tested, all were vulnerable to at least one attack. The paper title referenced the fact that signed PDFs are used for tax refund claims, insurance payouts, and financial contracts — if signatures could be forged, the financial exposure was enormous. This research is the subject of Chapter 24.

Year	Event	What happened and why it matters
2020	**ISO 32000-2:2020** — PDF 2.0, second edition	The second edition of PDF 2.0 incorporated corrections and clarifications, including tightened language around signature validation motivated by the Bochum attacks. In the same year, Adobe issued **CVE-2020-9592** and **CVE-2020-9596** and patched Acrobat against USF, ISA, and SWA. Foxit, Nitro, LibreOffice, Okular, and every other major viewer released their own patches. The PAdES standards were also updated (ETSI EN 319 142-1 V1.1.1) to align with the second edition of ISO 32000-2.
2021	**Shadow Attacks** (NDSS 2021)	The Bochum team published a follow-up: **Shadow Attacks**. Unlike the 2019 attacks (which modified a PDF *after* signing), Shadow Attacks prepared malicious content *before* the document was signed. The attacker created a PDF with hidden content (invisible text, overlapping images, replaced form fields) that the signer could not see during signing — but that became visible when the recipient opened the document. The signature validated correctly because the signed bytes were genuinely signed by the legitimate signer. The attack exploited the gap between what the signer *saw* and what the bytes *contained* — a fundamental problem in any visual-document signing model. Three variants were demonstrated: **Hide** (hiding content behind overlapping elements), **Replace** (swapping visible content via interactive form features), and **Hide-and-Replace** (combining both). Again, all major viewers were vulnerable.
2022	**ETSI EN 319 142-1 V1.2.1** — PAdES update	ETSI updated the PAdES standard to align with ISO 32000-2:2020 and to incorporate lessons from the Bochum attacks. The update clarified requirements around signature scope validation, incremental update handling, and the relationship between the byte range and the visible document content. It also formalized the PAdES-B-LTA profile's requirements for archival timestamps in the DSS (Document Security Store) — the mechanism that enables long-term validation decades after the signing certificate has expired.

Year	Event	What happened and why it matters
2024	**eIDAS 2.0** (EU 2024/1183) enacted	The EU adopted the revised eIDAS regulation, which introduced the **European Digital Identity (EUDI) Wallet** — a smartphone-based identity wallet that every EU member state must offer to its citizens by 2026. The EUDI wallet will carry government-issued identity attestations (PID — Person Identification Data) and qualified electronic attestations of attributes (QEAA), enabling citizens to sign documents with qualified electronic signatures from their phone. This represents the most ambitious attempt to make QES mainstream — moving it from specialized HSM-backed signing services to consumer smartphones. The Architecture Reference Framework (ARF) defines the technical specifications. Chapter 27 covers the EUDI wallet and its impact on PDF signing.
2025–26	**EUDI wallet pilots & deployment**	EU member states are conducting large-scale pilot programs for the EUDI wallet across sectors: banking (identity verification), travel (digital travel credentials), education (diplomas), healthcare (prescriptions), and government services. The pilots test the wallet's signing capabilities, including PAdES-B-LTA signature creation from mobile devices, remote QSCD integration, and cross-border signature validation. The Cloud Signature Consortium (CSC) API — the standard interface for remote signing — is being adapted for wallet integration. The outcome of these pilots will determine whether QES becomes a mainstream consumer technology or remains a specialized enterprise tool.

The table omits minor point releases and focuses on the events that changed what developers need to know. If you are working with PDF signatures today, every row in that table has left its fingerprint on the code you are writing — whether it's the byte range gap from 1999, the PAdES constraints from 2009, the trust model questions raised by DigiNotar in 2011, or the validation rewrites forced by Bochum in 2019–2021.

0.1.7 Acknowledgments

This book would not exist without the open-source communities that make PDF signature work accessible to everyone. The **Apache PDFBox** team — particularly Andreas Lehmkühler, Tilman Hausherr, and the many contributors who maintain the signing infrastructure — have built the most widely used open-source PDF library in the Java ecosystem. The **Legion of the Bouncy Castle** (yes, that's their actual name) gave us BouncyCastle, which provides the CMS and X.509 implementations that most Java signing code depends on, whether it knows it or not.

Special acknowledgment goes to the security researchers at **Ruhr University Bochum** — Vladislav Mladenov, Christian Mainka, Karsten Meyer zu Selhausen, Martin Grothe, and Jörg Schwenk — whose systematic analysis of PDF signature validation between 2019 and 2021 exposed fundamental flaws in how every major PDF viewer handled signatures. Their papers — *"1 Trillion Dollar Refund: How To Spoof PDF Signatures"* (CCS 2019), *"Shadow Attacks: Hiding and Replacing Content in Signed PDFs"* (NDSS 2021), and several follow-ups — demonstrated that attackers could forge signatures, hide content changes behind valid signatures, and bypass validation in Adobe Acrobat, Foxit Reader, and dozens of other viewers. The industry patched, but the research fundamentally changed how we think about PDF signature security. Their work is cited throughout this book, and Chapter 24 is dedicated to a technical analysis of their attacks.

What they found was this: PDF viewers did not actually verify what they claimed to verify. The Bochum team identified three major attack classes. **Universal Signature Forgery (USF)** showed that some viewers only checked whether a signature *existed* — not whether its cryptographic value was correct. An attacker could craft a PDF with an empty or garbage `/Contents` value, and viewers like Adobe Acrobat DC (before the patch) would display it as "signed and valid." **Incremental Saving Attack (ISA)** exploited the incremental update mechanism that PDF signatures depend on: an attacker could append a new revision to a signed PDF — changing text, adding pages, replacing content entirely — and because viewers did not properly check whether the appended changes fell outside the signed byte range, the signature still showed as valid. **Signature Wrapping Attack (SWA)** relocated the signed content within the file so that the byte range pointed at attacker-controlled data while the original signature remained structurally present. The follow-up **Shadow Attacks** paper (NDSS 2021) went further: an attacker could prepare a PDF *before* it was signed, embedding hidden content that would only become visible after signing — the signer would see one document, but the recipient would see another, and the signature would validate on both views.

The team tested 22 desktop PDF viewers and 7 online validation services. Every single one was vulnerable to at least one attack class. Adobe issued CVE-2020-9592 and CVE-2020-9596 and patched Acrobat in May 2020. Foxit, Nitro, LibreOffice, and every other major viewer followed with their own patches. The research forced a fundamental rethinking of PDF signature validation — in particular, how viewers handle incremental updates after signing, how byte range coverage is verified, and how the relationship between the CMS signature and the actual document bytes is validated. Before Bochum, most viewers treated signature validation as a checkbox. After Bochum, it became a security-critical code path that required adversarial thinking.

The **European Telecommunications Standards Institute (ETSI)** deserves credit for the PAdES standards that brought PDF signatures into the regulated world. Whatever one may think about the complexity of ETSI EN 319 142, it solved the real problem of making PDF signatures legally interoperable across borders.

And the PDF specification itself is the product of hundreds of engineers at Adobe, ISO, and ETSI who somehow managed to build a digital signature framework into a page description language originally designed for printing. That it works as well as it does is a testament to their effort. That it has the quirks it has is a testament to the fact that three decades of backward compatibility leaves marks.

Chapter 1

PDF Digital Signatures

1.1 Signature Architecture

This chapter dissects the internal architecture of PDF digital signatures as specified in ISO 32000-2:2020 §12.8. We examine every dictionary, every key, and every byte — from the AcroForm field tree down to the raw CMS `SignedData` blob embedded in `/Contents`.

But more importantly, we talk about what actually matters when you're staring at a broken signature at 2 AM, wondering why Adobe says "invalid" while your code says "fine."

1.1.1 Overview: What a PDF Signature Actually Is

Here's the thing most people get wrong about PDF signatures: they think of them as something *attached to* a document, like a wax seal on an envelope. They're not.

A PDF digital signature is a set of **PDF objects embedded within the document itself**. The signature *is part of the document*. This seemingly minor architectural choice has profound consequences — it's why PDF signatures are simultaneously powerful and maddening, why they survive copy operations but break on re-saves, and why you can have a perfectly valid signature on a document that's been modified after signing (yes, really).

The four pieces of a PDF signature:

1. A **signature field** (`PDSignatureField`) in the AcroForm field tree — the form field that "holds" the signature
2. A **signature dictionary** (`/Type /Sig`) — the actual cryptographic payload and metadata
3. A **widget annotation** — the visual rectangle on a page (may be zero-size for invisible signatures)
4. An **appearance stream** (`/AP`) — what the viewer renders in the signature rectangle

The signature field is a form field like any other (`/FT /Sig`). The signature dictionary is the *value* of that field (`/V`). An unsigned field has `/V` absent or null — the field exists but has no signature value yet.

Think of it like an empty checkbox on a paper form. The box is there, waiting to be checked. A document author can place signature fields in a PDF template, and signers fill them in later — each one independently, at different times, possibly on different continents.

When you encounter a PDF with signature problems, start here. Open the document in a COS-level viewer (like PDFalyzer, or even a hex editor) and check:

1. Does the AcroForm exist? (Some PDFs have signature fields floating without an AcroForm — instant headache.)
2. Does `/SigFlags` include bit 2 (AppendOnly)? If not, any editor can destroy all signatures with a simple save.
3. Is the `/V` entry present on the signature field? If absent, the field is unsigned.
4. Does the widget actually have a page reference? Some tools create orphaned widgets.

This five-minute checklist saves hours of debugging.

1.1.2 Signature Fields in the AcroForm Tree

> A signature field (field type Sig) shall represent a digital signature that may or may not have a signature value.
>
> — ISO 32000-2:2020 §12.7.4.5

Signature fields live in the document's interactive form structure:

1.1.2.1 The SigFlags Trap

The `/SigFlags` entry is one of those things that's easy to overlook and devastating to get wrong.

Table 1.1: SigFlags — ISO 32000-2:2020 §12.7.2, Table 225

Name	Bit	Req	Spec Ref	Effect
`SignaturesExist`	1	Required	§12.7.2	Tells the viewer "this document contains at least one signature field." Conforming readers **shall not** allow saving in a way that invalidates existing signatures.
`AppendOnly`	2	Required	§12.7.4.5	The document **shall** be saved using incremental updates only. This is the mechanism that protects existing signatures from being destroyed by a full rewrite.

Setting `/SigFlags` to 3 (both bits) is mandatory when a document contains signatures. If bit 2 is missing, a naive PDF editor may do a full save and destroy all signatures. Many real-world PDFs get this wrong.

Here's what happens in practice: someone creates a PDF with LibreOffice, adds a signature field, signs it — and LibreOffice doesn't set bit 2. The document gets emailed to a colleague who opens it in some random PDF editor to add an annotation, hits save, and the editor does a full rewrite. Every signature is now invalid. The colleague has no idea what happened. The signer gets an angry phone call.

The fix is always the same: if you're building a tool that creates signature fields, set `/SigFlags` to 3 the moment any field has a value. And if you're building a tool that edits PDFs,

check for `/SigFlags` bit 2 before saving — if it's set, you *must* use incremental save. If your tool doesn't support incremental save... well, you need to support it now. Welcome to PDF.

1.1.2.2 Field Dictionary Structure

A signature field's COS dictionary looks like this. Every key matters, and I'll explain why:

Here is the complete COS dictionary of a typical signature field. In most PDFs, the field and its widget annotation are merged into a single object — the spec (ISO 32000-2:2020 §12.5.6.19) allows splitting them, but few tools bother:

```
<<
  /Type /Annot
  /Subtype /Widget
  /FT /Sig
  /T (Signature1)
  /V 42 0 R
  /Rect [100 500 300 550]
  /AP << /N 43 0 R >>
  /P 7 0 R
  /Lock <<
    /Type /SigFieldLock
    /Action /All
  >>
  /SV <<
    /Type /SV
    /Filter /Adobe.PPKLite
    /SubFilter [/adbe.pkcs7.detached /ETSI.CAdES.detached]
    /DigestMethod [/SHA256 /SHA384 /SHA512]
    /Reasons [(Approved)(Reviewed)]
    /MDP << /P 2 >>
  >>
>>
```

The keys fall into three groups: annotation/widget keys (how it looks and where it lives), signature field keys (what it is and what it points to), and sub-dictionaries that constrain signing behavior.

Table 1.2: Annotation/Widget Keys — ISO 32000-2:2020 §12.5.6.19

Key	Type	Req	Spec Ref	Description
`/Type`	name	Optional	§12.5.6	Always `/Annot` for the merged field+widget.
`/Subtype`	name	Required	§12.5.6	Always `/Widget` — this is a widget annotation.
`/Rect`	rectangle	Required	§12.5.6	Widget rectangle in page coordinates: `[llx lly urx ury]`. If `[0 0 0 0]`, the signature is invisible.

Key	Type	Req	Spec Ref	Description
`/P`	reference	Optional	§12.5.6	Page reference — which page this widget is on. Some poorly generated PDFs omit this, and then viewers have to guess. Don't be that tool.
`/AP`	dictionary	Optional	§12.5.6.19	Appearance stream dictionary. `/N` is the normal appearance — what the viewer draws. Without this, many viewers show a generic placeholder. Adobe is particularly opinionated about appearances.

Table 1.3: Signature Field Keys — ISO 32000-2:2020 §12.7.5.5

Key	Type	Req	Spec Ref
`/FT`	name	Required	§12.7.4.1
Field type — `/Sig` marks this as a signature field. Non-negotiable. If you see `/FT /Tx` on something that's supposed to be a signature, someone made a mistake.			
`/T`	text string	Required	§12.7.4.1
Partial field name — used to identify the field programmatically. This is how your code finds "the signature field called Signature1."			
`/V`	reference	Optional	§12.7.5.5
Reference to the **signature dictionary** (`/Type /Sig`) — the cryptographic payload. If absent, the field is unsigned. When present, this is the key entry that connects the form field to the actual signature. The signature dictionary is detailed in The Signature Dictionary ().			

The remaining two keys — `/Lock` and `/SV` — are sub-dictionaries that deserve their own tables.

Table 1.4: Lock Dictionary (`/Type /SigFieldLock`) — ISO 32000-2:2020 §12.7.5.5, Table 237

Key	Type	Req	Spec Ref
`/Type`	name	Optional	§12.7.5.5
Always `/SigFieldLock`.			
`/Action`	name	Required	§12.7.5.5, Table 237
What to lock after signing: `/All` (lock everything), `/Include` (lock fields listed in `/Fields`), or `/Exclude` (lock everything *except* fields listed in `/Fields`). This is how approval signatures can lock specific fields — see ???.			
`/Fields`	array	Conditional	§12.7.5.5, Table 237

Key	Type	Req	Spec Ref
Array of field names to include or exclude from locking. Required when `/Action` is `/Include` or `/Exclude`.			

1.1.2.3 Seed Value Dictionary (`/SV`): The Unsung Hero

> The seed value dictionary provides constraint information that shall be used at the time the signature is applied.
>
> — ISO 32000-2:2020 §12.7.5.5

The seed value dictionary is one of the most powerful and underused features in the entire PDF signature specification. Most people don't even know it exists, which is a shame, because it solves a problem that plagues every organization deploying PDF signatures at scale: **how do you ensure signers use the right algorithm, the right format, and the right handler?**

Table 1.5: Seed Value Dictionary (`/Type /SV`) — ISO 32000-2:2020 §12.7.5.5, Table 238

Key	Type	Req	Spec Ref
`/Filter`	name	Optional	§12.7.5.5, Table 238
Required signature handler (e.g., `/Adobe.PPKLite`). Use this to ensure your documents work with specific software.			
`/SubFilter`	array of names	Optional	§12.7.5.5, Table 238
Acceptable signature formats. Set this to `[/ETSI.CAdES.detached]` if you need PAdES compliance. The signer's software **must** use one of these formats.			
`/DigestMethod`	array of names	Optional	§12.7.5.5, Table 238
Acceptable hash algorithms. Set this to `[/SHA256 /SHA384 /SHA512]` — and for the love of all that is holy, **do not include `/SHA1`**. It's 2026. SHA-1 has been broken since 2017.			
`/Reasons`	array of strings	Optional	§12.7.5.5, Table 238
Pre-defined reason strings the signer must choose from. Great for audit trails — instead of "asdf" in the reason field, signers pick from "Approved", "Reviewed", "Authorized".			
`/LegalAttestation`	array of strings	Optional	§12.7.5.5, Table 238
Legal attestation strings — rarely used, but available if you need legal boilerplate attached to the signature.			
`/Ff`	integer	Optional	§12.7.5.5, Table 238
Flags indicating which seed values are **required** (vs. recommended). If bit 1 is set on a constraint, it's mandatory — the signer's software must enforce it, not just suggest it.			

The `/MDP` and `/TimeStamp` entries are sub-dictionaries with their own structure:

Table 1.6: Seed Value MDP Dictionary (`/MDP`) — ISO 32000-2:2020 §12.7.5.5

Key	Type	Req	Spec Ref	Description
`/P`	integer	Required	§12.7.5.5	The DocMDP permission level this signature must enforce: 1 (no changes), 2 (form fills + signatures), 3 (form fills + signatures + annotations). This is how a template author says "whoever signs this field must also lock the document."

Table 1.7: Seed Value TimeStamp Dictionary (`/TimeStamp`) — ISO 32000-2:2020 §12.7.5.5

Key	Type	Req	Spec Ref	Description
`/URL`	ASCII string	Required	§12.7.5.5	URL of the RFC 3161 timestamp server.
`/Ff`	integer	Optional	§12.7.5.5	If bit 1 is set, timestamping is required (not just recommended). If you need PAdES-B-T, set this. The signer's software will contact the TSA automatically.

Practical advice for template authors:

If you're creating PDF templates that others will sign, use seed values aggressively. Don't trust signers to pick the right options — they won't. A financial services company once sent us PDFs where half the signatures used SHA-1 because some signers had old software and nobody constrained the algorithm. Those signatures were technically valid but would fail compliance audits.

Set `/DigestMethod` to `[/SHA256]`, set `/SubFilter` to `[/ETSI.CAdES.detached]`, and set the `/Ff` flags to make them mandatory. Your future self will thank you.

1.1.3 The Signature Dictionary (`/Type /Sig`)

The signature dictionary is the heart of a PDF digital signature. It's referenced from the field's `/V` entry, and it's where the real action is. Everything else — the field, the widget, the appearance — is just scaffolding. This dictionary is where the cryptographic proof lives.

Here is the complete structure of a typical signature dictionary — every key you'll encounter in production:

```
<<
  /Type /Sig
  /Filter /Adobe.PPKLite
  /SubFilter /ETSI.CAdES.detached
```

```
  /ByteRange [0 15000 15500 34500]
  /Contents <308206A7...00000000>
  /M (D:20260306120000+00'00')
  /Name (John Doe)
  /Location (Vienna, Austria)
  /Reason (Approved)
  /ContactInfo (john@example.com)
  /Reference [
    <<
      /Type /SigRef
      /TransformMethod /DocMDP
      /TransformParams <<
        /Type /TransformParams
        /P 2
        /V /1.2
      >>
      /DigestMethod /SHA256
    >>
  ]
  /Prop_Build <<
    /App << /Name /PDFalyzer >>
    /Filter << /Name /Adobe.PPKLite >>
  >>
>>
```

The table below dissects each key. The core cryptographic keys come first, then metadata, then sub-dictionaries.

1.1.3.1 Core Signature Keys

These five keys define the cryptographic signature itself. Get any of them wrong and the signature is broken.

Table 1.8: Core keys of the Signature Dictionary (`/Type /Sig`) — ISO 32000-2:2020 §12.8.1, Table 255

Key	Type	Req	Spec Ref
`/Type`	name	Optional	§12.8.1
Always `/Sig` when present. Identifies this dictionary as a signature value. Optional per the spec, but always include it — defensive parsers rely on it.			
`/Filter`	name	Required	§12.8.1, Table 255
The signature handler name. In theory, this identifies which software can process the signature. In practice, it's always `/Adobe.PPKLite`. Always. Even when the signing software has nothing to do with Adobe. The name stuck because Adobe defined the original signature spec in 1999 and everyone just... went with it.			
`/SubFilter`	name	Required	§12.8.1, Table 255

Key	Type	Req	Spec Ref
The actual encoding format of the signature value. *This* is the key that determines how `/Contents` is structured — see SubFilter Values: A Brief History of Getting It Right for the full history and current recommendation.			
`/ByteRange`	array	Required	§12.8.1, Table 255
Four integers `[offset1 length1 offset2 length2]` defining which bytes of the file are signed. The gap between the two ranges is where `/Contents` lives. The most important array in the entire dictionary — a deep dive is in ???.			
`/Contents`	string	Required	§12.8.1, Table 255
Hex-encoded CMS `SignedData` blob (RFC 5652 — Cryptographic Message Syntax (CMS) §5), padded with zeros. This is the actual cryptographic signature. Everything else in this dictionary is metadata. This is the payload. See The Value: CMS SignedData.			

1.1.3.2 Metadata Keys

These keys carry informational data about the signing event. They are part of the signed bytes and cannot be changed after signing — but they are set by whoever controls the signing process, **not verified against any external authority**.

Table 1.9: Metadata keys of the Signature Dictionary — ISO 32000-2:2020 §12.8.1, Table 255

Key	Type	Req	Spec Ref	Description
`/M`	date string	Optional	§12.8.1	Signing time in PDF date format. **Do not trust this.** It's the signer's local clock. Use the CMS timestamp attribute for authoritative time.
`/Name`	text string	Optional	§12.8.1	Signer's name. **Do not trust this either.** It's free text. The authoritative name is in the X.509 certificate inside `/Contents`.
`/Location`	text string	Optional	§12.8.1	Claimed signing location. Informational, unverified.
`/Reason`	text string	Optional	§12.8.1	Claimed signing reason (e.g., "Approved", "Reviewed"). Informational, unverified.
`/ContactInfo`	text string	Optional	§12.8.1	Contact info. Informational, optional, you see the pattern.

The `/M`, `/Name`, `/Location`, `/Reason`, and `/ContactInfo` fields are **unauthenticated metadata**. Yes, they're part of the signed bytes. Yes, they can't be changed after signing. But they're set *before* signing, by whoever controls the signing process. If a malicious signer puts "Barack

Obama" in the `/Name` field and signs with a self-signed certificate, the signature will be structurally valid. **The only trustworthy identity information is the X.509 certificate in `/Contents`.** Never display `/Name` to users as if it were verified identity. This mistake is shockingly common in PDF viewer implementations.

1.1.3.3 Signature Reference Dictionary (`/Reference`)

The `/Reference` entry is an array of **signature reference dictionaries** (`/Type /SigRef`) that define transform methods — rules about what can and cannot change after signing. This is the mechanism behind certification signatures and field-level locking (covered in depth in ???).

Table 1.10: Signature Reference Dictionary (`/Type /SigRef`) — ISO 32000-2:2020 §12.8.1, Table 256

Key	Type	Req	Spec Ref
`/Type`	name	Optional	§12.8.1, Table 256
Always `/SigRef`.			
`/TransformMethod`	name	Required	§12.8.1, Table 256
The transform method: `/DocMDP` (document modification detection and prevention), `/FieldMDP` (field-level locking), or `/UR` (usage rights — deprecated in PDF 2.0). DocMDP is by far the most common. See ???.			
`/TransformParams`	dictionary	Optional	§12.8.1, Table 256
Parameters for the transform method. For DocMDP, this contains `/P` (permission level: 1 = no changes, 2 = form fills + signatures, 3 = form fills + signatures + annotations) and `/V` (version).			
`/DigestMethod`	name	Required	§12.8.1, Table 256
Hash algorithm for this reference (e.g., `/SHA256`). Must match the algorithm used in the CMS structure.			

1.1.3.4 Build Properties Dictionary (`/Prop_Build`)

The `/Prop_Build` dictionary identifies the software that created the signature. It's optional and purely informational — but surprisingly useful for debugging.

Table 1.11: Build Properties Dictionary — ISO 32000-2:2020 §12.8.1, Table 258

Key	Type	Req	Spec Ref	Description
`/App`	dictionary	Optional	§12.8.1	Application that created the signature. Contains `/Name` (e.g., `/PDFalyzer`, `/AdobeAcrobatDC`).
`/Filter`	dictionary	Optional	§12.8.1	Signature handler info. Contains `/Name` (e.g., `/Adobe.PPKLite`).

Key	Type	Req	Spec Ref	Description
/PubSec	dictionary	Optional	§12.8.1	Public security handler info. Rarely populated.

The /Prop_Build dictionary is your friend when debugging.

When a signature doesn't validate, the first thing I check is /Prop_Build. It tells you which software created the signature. If it says /iText with a version from 2015, you know you might be dealing with an old CMS structure. If it says /Adobe Acrobat and the signature is invalid, something is genuinely wrong because Adobe's own tool should produce signatures that Adobe validates. If /Prop_Build is missing entirely, the signing software was lazy — which tells you something about the quality of the rest of the signature, too.

1.1.3.5 SubFilter Values: A Brief History of Getting It Right

The /SubFilter value determines how the /Contents bytes are structured. Getting this right is the difference between "signature works everywhere" and "signature works in Adobe but nowhere else" (or vice versa).

The history of SubFilter values is essentially the history of PDF signatures trying — and sometimes failing — to keep up with cryptographic best practices:

The first two SubFilter values are deprecated — they use SHA-1, which has been broken for collision resistance since the SHAttered attack of February 2017. You will encounter them in legacy documents, but you should never use them for new signatures.

Table 1.12: Deprecated SubFilter Values (do not use for new signatures)

SubFilter	Standard	Req	Spec Ref
/adbe.x509.rsa_sha1	PDF 1.3+	Deprecated	ISO 32000-2:2020 §12.8.3.2
Do not use. Raw RSA signature with the certificate in a separate /Cert key instead of inside /Contents. No CMS wrapper. SHA-1 only. This was PDF's first attempt at digital signatures in 1999. It made sense at the time. It doesn't anymore.			
/adbe.pkcs7.sha1	PDF 1.3+	Deprecated	ISO 32000-2:2020 §12.8.3.4
Do not use. CMS SignedData where the SHA-1 digest of the byte range is *encapsulated* in the CMS. This is the worst of both worlds: SHA-1 (broken) and encapsulated content (unnecessary complexity). Nobody misses this format.			

The following three values are the current standard. For new signatures, use /ETSI.CAdES.detached unless you have a specific reason not to.

Table 1.13: Current SubFilter Values

SubFilter	Standard	Req	Spec Ref
`/adbe.pkcs7.detached`	PDF 1.3+	Current	ISO 32000-2:2020 §12.8.3.3
CMS `SignedData` with detached content. The workhorse format — widely supported since PDF 1.3. The signed bytes are the PDF byte range, not embedded in CMS. If you need maximum compatibility with older software, this is your format.			
`/ETSI.CAdES.detached`	PDF 2.0 / PAdES	Modern	ETSI EN 319 142-1 V1.1.1 (2016-04) §5.1
CMS `SignedData` with mandatory CAdES signed attributes, specifically the `signing-certificate-v2` attribute. Required for PAdES-B-B and higher. **This is the modern standard. Use this.** The CMS structure is identical to `/adbe.pkcs7.detached` at the container level — the difference is one signed attribute (~50 bytes).			
`/ETSI.RFC3161`	PDF 2.0 / PAdES	Modern	ETSI EN 319 142-1 V1.1.1 (2016-04) §5.4
RFC 3161 timestamp token — not a signature, but a document-level timestamp. Used for PAdES-B-T and long-term archival. You'll encounter these when dealing with LTV (Long-Term Validation).			

Here's a trap that catches people all the time: Adobe Acrobat and most viewers accept *all* SubFilter values for backward compatibility. Your signature will "work" in Acrobat regardless of which SubFilter you use. But **only** `/ETSI.CAdES.detached` is valid for PAdES-compliant signatures under eIDAS. If you need legal standing in the EU — and increasingly, that's everyone — this is the only acceptable value. Don't let "it works in Acrobat" lull you into using `/adbe.pkcs7.detached` for new signatures. Acrobat is not the arbiter of legal validity; the eIDAS regulation is.

The compatibility question everyone asks:

"Can I use `/ETSI.CAdES.detached` if my users have old PDF readers?"

Yes. The CMS structure is identical to `/adbe.pkcs7.detached` at the container level. Old readers that don't understand the SubFilter name will typically fall back to parsing the `/Contents` as generic CMS/PKCS#7 — and it will work. The difference is one signed attribute (`signing-certificate-v2`), which old readers simply ignore.

The only scenario where it breaks: truly ancient software (pre-2010) that strictly checks the SubFilter name against a hardcoded whitelist and rejects unknown values. If your users are running PDF readers from 2008, they have bigger problems than SubFilter compatibility.

1.1.4 The `/Contents` Value: CMS SignedData

The `/Contents` entry is a **hex-encoded byte string** containing a DER-encoded CMS `SignedData` structure (RFC 5652 — Cryptographic Message Syntax (CMS) §5).

This is where the actual cryptographic signature lives. Everything else in the signature dictionary is metadata. This is the payload.

1.1.4.1 Why Hex-Encoded?

PDF strings can be literal `(…)` or hex `<…>`. The hex form is used because CMS blobs contain arbitrary binary data (DER-encoded ASN.1), and literal strings would need extensive escaping for bytes like `\(`, `\)`, `\\`. Hex encoding doubles the size but is safe and unambiguous.

A typical `/Contents` value looks like `<308206A7…several thousand hex characters…0000000000>`. The leading bytes (`3082…`) are the DER-encoded ASN.1 SEQUENCE tag, and the trailing zeros are padding. Which brings us to one of the more elegant hacks in the PDF specification:

1.1.4.2 Pre-allocation: The Chicken-and-Egg Problem

Here's the fundamental problem with PDF signatures:

1. The signature covers the file's bytes (including the signature dictionary itself)
2. But the signature dictionary must contain the CMS blob
3. But the CMS blob can only be computed *after* you know which bytes to sign
4. But you can't know which bytes to sign until the dictionary is written

The solution is a trick that every PDF signing implementation uses: **pre-allocate** a large enough `/Contents` value filled with zeros, write the file, *then* overwrite the zeros with the actual CMS bytes:

The pre-allocated size must be estimated accurately, and this is where things go wrong in the real world:

- **Too small** — the CMS blob doesn't fit — signing fails. You'll get an exception from PDFBox and no output file. This is the most common "my signing suddenly broke" bug, usually triggered by adding a certificate chain or a timestamp that makes the CMS blob bigger than expected.
- **Too large** — wasted bytes, but the signature is valid. A few KB of zeros at the end of the file never hurt anyone.

Always err on the side of too large. Here are sensible defaults:

Scenario	Hex Size	Bytes
Basic signature (no chain, no timestamp)	`0x2500` (PDFBox default)	~9 KB
Signature with certificate chain	`0x4000`	~16 KB
PAdES-B-T (with RFC 3161 timestamp)	`0x6000`	~24 KB
PAdES-B-LT (embedded OCSP + CRLs)	`0x18000`	~96 KB
PAdES-B-LTA (archival, large CRL)	`0x28000`	~160 KB

I've seen production systems fail because someone used the PDFBox default (`0x2500`) for PAdES-B-T signatures. The first 500 signatures worked fine because the TSA responses were

small. Then the TSA renewed its certificate with a longer chain, the responses grew by 2 KB, and every signing operation started failing on a Friday afternoon. Set it to `0x6000` minimum and forget about it.

1.1.4.3 CMS SignedData: The Cryptographic Core

The CMS blob inside `/Contents` follows RFC 5652 — Cryptographic Message Syntax (CMS) §5. If you've worked with S/MIME email signatures or code signing, this structure will look familiar — it's the same `SignedData` format used across the entire PKI ecosystem:

```
SignedData ::= SEQUENCE {
    version            CMSVersion,
    digestAlgorithms   DigestAlgorithmIdentifiers,
    encapContentInfo   EncapsulatedContentInfo,
    certificates   [0] IMPLICIT CertificateSet OPTIONAL,
    crls           [1] IMPLICIT RevocationInfoChoices OPTIONAL,
    signerInfos        SignerInfos
}
```

- Version: 1 for basic CMS, 3 if attribute certificates are present
- Set of digest algorithms used (typically just `{SHA-256}`)
- Content type: `id-data` (1.2.840.113549.1.7.1) with **absent** content — this is what "detached" means. The signed data (the PDF byte range) is not inside the CMS structure. It's outside, in the PDF file itself.
- Certificates — the signer's certificate and optionally the full chain (intermediates, possibly root). **Always include the full chain.** I know it makes the blob bigger. I know storage is a concern. Do it anyway. A signature without a chain is a signature that can only be validated by someone who already has all your certificates. That's almost nobody.
- CRLs — rarely embedded here in the modern PAdES world. In PAdES-B-LT and later, revocation data goes in the DSS dictionary instead.
- Signer information — the actual signature value and signed attributes. This is where the cryptography lives.

1.1.4.3.1 SignerInfo: Where the Cryptography Lives

```
SignerInfo ::= SEQUENCE {
    version                   CMSVersion,
    sid                       SignerIdentifier,
    digestAlgorithm           DigestAlgorithmIdentifier,
    signedAttrs           [0] IMPLICIT SignedAttributes OPTIONAL,
    signatureAlgorithm        SignatureAlgorithmIdentifier,
    signature                 SignatureValue,
    unsignedAttrs         [1] IMPLICIT UnsignedAttributes OPTIONAL
}
```

- Identifies the signer's certificate — either by issuer DN + serial number, or by subject key identifier. Both work. Issuer+serial is more traditional; subject key identifier is more robust when certificates are reissued.
- Hash algorithm used to digest the PDF byte range. SHA-256 is the minimum acceptable. SHA-384 or SHA-512 if you're paranoid or work in defense.

- **Signed attributes** — authenticated metadata. See the critical attributes table below. These are *signed*, meaning any modification invalidates the signature.
- Signature algorithm (e.g., SHA256withRSA, SHA256withECDSA). Note: the *signature* is computed over the DER-encoded signed attributes, not directly over the PDF bytes. This is a CMS subtlety that trips up many implementers.
- The actual cryptographic signature bytes.
- **Unsigned attributes** — unauthenticated extensions. Timestamps go here.

1.1.4.3.2 The Signed Attributes That Matter The first two attributes are required by CMS whenever signed attributes are present. The remaining attributes add PAdES compliance and defense-in-depth.

Table 1.14: Required CMS Signed Attributes — RFC 5652 — Cryptographic Message Syntax (CMS) §11

Attribute	OID	Req	Spec Ref
`content-type`	1.2.840.113549.1.9.3	Required	RFC 5652 — Cryptographic Message Syntax (CMS) §11.1
Must be `id-data` (1.2.840.113549.1.7.1). Required by CMS when signed attributes are present. Forgetting this is a classic implementation bug — the signature appears valid in some tools but fails in others.			
`message-digest`	1.2.840.113549.1.9.4	Required	RFC 5652 — Cryptographic Message Syntax (CMS) §11.2
SHA-256 hash of the PDF byte range. This is what's actually verified during validation. If this hash doesn't match the file bytes, the signature is invalid. Full stop.			

Table 1.15: PAdES/CAdES Signed Attributes

Attribute	OID	Req	Spec Ref
`signing-certificate-v2`	1.2.840.113549.1.9.16.2.47	Required (PAdES)	RFC 5035 §3
Hash of the signer's certificate (SHA-256). **Required for PAdES / CAdES.** This single attribute prevents certificate substitution attacks — an attacker can't swap in a different certificate without breaking the signature. It's ~50 bytes. There is no reason not to include it.			
`signing-time`	1.2.840.113549.1.9.5	Optional	RFC 5652 — Cryptographic Message Syntax (CMS) §11.3
Claimed signing time. Not authoritative — it's whatever the signer's clock said. For authoritative time, you need a timestamp (see unsigned attributes). Optional, but almost everyone includes it.			
`algorithm-protection`	1.2.840.113549.1.9.52	Recommended	RFC 6211
Protects against algorithm substitution attacks by binding the digest and signature algorithms into the signed attributes. Good hygiene to include.			

The `signing-certificate-v2` attribute is what makes the difference between `/adbe.pkcs7.detached` and `/ETSI.CAdES.detached`.

Without it: plain CMS (PKCS#7) — vulnerable to certificate substitution attacks. With it: CAdES — the signer's certificate is bound to the signature by hash.

This single attribute is the gateway to PAdES compliance. It's about 50 bytes in the CMS structure. There is no reason not to include it, even if you're using `/adbe.pkcs7.detached` as your SubFilter.

Certificate substitution attacks are not theoretical. In 2019, researchers at Ruhr University Bochum demonstrated practical attacks against multiple PDF viewers that exploited the absence of certificate binding. The fix was trivially simple: check `signing-certificate-v2`. Several major viewers had to be patched.

1.1.4.3.3 Unsigned Attributes: The Afterthought That Became Essential

Table 1.16: Unsigned Attributes — RFC 5652 — Cryptographic Message Syntax (CMS) §11

Attribute	OID	Req	Spec Ref
`signature-time-stamp`	1.2.840.113549.1.9.16.2.14	Required (PAdES-B-T)	RFC 3161 — Internet X.509 PKI Time-Stamp Protocol (TSP), ETSI EN 319 142-1 V1.1.1 (2016-04) §5.3
RFC 3161 timestamp token over the signature value. Provides authoritative proof of signing time from a trusted TSA. This is the only way to prove *when* a signature was made. The timestamp token has its own signature from the TSA, making it self-authenticating — which is why it's safe as an unsigned attribute.			
`certificate-values`	1.2.840.113549.1.9.16.2.28	Optional	RFC 5126 §5.6.1
Full certificate chain values. Used in CAdES-X-Long. In the PAdES world, these typically go in the DSS dictionary instead (see ???).			
`revocation-values`	1.2.840.113549.1.9.16.2.24	Optional	RFC 5126 §5.6.1
OCSP responses and CRLs. Used in CAdES-X-Long. In PAdES, these go in the DSS dictionary instead.			

The timestamp attribute deserves special attention. Unsigned attributes are, by definition, not protected by the signature — they can be added after signing without invalidating anything. This is *by design*. A timestamp proves that the signature existed at a certain time, and it's intentionally added after the signature is created (because you need the signature value to timestamp it).

This is also why you should never put anything security-critical in unsigned attributes that isn't self-authenticating (like a timestamp token, which has its own signature from the TSA).

1.1.5 The Signing Process: Step by Step

The process of creating a PDF digital signature is intricate because the signature must cover bytes that include the signature dictionary itself — a chicken-and-egg problem solved by the

`/Contents` pre-allocation we discussed above.

Here's the full sequence, and understanding it is essential for debugging:

1.1.5.1 Step-by-Step Detail

Step	Detail
1	**Create or locate the signature field.** If the document already has an unsigned field (`/FT /Sig` with no `/V`), use it. Otherwise, create a new field in the AcroForm `/Fields` array with a widget annotation on the desired page. *Practical note:* if the document was prepared by someone else (a template), always prefer using existing unsigned fields. Creating new fields in a document that already has signatures is technically possible via incremental save, but it makes some validators nervous.
2	**Build the signature dictionary** with placeholder values. `/ByteRange` is set to `[0 0 0 0]` (will be overwritten). `/Contents` is filled with `00` bytes at the estimated maximum size. All other keys (`/Filter`, `/SubFilter`, `/M`, `/Name`, `/Reason`, `/Reference`) are set to their final values *now* — they become part of the signed bytes and can't change later.
3	**Write the PDF using incremental save.** The new objects (signature dict, updated field, updated xref) are appended after the existing `%%EOF`. This is critical — a full rewrite would invalidate any existing signatures. *This is non-negotiable.* Even if the document has no prior signatures, use incremental save. It's the only safe habit.
4	**Compute actual byte offsets.** After writing, the exact byte positions of the `/Contents` hex string are known. Update `/ByteRange` to `[0, contentsStart, contentsEnd, fileLength - contentsEnd]`. The two segments cover everything *except* the `/Contents` value itself.
5	**Read the signed bytes.** Concatenate `file[0 .. contentsStart-1]` and `file[contentsEnd .. fileLength-1]`. These are the exact bytes the signature covers — everything except the `/Contents` hex value. *Debug tip:* if you suspect byte range issues, dump these bytes to a file and hash them manually. Compare with the `message-digest` attribute in the CMS.
6	**Hash the signed bytes.** Compute SHA-256 (or SHA-384/512) over the concatenated byte ranges. This hash becomes the `message-digest` signed attribute in the CMS structure.
7–8	**Sign the hash.** The private key (RSA or ECDSA) signs the DER-encoded signed attributes (which include the hash). This may happen locally or via a remote HSM/signing service. In the remote case, only the hash leaves your system — the private key never does. More on remote signing in ???.
9	**Build the CMS `SignedData` structure.** Include: digest algorithm, signer certificate, certificate chain, signed attributes (content-type, message-digest, signing-certificate-v2), signature value. DER-encode the result.

Step	Detail
10	**Write CMS bytes into `/Contents`.** Hex-encode the DER bytes and write them at the exact byte offset of the `/Contents` value in the file. Remaining space is left as `00` padding. No other bytes in the file are modified.

Step 10 is a **raw byte write** — not a PDF object update. The file is opened as a random-access binary stream, the cursor is positioned at the `/Contents` hex string's start offset, and the hex chars are overwritten in place. If the CMS blob is larger than the pre-allocated space, signing fails. If even a single byte is written at the wrong offset, the signature is invalid and the PDF may be corrupted. This is why you don't implement PDF signing from scratch — you use PDFBox or a similar library that handles the byte-level bookkeeping for you.

1.1.6 Incremental Save: The Foundation of Multi-Signature

Every signature operation uses **incremental save** (ISO 32000-2:2020 §7.5.6). The original file bytes are never modified — new objects are appended after the existing `%%EOF`.

This is the mechanism that makes multiple independent signatures possible. Without incremental save, adding a second signature would modify the bytes covered by the first signature, invalidating it. With incremental save, the first signature's bytes remain untouched — the second signature's data is simply appended at the end.

Each signature's `/ByteRange` covers everything up to and including its own incremental update — but **not** the updates that come after:

- Signature 1's byte range: `[0, …, …, M]` — covers original + update #1
- Signature 2's byte range: `[0, …, …, P]` — covers original + update #1 + update #2

This is why Signature 1 remains valid after Signature 2 is added: Signature 2's bytes are **appended after** the range that Signature 1 covers. From Signature 1's perspective, those bytes are "trailing" — present in the file but outside the signed range.

"The file has been modified since signing" — don't panic.

This is the single most common false alarm in PDF signature validation. A validator detects trailing bytes after a signature's byte range and flags it as "modified." But in the vast majority of cases, those trailing bytes are simply a later incremental update — another signature, a form fill, or an annotation.

What to check:

1. Is the trailing data a valid incremental update? (Look for xref table + trailer + %%EOF.)
2. Does it correspond to an expected operation? (Another signature, a form field being filled.)
3. Did the byte range change? (The original signature's bytes at their original offsets should be byte-for-byte identical.)

If the answers are yes, yes, and no — the signature is fine. The "modification" is just the normal operation of PDF's append-only architecture.

When to actually worry:

- Trailing bytes that aren't a valid incremental update — raw data appended to the file
- An incremental update that modifies objects covered by an existing signature in unexpected ways (e.g., page content changes)
- Multiple %%EOF markers where the trailing section isn't a proper PDF update

1.1.7 Widget Annotations and Page Placement

Every signature field has an associated widget annotation (ISO 32000-2:2020 §12.5.6.19) that defines its visual presence on a page.

1.1.7.1 The `/Rect` Coordinate System

The widget's `/Rect` array specifies `[llx, lly, urx, ury]` in **default user space** coordinates:

PDF coordinates have **origin at lower-left** with Y increasing upward. This is the opposite of screen coordinates (origin top-left, Y increasing downward). This will bite you. It bites everyone. You will write the conversion wrong the first time. Here it is, so you can copy it:

```
// viewport = page.getViewport({ scale: scale })
var left = bbox[0] * scale;
var top  = viewport.height - (bbox[1] + bbox[3]) * scale;
var width = bbox[2] * scale;
var height = bbox[3] * scale;
```

- X translates directly (both coordinate systems have X increasing rightward)
- Y must be flipped: PDF Y=0 is bottom, screen Y=0 is top. Note the `bbox[1] + bbox[3]` — that's lower-left-Y plus height, giving the top edge in PDF space, which maps to the top edge in screen space.
- Width scales linearly
- Height scales linearly

Where `bbox` is `[lowerLeftX, lowerLeftY, width, height]` as extracted by PDFBox's `PDRectangle`. Note that PDFBox gives you `[x, y, width, height]` while the raw PDF `/Rect` is `[llx, lly, urx, ury]` — PDFBox does the conversion for you. Don't convert twice.

1.1.7.2 Invisible Signatures

A signature can be **invisible** — having no visual appearance on the page:

- **Zero-size rect:** `/Rect [0 0 0 0]` — the widget exists but has no area
- **No widget at all:** The field has no annotation entry (valid per spec, but some viewers complain)

Invisible signatures are common for:

- Certification signatures — the document author certifies the document without cluttering the layout
- Machine-generated signatures — automated workflows where nobody's looking at the PDF
- Counter-signatures — additional approval signatures in complex chains where visual space is limited

Should your signature be visible or invisible?

For approval signatures intended for human signers: make them visible. Users expect to *see* their signature on the page. A visible signature provides a visual anchor — ”I signed here” — and helps in audit scenarios.

For certification signatures and automated workflows: invisible is usually fine. The certification status is shown in the viewer’s signature panel, not on the page.

For machine-to-machine: always invisible. No human is looking at the page. Adding a visual appearance just wastes space and processing time.

One exception: if you’re in a regulated industry where auditors review printed copies of signed PDFs (yes, this still happens in 2026), visible signatures are essential because auditors can’t check digital signatures on paper. In that case, include the signer’s name, the signing time, and ideally a note like ”Digitally signed — verify electronically.”

1.1.8 AcroForm `/SigFlags`: Document-Level Signature Metadata

> If the document contains signature fields, the AcroForm dictionary shall include a SigFlags entry.
>
> — ISO 32000-2:2020 §12.7.2, Table 225

Table 1.17: SigFlags Bit Values — ISO 32000-2:2020 §12.7.2, Table 225

Name	Bit	Req	Spec Ref
`SignaturesExist`	1 (value 1)	Required	§12.7.2, Table 225
At least one signature field has a value. Readers should present signature-specific UI (e.g., the blue bar at the top of Adobe Acrobat). Must be set whenever any signature field contains a `/V` entry.			
`AppendOnly`	2 (value 2)	Required	§12.7.2, Table 225
The document **shall** only be saved incrementally. A conforming reader that allows editing **must not** do a full save — it must append. This is the single most important flag for signature preservation. Without it, a naive editor may do a full rewrite and destroy every signature.			

In practice, both bits should be set (`/SigFlags 3`) whenever the document contains any signed signature field. This was already discussed above, but it bears repeating: if you get this wrong, signatures will be destroyed by innocent save operations, and the blame will be on your tool.

1.1.9 PDFBox API Mapping

For those of you who skipped here — I don’t blame you, this is the section with the code — here’s how the spec concepts map to PDFBox 3.x classes:

PDF Concept	PDFBox Class	Key Methods
Signature field	`PDSignatureField`	`getSignature()`, `getWidgets()`, `getPartialName()`
Signature dictionary	`PDSignature`	`getByteRange()`, `getContents(byte[])`, `getSubFilter()`, `getName()`, `getSignDate()`
AcroForm	`PDAcroForm`	`getFields()`, `getFieldTree()`, `setSignaturesExist()`, `setAppendOnly()`
Widget annotation	`PDAnnotationWidget`	`getRectangle()`, `getPage()`, `getNormalAppearanceStream()`
Appearance stream	`PDFormXObject`	`getStream()`, `getBBox()`, `getResources()`
Adding a signature	`PDDocument`	`addSignature(PDSignature, SignatureInterface, SignatureOptions)`
Incremental save	`PDDocument`	`saveIncremental(OutputStream)`

1.1.9.1 Minimal Signing Code (Key Lines)

The essential PDFBox signing code in ~20 lines. This is the skeleton — the full, production-ready version is in Appendix A, but these are the lines that matter:

```
PDDocument doc = Loader.loadPDF(pdfBytes);
PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();

PDSignature signature = new PDSignature();
signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
signature.setSubFilter(PDSignature.SUBFILTER_ETSI_CADES_DETACHED);
signature.setName("John Doe");
signature.setSignDate(Calendar.getInstance());

SignatureOptions options = new SignatureOptions();
options.setPreferredSignatureSize(0x6000);
options.setVisualSignature(appearanceStream);

doc.addSignature(signature, signingInterface, options);
doc.saveIncremental(outputStream);
```

- Always load from bytes. Don't create a new `PDDocument()` and try to `saveIncremental()` — PDFBox 3.x will throw `IllegalStateException: document was not loaded from a file or a stream`. Yes, this error message has confused every single person who's ever encountered it.
- Filter is always `Adobe.PPKLite` for interoperability. Don't be creative here.

- SubFilter: use `ETSI.CAdES.detached` for PAdES compliance. See the earlier section for why.
- Pre-allocate 24KB for the CMS blob. **Not** the PDFBox default of 9KB. See the pre-allocation table above.
- Visual appearance — set this to `null` for invisible signatures. Or provide an `InputStream` of a pre-built appearance.
- Register the signature and the `SignatureInterface`. The interface's `sign()` method will be called during `saveIncremental()`.
- **Incremental save.** There is no other option. Do not call `save()` on a document you're signing.

1.1.9.2 The `SignatureInterface`: Your Extension Point

```
public interface SignatureInterface {
    byte[] sign(InputStream content) throws IOException;
}
```

- Called by PDFBox during `saveIncremental()`. The `InputStream` provides the concatenated byte ranges — this is the data you need to hash and sign. Your implementation builds the CMS `SignedData` blob and returns the DER-encoded bytes.

This is the **only thing you need to implement**. PDFBox handles the PDF structure, the byte range calculation, the incremental save, the widget creation, the appearance stream embedding. You handle the cryptography.

Keep your `SignatureInterface` implementation stateless and testable.

Don't put PDF logic in there. Don't reference the `PDDocument`. Your `sign()` method receives an `InputStream` of bytes and returns CMS bytes. That's it. This makes it trivially testable — pass in any byte stream, check that the output is valid CMS. It also makes it easy to swap between local signing (BouncyCastle) and remote signing (HSM, cloud) without touching any PDF code.

1.1.10 Common Mistakes and How to Avoid Them

After years of working with PDF signatures, the same problems come up over and over. Here's the hit list:

#	Mistake	Fix
1	**Trusting `/Name` for signer identity.** Many systems display the `/Name` field from the signature dictionary as if it's the verified signer name. It's not. It's free-text set by whoever controls the signing process.	Always extract the signer's identity from the X.509 certificate in the CMS structure. Display the certificate's Subject CN (or the full Subject DN for enterprise contexts).

#	Mistake	Fix
2	**Using `PDDocument.save()` instead of `saveIncremental()`.** Full save rewrites the entire file, destroying all existing signatures.	Always use `saveIncremental()`. Make it a code review checklist item: "Does this code path call `save()` on a signed document? Reject."
3	**Pre-allocating too little space for `/Contents`.** Works fine in testing with simple certificates, breaks in production when chains get longer or timestamps are added.	Use `0x6000` (24KB) minimum. For PAdES-B-LT, use `0x18000` (96KB). Test with your actual TSA and certificate chain, not with test certificates.
4	**Forgetting `signing-certificate-v2` in the CMS attributes.** Technically valid as PKCS#7, but not PAdES-compliant and vulnerable to certificate substitution.	Use BouncyCastle's `JcaSignerInfoGeneratorBuilder` with the correct `DigestCalculatorProvider`. It adds the attribute automatically when configured correctly. See the code in Appendix A.
5	**Not including the certificate chain.** A signature with just the signer's certificate can't be validated by anyone who doesn't already have the issuer certificates.	Always include the full chain (signer + intermediates). You can omit the root CA — validators typically have their own trust stores. But include everything else.
6	**Creating a fresh `PDDocument` and calling `saveIncremental()`.** PDFBox 3.x requires the document to have been loaded from bytes.	Create a blank PDF, save it to bytes, load it back, then sign. See the test code in this chapter.
7	**Setting `/SigFlags` to 1 instead of 3.** Bit 1 alone means "signatures exist" but doesn't enforce append-only saving.	Always set `/SigFlags` to 3. No exceptions.
8	**Modifying the PDF between adding the signature and calling `saveIncremental()`.** Any change to the document graph between `addSignature()` and `saveIncremental()` may change byte offsets, breaking the signature.	Set up everything before calling `addSignature()`. Once you've added the signature, the very next call should be `saveIncremental()`. Nothing in between.

1.1.11 Key Points

- A PDF signature = signature field + signature dictionary + widget + appearance stream — all embedded in the document
- The signature dictionary's `/Contents` holds a CMS `SignedData` blob (hex-encoded, zero-padded)

- `/ByteRange` defines exactly which file bytes are signed — everything except `/Contents` itself
- `/SubFilter` determines the signature format: use `/ETSI.CAdES.detached` for PAdES compliance — there's no good reason not to
- The `signing-certificate-v2` attribute is what distinguishes CAdES from plain PKCS#7 — and what prevents certificate substitution attacks
- All signatures use incremental save — original bytes are never modified, ever
- `/SigFlags` must be set to 3 (SignaturesExist + AppendOnly) to protect existing signatures — getting this wrong destroys other people's signatures
- Seed value dictionaries (`/SV`) can constrain how a field is signed — use them in templates to prevent signer mistakes
- Widget `/Rect` uses PDF coordinates (origin lower-left, Y up) — you will get the conversion wrong the first time, and that's OK
- PDFBox's `SignatureInterface.sign(InputStream)` is the single extension point for all signing logic — keep it clean, stateless, and testable
- When in doubt, pre-allocate more space for `/Contents` — wasted bytes are cheap, failed signatures are expensive
- "The file has been modified since signing" is usually not a problem — it's usually just a later incremental update

1.1.12 Specification References

Chapter 2

Byte Range Deep Dive

The byte range is where PDF signatures live or die. Everything in Chapter 12 — the `/Sig` dictionary, the `/SubFilter`, the certificate chain, the CMS structure — all of it is meaningless if the byte range is wrong. A single off-by-one error in the `/ByteRange` array, and the signature is cryptographically invalid. A subtle gap in coverage, and an attacker can inject content that the signature appears to endorse.

This chapter takes the byte range apart byte by byte. We will trace exactly which bytes are signed, where the gap is, why the gap exists, what lives in the gap, and what goes wrong when any of these assumptions are violated. We will look at hex dumps of real PDFs, visualize the byte range geometry, walk through multi-signature scenarios, and examine the exact attacks that exploit byte range weaknesses.

If Chapter 12 taught you *what* a byte range is, this chapter teaches you *how* it actually works in the raw file — and how to verify that it works correctly.

2.1 The Byte Range Model

2.1.1 What the Spec Says

ISO 32000-2:2020 §12.8.1 defines the `/ByteRange` array as:

> An array of pairs of integers (starting byte offset, length in bytes) that shall describe the exact byte range for the digest computation. Multiple discontiguous byte ranges shall be used to describe a digest that does not include the signature value itself.

In practice, every PDF signature uses exactly **one pair of byte ranges** — two segments — with a gap between them. The gap contains the `/Contents` value (the hex-encoded CMS signature). The two segments cover everything *else* in the file.

The `/ByteRange` array always has exactly four integers:

```
/ByteRange [offset1 length1 offset2 length2]
```

Where:

- `offset1` = 0 (always — the signed region starts at the first byte of the file)
- `length1` = number of bytes from the start of the file to the opening < of the `/Contents` hex string
- `offset2` = byte offset immediately after the closing > of the `/Contents` hex string
- `length2` = number of bytes from `offset2` to the end of the file

The math is simple:

- **Signed bytes** = `[0, length1)` + `[offset2, offset2 + length2)`
- **Unsigned gap** = `[length1, offset2)` — this contains only the `/Contents` hex string value
- **Total file size** = `offset2 + length2`

2.1.2 Visualizing the Byte Range

To make this concrete, let's look at a real signed PDF. Suppose we have a signed file of 45,000 bytes with a 32KB (32,768 hex characters = 16,384 bytes) reserved `/Contents` value:

```
File layout (not to scale):

Byte 0                                                        Byte 44,999
|==============================================================|
|  PDF header, objects, pages,    |  /Contents  |  xref, trailer |
|  /Sig dictionary (up to the    |  hex string |  %%EOF         |
|  opening < of /Contents)       |  (the gap)  |                |
|================================|=============|================|
|<---- length1 = 14,000 -------->|<-- gap  --->|<-- length2 -->|
|                                 |  = 16,408   |   = 14,592    |
|                                 |             |                |
|  SIGNED (segment 1)            |  NOT SIGNED |  SIGNED (seg 2)|
|================================|=============|================|

/ByteRange [0 14000 30408 14592]
```

The gap is slightly larger than 16,384 bytes because it includes the < and > delimiters that bracket the hex string, plus the '/Contents ' key prefix. The exact gap size depends on how PDFBox (or whatever library produced the file) serializes the signature dictionary.

2.1.3 Why the Gap Exists: The Chicken-and-Egg Problem

The gap exists because of a fundamental bootstrapping problem:

1. To **compute the signature**, you need to hash the bytes of the file
2. To **write the file**, you need to include the signature (inside `/Contents`)
3. But the signature is *part of* the file — so how can you hash a file that doesn't exist yet?

The solution: **exclude the signature value from the hash**. The `/Contents` hex string is the one part of the file that is *not* covered by the signature. Everything else — the PDF header, all objects, the signature dictionary itself (including `/ByteRange`, `/Filter`, `/SubFilter`, and all other keys), the cross-reference table, and the trailer — is signed.

This means the signing process works like this:

1. Write the entire PDF file, including the `/Sig` dictionary, but leave the `/Contents` value as a placeholder (all zeros)
2. Record the byte offsets of where the placeholder starts and ends
3. Compute the hash over everything *except* the placeholder: `hash = SHA256(segment1 || segment2)`
4. Construct the CMS `SignedData` using this hash
5. Hex-encode the CMS bytes and write them into the placeholder space

6. If the CMS is shorter than the reserved space (it almost always is), pad the remainder with zeros

This is why PDFBox's `SignatureInterface.sign(InputStream)` receives an `InputStream` — that stream contains the concatenation of segment 1 and segment 2 (the bytes *outside* the gap), ready to be hashed.

The `/ByteRange` array itself is part of the signed region. This is critical for security: if an attacker could modify the `/ByteRange` after signing, they could redirect the signature to cover different content. The byte range is committed before the hash is computed, and any change to it invalidates the signature.

2.2 The Gap in Detail

2.2.1 What's Actually in the Gap

Let's look at a hex dump of the gap region. In a properly signed PDF, the gap contains exactly one thing: the hex-encoded CMS `SignedData`, padded with zeros:

```
Byte 14000:  3C                                          -- opening '<'
Byte 14001:  30 38 30 36 30 39 ...                       -- CMS bytes (hex-encoded)
             ...
             ... (several KB of hex characters)
             ...
Byte 28500:  30 30 30 30 30 30 30 30 30 30 30 30 ...     -- zero padding ("000000...")
             ...
Byte 30407:  3E                                          -- closing '>'
```

The hex encoding doubles the size: a 4,096-byte CMS signature becomes 8,192 hex characters. The reserved space is typically 16KB (32,768 hex characters = 16,384 bytes of actual CMS capacity), which is enough for most PAdES-B-B signatures.

PAdES-B-LTA signatures are much larger than PAdES-B-B. A B-LTA signature includes:

- The CMS `SignedData` itself (~2–4 KB)
- The signing certificate and full chain (~2–5 KB)
- The timestamp token from the TSA (~2–4 KB)
- OCSP responses for each certificate in the chain (~1–3 KB each)
- CRL data if OCSP is not available (~5–50 KB)

A B-LTA signature can easily reach 20–30 KB, which requires 40–60 KB of hex-encoded space. If you're producing B-LTA signatures, reserve at least **64 KB** for `/Contents`:

```
// PDFBox: set signature content size (in bytes, before hex encoding)
SignatureOptions options = new SignatureOptions();
options.setPreferredSignatureSize(65536); // 64 KB
doc.addSignature(signature, signingInterface, options);
```

If the CMS exceeds the reserved space, PDFBox throws an `IOException` — the signing fails silently in the sense that no file is produced. This is the #1 cause of "signing worked in development but fails in production" — development certificates have short chains and no timestamps, production certificates have full chains, OCSP responses, and timestamps that exceed the default buffer.

2.2.2 The Zero-Padding Question

The zero padding at the end of the `/Contents` hex string is not part of the CMS. It's just filler — the hex string `00` decodes to a null byte, and CMS parsers stop reading at the end of the DER-encoded ASN.1 structure, ignoring trailing zeros.

Some validators are stricter than others about this padding:

- **Adobe Acrobat**: Ignores trailing zero padding — works fine
- **LibreOffice**: Ignores trailing zeros — works fine
- **Some custom validators**: May compute the hash of the entire `/Contents` hex string (including padding) and fail because the hash doesn't match what the signer computed over just the CMS bytes. This is a bug in the validator, not in the signed PDF.

When writing validation code, always parse the `/Contents` hex string as DER and stop at the end of the ASN.1 structure. Do not hash the raw hex string — decode it to binary, find the end of the DER sequence (the total length indicated in the ASN.1 header), and ignore everything after that.

2.3 Incremental Saves and the Byte Range

2.3.1 How Signing Creates an Incremental Update

When PDFBox signs a PDF, it performs an **incremental save** — it appends new data to the end of the existing file rather than rewriting the entire file. This is not optional for signing; it is how signing must work.

An incrementally saved PDF looks like this:

```
Original file (revision 0):
|  Header | Objects | Xref | Trailer | %%EOF |

After signing (revision 1):
|  Header | Objects | Xref | Trailer | %%EOF |  New objects | New Xref | New Trailer | %%EOF |
|<-------------- original file ------------->|<------------ appended by signing ------------->|
```

The "new objects" include the signature dictionary (or modifications to an existing signature field), any new annotation objects for the visual appearance, and updated AcroForm entries. The "new xref" is a cross-reference section that records the locations of these new objects, and the "new trailer" points to the new xref.

The entire appended region — including the new `%%EOF` — is covered by the byte range of the signature. The byte range starts at byte 0 (the original file's header) and extends to the end of the appended data, with only the `/Contents` gap excluded.

This means:

- The signature covers the **original file** in its entirety
- The signature covers the **new objects** added during signing (the signature dictionary, appearances, etc.)
- The signature covers the **new cross-reference table** and **new trailer**
- The only thing not covered is the CMS signature value itself

2.3.2 What Happens After Signing: The Trailing Bytes Problem

Here is one of the most security-critical aspects of byte range handling: **what happens to bytes that appear after the signed region?**

Consider this scenario:

```
After signing:
|  ... file content ...  |  /Contents  |  xref | trailer | %%EOF |
|<------- signed ------->|<--- gap --->|<-------- signed -------->|
                                                                  ^
                                                                  End of signed region
                                                                  (offset2 + length2 = file si
```

If `offset2 + length2` equals the total file size, the signature covers everything — good.

But what if someone appends data *after* the signed region?

```
After tampering:
|  ... file content ...  |  /Contents  |  xref | trailer | %%EOF |  MALICIOUS DATA  |
|<------- signed ------->|<--- gap --->|<-------- signed -------->|<--- NOT SIGNED -->|
                                                                  ^
                                                                  End of signed region
```

The byte range still validates — the hash over the signed segments hasn't changed. But the file now contains unsigned data at the end. If this unsigned data is a valid incremental update (with its own xref, trailer, and `%%EOF`), a PDF viewer might load it and display the *attacker's* content while showing the original signature as valid.

This is exactly the **Incremental Saving Attack (ISA)** that the Bochum team demonstrated in 2019 (Chapter 24). The attack works because PDF viewers before the patches would happily load incremental updates appended after the signed byte range, display the new content, and still report the signature as valid — because the signed bytes hadn't been modified.

The trailing bytes check is the most critical validation step.

When verifying a PDF signature, you **must** verify that:

```
offset2 + length2 == actual file size
```

If the file is larger than the signed region, something was appended after signing. This doesn't necessarily mean the file was tampered with — a second legitimate signature would also append data — but it means you must validate the second revision's signature independently.

If there is unsigned trailing data that is *not* a valid subsequent signature revision, the document has been modified after signing and the signature should be treated as covering only the original content.

Chapter 24 provides the full attack taxonomy and defense code.

2.3.3 Multi-Signature Scenarios

Enterprise PDF workflows often involve multiple signatures — a document might be signed by the author, countersigned by a manager, and certified by a compliance officer. Each signature creates its own incremental update.

The byte range geometry for a multiply-signed PDF looks like this:

```
Signature 1 (author):
|  Original PDF  |  Sig1 objects  |  /Contents1  |  Sig1 xref + trailer + %%EOF  |
|<-- signed 1 -->|<-- signed 1 -->|<--- gap 1 -->|<---------- signed 1 --------->|

Signature 2 (manager):
|  Original PDF  |  Sig1 objects  |  /Contents1  |  Sig1 xref  |  Sig2 objects  |  /Contents2  |
|<------------------------------------ signed 2 (except gap 2) ---------------------
|<--- gap 2 -->|<---------- signed 2 --------->|

Signature 3 (compliance):
|  Everything from signatures 1 and 2  |  Sig3 objects  |  /Contents3  |  Sig3 xref + trailer + %
|<------------- signed 3 (except gap 3) --------------------------------|<--- gap 3 -->|<--------
```

Each signature covers everything before its own gap (including previous signatures' gaps — the hex bytes of `/Contents1` and `/Contents2` are part of the signed data for signatures 2 and 3). This nesting means:

- **Signature 1** covers the original PDF + the objects and xref added during the first signing
- **Signature 2** covers everything that signature 1 covers PLUS the objects added during the second signing. This means signature 2 implicitly endorses signature 1's existence (though not necessarily its validity)
- **Signature 3** covers everything, providing the broadest coverage

Each signature is a snapshot of the document at the time it was applied. Signature 1 saw the document in state S1. Signature 2 saw it in state S2 (which includes signature 1). If you need to know what the document looked like when signature 1 was applied, you must extract revision 1 by truncating the file at the end of signature 1's byte range — everything after that was added later.

PDFBox provides this capability:

```
// Extract the revision that a specific signature covers
PDDocument doc = Loader.loadPDF(signedFile);
List<PDSignature> signatures = doc.getSignatureDictionaries();
for (PDSignature sig : signatures) {
    int[] byteRange = sig.getByteRange();
    int revisionEnd = byteRange[2] + byteRange[3];

    // Extract this revision
    byte[] revision = new byte[revisionEnd];
    try (InputStream is = new FileInputStream(signedFile)) {
        is.readNBytes(revision, 0, revisionEnd);
    }
    // 'revision' is now the PDF as it existed when this signature was applied
}
```

2.3.4 Permission-Based Restrictions: What Can Change After Signing?

Not all changes after signing are attacks. The PDF specification defines three **modification detection and prevention (MDP)** permission levels that control what changes are allowed after a certification signature:

Table 2.1: DocMDP Permission Levels (ISO 32000-2:2020 §12.8.2.2.2)

Level	Name	Req	Description
P=1	No changes	Optional	No changes whatsoever are allowed. Any modification — including filling form fields or adding annotations — invalidates the signature. This is the most restrictive level.
P=2	Form filling + signing	Default	Users may fill form fields and apply additional signatures. Other changes (adding pages, modifying text, changing form field definitions) invalidate the signature. This is the most common level for government forms.
P=3	Form filling + signing + annotations	Optional	Users may fill form fields, sign, and add annotations (comments, highlights, stamps). This is the most permissive level while still detecting unauthorized modifications.

When validating a signature with MDP restrictions, you must check not only whether the byte range is intact, but also whether the *content* of subsequent incremental updates falls within the allowed permissions. This is where validation gets complex — you need to compare the PDF objects in the signed revision against the current revision and determine whether the differences are permitted.

Adobe Acrobat does this analysis automatically and shows warnings like ”The document has been altered since the signature was applied, but the changes are permitted.” Building this analysis yourself requires understanding of the PDF object model at a level covered in Chapters 7 and 8.

2.4 Size Estimation: How Much Space to Reserve

2.4.1 The Problem

Before you can sign a PDF, you must write it to disk with a placeholder for `/Contents`. The placeholder must be large enough to hold the final CMS signature. If it’s too small, the signing fails. If it’s too large, you waste space (and the file contains kilobytes of zero padding).

The challenge: the exact CMS size depends on:

- **The signing certificate** — larger key sizes (RSA-4096 vs RSA-2048) produce larger signatures
- **The certificate chain** — more intermediates = more bytes
- **The signed attributes** — PAdES requires ESS-signing-certificate-v2, content-type, message-digest; more attributes = more bytes
- **Timestamps** — a TSA response is itself a CMS `SignedData`, typically 2–4 KB

- **OCSP responses** — each certificate in the chain may have a 1–3 KB OCSP response
- **CRL data** — if a certificate's revocation information is provided via CRL instead of OCSP, CRLs can be 5–500 KB depending on the CA's revocation volume

You won't know the exact sizes until you've fetched the timestamp and revocation data — but you need to commit the space *before* computing the hash.

2.4.2 Estimation Strategies

Strategy 1: Fixed generous buffer (simple, recommended)

Reserve a fixed buffer that's large enough for worst-case scenarios:

```
// Signature size estimates by PAdES profile
int estimatedSize;
switch (profile) {
    case B_B:   estimatedSize = 16384;  break; // 16 KB — basic CMS, no timestamp
    case B_T:   estimatedSize = 32768;  break; // 32 KB — CMS + timestamp token
    case B_LT:  estimatedSize = 65536;  break; // 64 KB — CMS + timestamp + OCSP/CRL
    case B_LTA: estimatedSize = 131072; break; // 128 KB — CMS + timestamp + OCSP/CRL + archival TS
}
SignatureOptions options = new SignatureOptions();
options.setPreferredSignatureSize(estimatedSize);
```

This wastes some space (a B-B signature typically needs only 4–8 KB of the 16 KB reserved), but the waste is negligible compared to the content of a typical PDF. A 100 KB PDF with 16 KB of zero padding in `/Contents` is 116 KB — nobody cares.

Strategy 2: Pre-compute exact size (complex, rarely necessary)

If you're generating millions of signed PDFs per day and storage costs matter, you can:

1. Fetch the timestamp and revocation data *before* creating the placeholder
2. Compute the CMS size exactly
3. Reserve only the exact space needed (plus a small safety margin)

This is significantly more complex because you need to construct a "dry run" CMS to measure it, and the timestamp/OCSP data might change between the dry run and the actual signing. In practice, this optimization is almost never worth the complexity.

Just use 64 KB (65536 bytes) as your default. It's enough for PAdES-B-LT in the vast majority of cases, it wastes a trivial amount of space, and it eliminates the entire class of "signature too large for reserved space" failures. Only reduce it if you have a specific reason (embedded systems with extreme storage constraints) and only increase it if you're producing B-LTA signatures with CRLs from large CAs.

2.4.3 What Happens When the Estimate Is Wrong

If the CMS signature is **smaller** than the reserved space: no problem. The remaining space is filled with zero bytes. All conforming validators ignore the padding.

If the CMS signature is **larger** than the reserved space: **the signing fails**. PDFBox throws an `IOException` with a message like:

```
IOException: Can't write signature, not enough space
```

When this happens:

1. The output file is corrupted (partial write) — discard it
2. Increase `SignatureOptions.setPreferredSignatureSize()` and retry
3. To debug: log the actual CMS size from your `SignatureInterface.sign()` method and compare it to the reserved size

```
@Override
public byte[] sign(InputStream content) throws IOException {
    byte[] cmsBytes = createCMSSignature(content);
    log.info("CMS signature size: {} bytes (reserved: {} bytes)",
             cmsBytes.length, reservedSize);
    return cmsBytes;
}
```

2.5 Byte Range Verification: How to Check It

2.5.1 The Verification Algorithm

When validating a signature, the byte range verification follows these steps:

1. **Read the `/ByteRange` array** from the signature dictionary: `[offset1, length1, offset2, length2]`
2. **Verify structural sanity**:
   - `offset1` must be 0
   - `length1` must be positive
   - `offset2` must equal `length1` + gap size (where gap size = the length of the `/Contents` hex string including delimiters)
   - `length2` must be positive
   - `offset2 + length2` should equal the file size (or the size of the revision, for multi-signature documents)
3. **Verify the gap contains only `/Contents`**: the bytes in `[length1, offset2)` should be a hex string containing the CMS signature value
4. **Extract the signed bytes**: concatenate `[0, length1)` and `[offset2, offset2 + length2)`
5. **Compute the hash**: `hash = SHA-256(signed_bytes)` (or whatever digest algorithm the CMS specifies)
6. **Verify the CMS**: parse the CMS `SignedData` from `/Contents`, extract the signed `messageDigest` attribute, and compare it to the hash you just computed. If they match, and the CMS cryptographic verification passes, the byte range is intact.

2.5.2 Implementation in Java

```
/**
 * Verifies the byte range integrity of a PDF signature.
 * Returns true if the byte range covers the expected regions
 * and the hash matches the CMS digest.
 */
public boolean verifyByteRange(byte[] pdfBytes, PDSignature signature)
        throws IOException, CMSException {
```

```
    int[] byteRange = signature.getByteRange();

    // Step 1: Structural checks
    if (byteRange.length != 4) {
        log.error("ByteRange must have exactly 4 elements, got {}", byteRange.length);
        return false;
    }

    int offset1 = byteRange[0];
    int length1 = byteRange[1];
    int offset2 = byteRange[2];
    int length2 = byteRange[3];

    if (offset1 != 0) {
        log.error("ByteRange offset1 must be 0, got {}", offset1);
        return false;
    }

    // Step 2: Check coverage
    int signedEnd = offset2 + length2;
    if (signedEnd != pdfBytes.length) {
        log.warn("ByteRange ends at {} but file size is {} — " +
                 "{} trailing bytes are unsigned",
                 signedEnd, pdfBytes.length, pdfBytes.length - signedEnd);
        // This is a WARNING, not necessarily an error —
        // subsequent signatures would extend the file
    }

    // Step 3: Extract and hash signed bytes
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    digest.update(pdfBytes, offset1, length1);       // segment 1
    digest.update(pdfBytes, offset2, length2);        // segment 2
    byte[] computedHash = digest.digest();

    // Step 4: Parse CMS and compare
    byte[] cmsBytes = signature.getContents();
    CMSSignedData cmsData = new CMSSignedData(cmsBytes);
    SignerInformation signer = cmsData.getSignerInfos()
            .getSigners().iterator().next();

    // Extract the messageDigest from signed attributes
    Attribute digestAttr = signer.getSignedAttributes()
            .get(CMSAttributes.messageDigest);
    byte[] cmsDigest = ASN1OctetString.getInstance(
            digestAttr.getAttrValues().getObjectAt(0)).getOctets();

    boolean hashMatch = MessageDigest.isEqual(computedHash, cmsDigest);
    if (!hashMatch) {
        log.error("Hash mismatch: computed={} vs CMS={}",
                  Hex.toHexString(computedHash), Hex.toHexString(cmsDigest));
    }
    return hashMatch;
}
```

The code above verifies that the **hash** matches — it does not verify the **CMS signature** (the

RSA/ECDSA computation over the signed attributes). Full verification requires both:

1. Byte range hash matches CMS `messageDigest` attribute (shown above)
2. CMS signature over `signedAttrs` verifies with the signer's certificate

Chapter 31 provides complete verification code that covers both steps plus certificate chain validation.

2.6 Hex Dump Walkthrough: A Real Signed PDF

Let's trace the byte range through an actual signed PDF, byte by byte. This is the kind of analysis you do when debugging — you open a hex editor (or use `xxd`, `hexdump`, or QPDF) and look at the raw bytes.

2.6.1 Finding the Signature Dictionary

First, locate the signature dictionary in the file. With QPDF:

```
qpdf --show-object=15 signed-document.pdf
```

(Where object 15 is the signature dictionary — find it by tracing from the AcroForm Fields [signature field] /V.)

Output:

```
15 0 obj
<<
  /ByteRange [0 14231 47007 1849]
  /ContactInfo ()
  /Contents <3082...0000>
  /Filter /Adobe.PPKLite
  /Location ()
  /M (D:20260308120000+01'00')
  /Name (Test Signer)
  /Reason ()
  /SubFilter /adbe.pkcs7.detached
  /Type /Sig
>>
endobj
```

Now we know:

- **Segment 1**: bytes `[0, 14231)` — 14,231 bytes from file start
- **Gap**: bytes `[14231, 47007)` — 32,776 bytes containing `/Contents`
- **Segment 2**: bytes `[47007, 48856)` — 1,849 bytes at file end
- **Total file size**: 47,007 + 1,849 = 48,856 bytes

2.6.2 Examining the Gap Boundaries

Let's look at the bytes right around the gap:

```
xxd -s 14220 -l 30 signed-document.pdf
```

Output:

```
0000378c: 2f43 6f6e 7465 6e74 7320 3c30 3830 3230  /Contents <08020
0000379c: 3036 3032 3031 3030 3330 3832 3032 3032  060201003082020
```

We can see `/Contents <` at offset ~14,223. The < delimiter is at the byte just before the hex string begins. The `/ByteRange` says segment 1 ends at offset 14,231 — which is the < character. Actually, the exact offset of < depends on how PDFBox serializes the dictionary.

Now the end of the gap:

```
xxd -s 47000 -l 20 signed-document.pdf
```

Output:

```
0000b798: 3030 3030 3030 3030 3e0a 2f46 696c 7465  00000000>./Filte
```

The > delimiter is at byte 47,007 (approximately), followed by a newline and the next dictionary key. Segment 2 starts immediately after the >.

2.6.3 Verifying No Trailing Bytes

```
wc -c signed-document.pdf
# Output: 48856 signed-document.pdf
```

Byte range says: offset2 + length2 = 47,007 + 1,849 = 48,856. File size = 48,856. They match — no trailing unsigned bytes. The signature covers the entire file (minus the gap).

2.7 Common Byte Range Problems

2.7.1 Problem 1: Trailing Bytes After Signed Region

Symptom: Signature validates cryptographically, but the file contains content not covered by the signature.

Cause: Something was appended to the file after signing — possibly a legitimate incremental update (second signature, form fill), or possibly an Incremental Saving Attack.

Detection:

```
int signedEnd = byteRange[2] + byteRange[3];
int fileSize = pdfBytes.length;
if (signedEnd < fileSize) {
    int trailingBytes = fileSize - signedEnd;
    log.warn("{} bytes of unsigned data after signed region", trailingBytes);
    // Analyze the trailing data to determine if it's a legitimate revision
}
```

Resolution: Parse the trailing data. If it contains a valid cross-reference section and trailer pointing to a subsequent signature, it's likely a legitimate multi-signature document. If it contains modified objects (changed page content, replaced fonts, new pages), it may be an attack. Chapter 24 provides detailed analysis code.

2.7.2 Problem 2: `/Contents` Size Exceeded

Symptom: `IOException: Can't write signature, not enough space` during signing.

Cause: The CMS `SignedData` (after hex encoding) exceeds the reserved `/Contents` space.

Fix: Increase `SignatureOptions.setPreferredSignatureSize()`. See the size estimation section above for recommended values by PAdES profile.

2.7.3 Problem 3: ByteRange Doesn't Start at Zero

Symptom: Validator reports "invalid byte range" or "signature does not cover document header."

Cause: Extremely rare — would indicate a buggy signing library. The spec requires that the signed region start at byte 0 to cover the PDF header.

Resolution: This is unfixable after the fact — the PDF must be re-signed.

2.7.4 Problem 4: Gap Contains More Than Just `/Contents`

Symptom: Validator reports "unexpected data in signature gap" or hash mismatch.

Cause: The bytes between the two signed segments should contain *only* the `/Contents` hex string. If any other data exists in this gap — for example, if a tool inserted extra whitespace or a stray dictionary key — the byte range geometry is broken.

Detection: Parse the gap region and verify it matches the pattern `<[0-9A-Fa-f]+>` with optional zero padding.

2.7.5 Problem 5: Hash Algorithm Mismatch

Symptom: Computed hash doesn't match CMS `messageDigest` attribute, even though the byte range looks correct.

Cause: You're hashing with a different algorithm than the one specified in the CMS. The CMS `SignerInfo.digestAlgorithm` tells you which algorithm was used — it might be SHA-256, SHA-384, SHA-512, or (for legacy documents) SHA-1.

Fix: Always read the digest algorithm from the CMS before computing the hash:

```
SignerInformation signer = cmsData.getSignerInfos().getSigners().iterator().next();
String digestAlgOid = signer.getDigestAlgOID();
// Map OID to algorithm name
String algName = switch (digestAlgOid) {
    case "2.16.840.1.101.3.4.2.1" -> "SHA-256";
    case "2.16.840.1.101.3.4.2.2" -> "SHA-384";
    case "2.16.840.1.101.3.4.2.3" -> "SHA-512";
    case "1.3.14.3.2.26"          -> "SHA-1";   // deprecated
    default -> throw new IllegalArgumentException("Unknown digest OID: " + digestAlgOid);
};
MessageDigest digest = MessageDigest.getInstance(algName);
```

2.8 Key Takeaways

- The byte range divides a signed PDF into exactly **two signed segments** with **one unsigned gap** between them

- The gap contains only the hex-encoded CMS signature value (the `/Contents` string) — nothing else
- `offset2 + length2` must equal the file size; any trailing bytes are unsigned and potentially malicious
- Each signature in a multi-signature document covers all previous signatures, creating a nested coverage model
- Always reserve generous space for `/Contents` — 64 KB is a safe default for most PAdES profiles
- The `/ByteRange` array itself is signed — it cannot be modified after signing without detection
- Byte range verification is the **first step** of signature validation — if it fails, there is no need to check the CMS
- The Incremental Saving Attack (ISA) exploits the trailing bytes problem; the trailing bytes check is your primary defense

Chapter 3

Approval vs. Certification Signatures

PDF digital signatures come in two fundamentally different flavors, and confusing them is one of the most common mistakes in signature implementation. An **approval signature** says ”I have reviewed and agree with this document.” A **certification signature** says ”I am the author of this document, and I am defining what changes are allowed after I sign it.”

The difference is not just semantic — it is structural. A certification signature carries a `/DocMDP` (Modification Detection and Prevention) transform that defines permission levels, restricts what subsequent signers and form fillers can do, and triggers different validation behavior in viewers. Getting this wrong means your signatures either restrict too much (form fields that can’t be filled after signing) or too little (no protection against unauthorized modifications).

This chapter explains both signature types, the DocMDP permission model, Lock and FieldMDP dictionaries, and the interaction rules that govern what happens when multiple signatures coexist.

3.1 The Two Signature Types

3.1.1 Approval Signatures

An **approval signature** is what most people think of when they hear ”digital signature.” It means: ”I, the signer, have reviewed the document and give my approval.”

Technically, an approval signature is a `/Sig` dictionary with:

- `/Filter`: `/Adobe.PPKLite`
- `/SubFilter`: `adbe.pkcs7.detached` or `ETSI.CAdES.detached`
- `/ByteRange`: covers the document up to this point
- No `/TransformMethod` in any associated signature reference dictionary

Approval signatures have these properties:

- **Multiple allowed**: A document can have any number of approval signatures. Each one adds an incremental update.
- **No permission restrictions**: An approval signature does not limit what subsequent users can do with the document. After you apply an approval signature, someone else can add another signature, fill form fields, add annotations, or even modify the document (though modifications will cause the signature to show as ”modified since signing”).
- **Order matters**: Each approval signature covers everything up to the point it was applied. Signature 2 covers signature 1. If the document is modified between signature

1 and signature 2, signature 1 will show as ”document modified” while signature 2 will show as ”valid” (because it was applied after the modification).

3.1.2 Certification Signatures

A **certification signature** (also called an ”author signature” or ”MDP signature”) is structurally different. It carries a `/TransformMethod` of `/DocMDP` in its signature reference dictionary, which embeds a set of permission rules into the signature.

Technically, a certification signature includes:

- Everything an approval signature has
- A `/Reference` array in the signature dictionary containing a `/SigRef` (Signature Reference) dictionary
- The `/SigRef` dictionary specifies `/TransformMethod /DocMDP`
- A `/TransformParams` dictionary with a `/P` key (the permission level)

And at the document level:

- The document catalog’s `/Perms` dictionary includes a `/DocMDP` entry pointing to the certifying signature

Certification signatures have these properties:

- **Only one per document**: A document can have at most one certification signature. It is always the first signature applied. If you try to add a certification signature to a document that already has one, the behavior is undefined — some viewers will reject it, others will ignore the previous one.
- **Defines permissions**: The `/P` value (1, 2, or 3) controls what changes are allowed after certification. This is the defining feature that makes certification signatures different from approval signatures.
- **First in the chain**: The certification signature is always the foundational signature. All subsequent signatures are approval signatures that build on the certified baseline.

```
<< /Type /Sig
   /Filter /Adobe.PPKLite
   /SubFilter /adbe.pkcs7.detached
   /ByteRange [0 14231 47007 1849]
   /Contents <3082...>
   /Reference [
     << /Type /SigRef
        /TransformMethod /DocMDP
        /TransformParams << /Type /TransformParams /P 2 /V /1.2 >>
     >>
   ]
   /Name (Document Author)
   /M (D:20260308120000+01'00')
>>
```

The document catalog connects this signature to the document-level permission system:

```
<< /Type /Catalog
```

```
   /AcroForm << ... >>
   /Perms << /DocMDP 15 0 R >>    % Reference to the certifying signature object
   /Pages 3 0 R
>>
```

3.2 The DocMDP Permission Model

3.2.1 Permission Level 1: No Changes Allowed

`/P 1` is the strictest level. After certification with P=1, **no changes whatsoever are allowed** to the document. This means:

- No form fields can be filled
- No additional signatures can be added
- No annotations can be created
- No pages can be added or removed
- Any incremental update invalidates the certification

This level is appropriate for documents that are complete and final — archived records, published regulations, completed certificates. In practice, it is rarely used because most signing workflows require at least form filling or countersigning.

P=1 is so restrictive that it prevents the document from being countersigned. If your workflow requires multiple signatures (author signs, then reviewer countersigns), **do not use P=1**. Use P=2 instead — it allows additional signatures while preventing content modifications.

3.2.2 Permission Level 2: Form Filling and Signing Only

`/P 2` is the **default and most common** permission level. After certification with P=2:

- **Allowed**: Filling form fields, applying additional (approval) signatures, adding signature appearances
- **Not allowed**: Modifying page content, adding pages, removing pages, changing form field definitions (adding/removing fields), modifying JavaScript actions
- Any change beyond form filling and signing invalidates the certification

P=2 is the standard for government forms, contracts, and regulated documents. The document author sets up the form (fields, layout, instructions), certifies it, and then the form recipient fills in the fields and applies their own approval signature — without being able to alter the form structure or content.

This is the level you should use unless you have a specific reason to choose P=1 or P=3.

3.2.3 Permission Level 3: Form Filling, Signing, and Annotations

`/P 3` is the most permissive certification level. After certification with P=3:

- **Allowed**: Everything P=2 allows, plus adding annotations (comments, highlights, stamps, sticky notes, text markup)
- **Not allowed**: Modifying page content, adding or removing pages, changing form field definitions

P=3 is appropriate for review workflows where multiple people need to comment on a certified document without altering its content. Think of a contract that needs review comments from legal, compliance, and management before final signatures.

3.2.4 What "Invalidates the Certification" Actually Means

When we say a change "invalidates the certification," we mean:

- The **byte range** of the certification signature remains intact — the hash still matches
- The **CMS signature** still verifies cryptographically
- But a **conforming viewer** detects that the modifications in subsequent revisions exceed the allowed permissions and displays a warning

In Adobe Acrobat, this shows as: "The document has been altered or corrupted since the Signature was applied." In LibreOffice, it shows as: "Document has been modified since it was last signed."

The exact wording varies by viewer, and — this is important — **not all viewers enforce MDP permissions correctly**. Some viewers only check whether the byte range is intact and don't analyze the incremental updates for permission compliance. This is one of the weaknesses that the Bochum team exploited in the Incremental Saving Attack (Chapter 24).

3.3 Lock Dictionaries: Field-Level Protection

3.3.1 What Lock Dictionaries Do

Beyond document-level MDP permissions, PDF supports **field-level locking** via the `/Lock` dictionary on a signature field. A Lock dictionary specifies which form fields should be locked (made read-only) once this particular signature is applied.

The Lock dictionary lives in the signature field dictionary (not the signature value dictionary), and it applies when the field is signed:

```
<< /Type /Annot
   /Subtype /Widget
   /FT /Sig
   /T (SignatureField1)
   /V 15 0 R               % Reference to the /Sig dictionary
   /Lock <<
     /Type /SigFieldLock
     /Action /Include       % or /Exclude or /All
     /Fields [(FieldA) (FieldB)]
   >>
>>
```

3.3.2 Lock Actions

Table 3.1: Signature Field Lock Actions (ISO 32000-2:2020 §12.8.2.4)

Action	Type	Req	Description
`/All`	name	—	Lock all fields in the document. No `/Fields` array is needed. After this signature is applied, every form field in the document becomes read-only. This is the simplest but most restrictive lock.
`/Include`	name	—	Lock only the fields listed in the `/Fields` array. The listed fields become read-only; all other fields remain fillable. Use this when you want to protect specific fields (e.g., ”Price” and ”Total”) while allowing others (e.g., ”Customer Signature”) to be filled.
`/Exclude`	name	—	Lock all fields **except** those listed in the `/Fields` array. The listed fields remain fillable; everything else is locked. Use this when most fields should be locked but a few specific fields must remain editable (e.g., a ”Date Received” field that an administrator fills after the signer has signed).

3.3.3 The `/Fields` Array

The `/Fields` array in the Lock dictionary contains the **fully qualified field names** of the fields to include or exclude. In a hierarchical field tree, the fully qualified name is the concatenation of parent and child names separated by periods:

```
/Fields [(PersonalInfo.FirstName) (PersonalInfo.LastName) (Signature2)]
```

Field name matching is case-sensitive and must be exact. A typo in the Lock dictionary’s `/Fields` array means the field won’t be locked — and there will be no error message. The signature applies successfully, the Lock dictionary is present, but the misspelled field remains editable.

Always test your Lock configurations by:

1. Signing the document with the Lock dictionary
2. Opening it in Adobe Acrobat
3. Attempting to edit the fields that should be locked
4. Verifying that Acrobat prevents editing and shows the fields as read-only

3.4 FieldMDP: Fine-Grained Modification Detection

3.4.1 FieldMDP vs. DocMDP

While DocMDP controls what categories of changes are allowed (form filling, annotations, etc.), **FieldMDP** controls which specific fields can be modified. It is a more targeted mecha-

nism.

FieldMDP uses the same `/Reference` array mechanism as DocMDP but with `/TransformMethod /FieldMDP`:

```
<< /Type /SigRef
   /TransformMethod /FieldMDP
   /TransformParams <<
     /Type /TransformParams
     /Action /Include
     /Fields [(ProtectedField1) (ProtectedField2)]
     /V /2.0
   >>
>>
```

The `/Action` and `/Fields` keys work identically to Lock dictionary actions — `/Include`, `/Exclude`, and `/All` with the same semantics.

The difference from Lock dictionaries:

- **Lock dictionary**: Controls whether a field is **editable** in the viewer UI (read-only vs. writable). It's a presentation-level restriction — the viewer *should* prevent the user from typing in a locked field.
- **FieldMDP**: Controls whether modifications to specific fields are **detected** during validation. If a FieldMDP-protected field is changed after signing, the signature shows as modified. It's a validation-level mechanism.

In practice, Lock and FieldMDP are often used together: Lock prevents casual editing in a UI, and FieldMDP ensures that any bypass of the Lock (via direct file manipulation) is detected during validation.

3.5 Interaction Rules: When Multiple Signatures Coexist

3.5.1 The Certification + Approval Pattern

The most common multi-signature pattern is:

1. **Author creates** the document with form fields
2. **Author certifies** with P=2 (form filling and signing allowed) and optionally locks specific fields
3. **Recipient fills** form fields
4. **Recipient applies** an approval signature (which adds an incremental update with the filled values and the new signature)

This works because:

- The certification (P=2) allows form filling and additional signatures
- The approval signature is applied to a document that already has a certification
- The certification signature remains valid because the changes (form fill + new signature) are within the P=2 permissions

3.5.2 What Happens When Permissions Conflict

Things get complicated when:

Scenario 1: Approval signature before certification

Technically, any signature can be applied to any document. If someone applies an approval signature first and then a certification signature second, the certification signature covers (signs over) the approval signature. But the certification's MDP permissions only apply to changes *after* the certification — they don't retroactively validate the approval signature's changes.

In practice, this scenario is unusual and may confuse validators. **Always apply the certification signature first.**

Scenario 2: Lock dictionary conflicts with MDP

If the certification says P=2 (form filling allowed) but a Lock dictionary locks all fields — the Lock takes precedence for the locked fields. The MDP permission defines the ceiling (what's *possible*), and Lock dictionaries further restrict within that ceiling.

Scenario 3: Multiple Lock dictionaries from different signatures

Each signature field can have its own Lock dictionary. When multiple signatures with Lock dictionaries coexist:

- Locks are cumulative — each new signature can lock additional fields
- Locks are permanent — once a field is locked by any signature, no subsequent signature can unlock it
- A field is locked if *any* Lock dictionary in the chain locks it

This means the first signer can lock fields A and B, the second signer can lock fields C and D, and after both signatures are applied, fields A, B, C, and D are all locked.

```
// Signature 1 (Author): Lock fields A, B
SignatureField1.Lock = { Action: Include, Fields: [A, B] }

// After Sig1: A=locked, B=locked, C=editable, D=editable

// Signature 2 (Reviewer): Lock fields C, D
SignatureField2.Lock = { Action: Include, Fields: [C, D] }

// After Sig2: A=locked, B=locked, C=locked, D=locked
```

3.6 Implementation: Certification Signatures with PDFBox

3.6.1 Creating a Certification Signature

```
PDDocument doc = Loader.loadPDF(inputFile);

// Create the signature
PDSignature signature = new PDSignature();
signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
signature.setSubFilter(PDSignature.SUBFILTER_ADBE_PKCS7_DETACHED);
```

```
signature.setName("Document Author");
signature.setSignDate(Calendar.getInstance());

// Set certification (MDP) permissions — P=2 is recommended
doc.addSignature(signature, signingInterface);

// Set the access permissions
// 2 = form filling and signing allowed
AccessPermission accessPermission = new AccessPermission();
accessPermission.setDocMDP(2);
doc.setDocumentPermissions(signature, accessPermission);

doc.saveIncremental(outputStream);
doc.close();
```

PDFBox's API for certification signatures evolved between versions. In PDFBox 2.x, you used `doc.addSignature()` with a `SignatureOptions` and separately configured the MDP via the catalog's `/Perms` dictionary. In PDFBox 3.x, the `AccessPermission` class provides a cleaner API, but the underlying mechanism is the same — writing the `/DocMDP` reference into the catalog.

3.6.2 Adding Lock Dictionaries

Lock dictionaries are configured on the signature field before signing:

```
// Get or create the signature field
PDSignatureField sigField = findOrCreateSignatureField(doc, "AuthorSignature");

// Create lock dictionary
COSDictionary lockDict = new COSDictionary();
lockDict.setName(COSName.TYPE, "SigFieldLock");
lockDict.setName(COSName.getPDFName("Action"), "Include");

COSArray fieldsToLock = new COSArray();
fieldsToLock.add(new COSString("PriceField"));
fieldsToLock.add(new COSString("TotalField"));
fieldsToLock.add(new COSString("ContractTerms"));
lockDict.setItem(COSName.getPDFName("Fields"), fieldsToLock);

// Attach lock dictionary to the signature field
sigField.getCOSObject().setItem(COSName.getPDFName("Lock"), lockDict);

// Now sign the document
doc.addSignature(signature, signingInterface);
doc.saveIncremental(outputStream);
```

3.7 Seed Value Dictionaries: Constraining How a Signature Field Is Signed

3.7.1 Purpose of Seed Values

A **seed value dictionary** (`/SV`) on a signature field constrains *how* the field can be signed. It does not sign anything itself — it provides instructions to the signing tool about what

certificate to use, what algorithm to use, which SubFilter is required, and other signing parameters.

This is the mechanism that government form creators use to ensure that form fillers sign with the correct certificate type, use the correct algorithm, and produce signatures that will pass validation.

3.7.2 Key Seed Value Entries

Table 3.2: Seed Value Dictionary Keys (ISO 32000-2:2020 §12.8.2.3)

Key	Type	Req	Spec Ref
`/Filter`	name	Optional	§12.8.2.3, Table 237
The signature handler to use. In practice, always `/Adobe.PPKLite`. If set, the signing tool should use this handler — if a different handler is selected, the signing tool should warn the user.			
`/SubFilter`	array	Optional	§12.8.2.3, Table 237
Array of acceptable SubFilter values. The signing tool should use one of these SubFilter values. Example: `[/adbe.pkcs7.detached /ETSI.CAdES.detached]` means either detached format is acceptable.			
`/DigestMethod`	array	Optional	§12.8.2.3, Table 237
Array of acceptable digest algorithms. Example: `[/SHA256 /SHA384 /SHA512]`. If set, the signing tool must use one of these algorithms. `/SHA1` is deprecated and should not be included.			
`/Reasons`	array	Optional	§12.8.2.3, Table 237
Array of text strings presenting acceptable signing reasons. The signing tool should let the user choose from this list. Example: `[(I approve this document) (I have reviewed this document) (I am the authorized signatory)]`.			
`/MDP`	dictionary	Optional	§12.8.2.3, Table 237
The MDP parameters dictionary, specifying the default DocMDP permission level for signatures applied to this field.			
`/TimeStamp`	dictionary	Optional	§12.8.2.3, Table 237
Timestamp seed value dictionary specifying a timestamp authority URL and whether a timestamp is required. If the `/Ff` flag is set to 1 (required), the signing tool must include a timestamp from the specified TSA.			
`/Ff`	integer (flags)	Optional	§12.8.2.3, Table 237
Flags indicating which seed values are mandatory (bit 1 set) vs. recommended (bit 1 clear). If a seed value's corresponding Ff bit is set, the signing tool must respect the constraint — failure to do so should prevent the signature from being applied.			
`/LegalAttestation`	array	Optional	§12.8.2.3
Array of legal attestation strings. The signer may be required to accept one of these attestations as part of the signing ceremony.			

3.7.3 Timestamp Seed Values

The `/TimeStamp` sub-dictionary in seed values is particularly important for PAdES compliance:

```
/SV <<
  /TimeStamp <<
    /URL (http://tsa.example.com/timestamp)
    /Ff 1              % 1 = timestamp required, 0 = optional
  >>
  /SubFilter [/ETSI.CAdES.detached]
  /DigestMethod [/SHA256 /SHA384 /SHA512]
  /Ff 1                % All seed values are mandatory
>>
```

This seed value dictionary says: ”When signing this field, you must use CAdES detached format, SHA-256 or better, and you must include a timestamp from `tsa.example.com`. All of these requirements are mandatory (Ff=1).”

A conforming signing tool (Adobe Acrobat, for example) will enforce these constraints automatically. PDFBox-based signing code must read and respect these seed values manually — the library does not enforce them automatically.

Always read the seed value dictionary before signing. If you’re building a signing service that signs arbitrary PDFs, the seed values may impose constraints that your default configuration doesn’t meet. Ignoring them produces a signature that appears valid cryptographically but violates the form author’s intent — and may fail validation in systems that check seed value compliance.

```
PDSignatureField sigField = ...; // the field you're about to sign
COSDictionary sv = sigField.getCOSObject()
    .getCOSDictionary(COSName.getPDFName("SV"));
if (sv != null) {
    // Check SubFilter requirements
    COSArray subFilters = sv.getCOSArray(COSName.getPDFName("SubFilter"));
    // Check DigestMethod requirements
    COSArray digestMethods = sv.getCOSArray(COSName.getPDFName("DigestMethod"));
    // Check Timestamp requirements
    COSDictionary ts = sv.getCOSDictionary(COSName.getPDFName("TimeStamp"));
    if (ts != null) {
        String tsaUrl = ts.getString(COSName.getPDFName("URL"));
        int tsFlags = ts.getInt(COSName.getPDFName("Ff"), 0);
        boolean timestampRequired = (tsFlags & 1) != 0;
    }
}
```

3.8 Real-World Patterns and Mistakes

3.8.1 Mistake 1: Using Certification When Approval Is Needed

Scenario: A developer creates a multi-signature workflow where three people sign sequentially. They make the first signature a certification signature with P=1 (no changes allowed).

Result: The second and third signatures fail or invalidate the certification, because P=1

prohibits additional signatures.

Fix: Use P=2 for the first (certification) signature if additional signatures are needed. Or use approval signatures for all three if no MDP restrictions are needed.

3.8.2 Mistake 2: Not Setting MDP When It's Needed

Scenario: A government form is meant to be filled and signed by the citizen, with no other modifications allowed. The form creator signs it as a plain approval signature without MDP.

Result: The form has no modification restrictions. Anyone can modify the form structure, add fields, remove fields, or alter content — and the original signature only indicates "document has been modified" without specifying what was allowed.

Fix: Use a certification signature with P=2 to allow form filling while preventing structural changes.

3.8.3 Mistake 3: Lock Dictionary with Wrong Field Names

Scenario: The signature is configured to lock fields "Price", "Total", and "Terms". But the actual field names in the PDF are "price_field", "total_amount", and "contract_terms" (because the form designer used different naming).

Result: The Lock dictionary is present and syntactically correct, but no fields are actually locked — the names don't match. There is no error message or warning.

Fix: Enumerate all field names before configuring the Lock dictionary. In PDFBox:

```
PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();
for (PDField field : acroForm.getFieldTree()) {
    log.info("Field: {} (type: {})",
             field.getFullyQualifiedName(), field.getFieldType());
}
```

3.8.4 Mistake 4: Assuming All Viewers Enforce MDP

Scenario: A developer relies on MDP P=2 to prevent content modifications, assuming all viewers will refuse to let users edit non-form content after certification.

Result: Adobe Acrobat enforces MDP correctly. Foxit enforces it mostly correctly. LibreOffice enforces it partially. PDFBox (when used as a library) does not enforce MDP at all — it allows any modification programmatically. Browser-based PDF viewers (Chrome, Firefox) often ignore MDP entirely because they don't support editing.

Lesson: MDP is a *validation* mechanism, not an *access control* mechanism. It tells you whether changes were permitted, not whether they were prevented. A determined user with a hex editor or a programmatic tool can always modify a PDF — MDP ensures you can *detect* that the modification occurred.

3.9 Key Takeaways

- **Approval signatures**: "I approve this document." Multiple allowed, no restrictions on subsequent changes, each covers all previous content.

- **Certification signatures**: ”I authored this document and define what can change.” Only one per document, must be first, carries DocMDP permissions (P=1/2/3).
- **P=2 is the default and most common** permission level — allows form filling and additional signatures while preventing content modifications.
- **Lock dictionaries** control which fields become read-only after a specific signature is applied. Cumulative and permanent.
- **FieldMDP** provides validation-level detection of changes to specific fields — complements Lock dictionaries.
- **Seed value dictionaries** constrain how a signature field must be signed — SubFilter, digest algorithm, timestamp requirements.
- MDP is a **detection mechanism**, not a prevention mechanism. Not all viewers enforce it. Always verify compliance programmatically.
- **Always apply certification signatures first**, then approval signatures. The certification establishes the permission framework.

Chapter 4

Visual Appearance of Signatures

A digital signature has two faces. The cryptographic face — the CMS `SignedData`, the byte range, the certificate chain — is what makes the signature *valid*. The visual face — what the user sees in the PDF viewer — is what makes the signature *trustworthy to a human reader*. These two faces are entirely independent, and understanding their independence is critical for both security and usability.

The visual appearance of a PDF signature is an **annotation appearance stream** — a miniature PDF drawing that shows the signer's name, date, reason, and optionally a handwritten signature image, a company logo, or other graphical elements. This appearance stream is part of the signed bytes (it lives inside the signature field's widget annotation), but it is *not* cryptographically bound to the CMS in any meaningful way. A valid CMS signature does not guarantee that the visual appearance accurately represents the signer — it only guarantees that the bytes haven't changed.

This distinction is exactly what Shadow Attacks (Chapter 24) exploit: an attacker can create a PDF where the visual appearance shows one thing to the signer and something different to the recipient, with the signature validating correctly in both cases.

This chapter explains how signature appearances work, how to create them with PDFBox, what the layer model means, and how different viewers handle appearance rendering.

4.1 Appearance Streams: The Basics

4.1.1 What an Appearance Stream Is

Every annotation in a PDF — not just signatures — can have an **appearance stream**. An appearance stream is a Form XObject (a `/Type /XObject /Subtype /Form` stream) that contains the drawing instructions for how the annotation looks.

For a signature widget annotation, the appearance stream is stored under the `/AP` (Appearance) key:

```
<< /Type /Annot
   /Subtype /Widget
   /FT /Sig
   /T (SignatureField1)
   /V 15 0 R                    % the /Sig dictionary
   /Rect [100 600 300 650]      % position on the page (x1 y1 x2 y2)
   /AP <<
     /N 20 0 R                  % Normal appearance stream
   >>
>>
```

The `/N` key holds the **normal** appearance — the one shown when the annotation is not being interacted with. Annotations can also have `/R` (rollover — shown on mouse hover) and `/D` (down — shown when clicked), but signature appearances only use `/N`.

The appearance stream (object 20 in this example) is a Form XObject:

```
20 0 obj
<< /Type /XObject
   /Subtype /Form
   /BBox [0 0 200 50]          % bounding box in the stream's coordinate space
   /Resources << /Font << /F1 21 0 R >> >>
   /Length 142
>>
stream
BT
/F1 10 Tf
0 35 Td
(Digitally signed by: Test Signer) Tj
0 -15 Td
(Date: 2026-03-08 12:00:00 UTC) Tj
0 -15 Td
(Reason: I approve this document) Tj
ET
endstream
endobj
```

This is a complete miniature PDF page: it has a bounding box, font resources, and drawing instructions using the standard PDF content stream operators (`BT`/`ET` for text blocks, `Tf` for font selection, `Td` for positioning, `Tj` for showing text).

4.1.2 Visible vs. Invisible Signatures

Not all signatures need a visual appearance. A signature can be:

Visible: The signature field has a non-zero `/Rect` (annotation rectangle) and an appearance stream. The user sees a signature block on a specific page.

Invisible: The signature field either has a zero-size `/Rect` (`[0 0 0 0]`), or has no appearance stream, or is not associated with any page. The signature exists cryptographically but doesn't appear on any page.

Invisible signatures are common in automated signing workflows where the PDF is being signed programmatically (e.g., a server signing all outgoing invoices) and there's no need for a visual indicator on the document. The signature is still listed in the viewer's signature panel — the user can still inspect it, verify the certificate, and check the validation status.

Even for invisible signatures, some viewers (notably Adobe Acrobat) display a blue banner at the top of the document saying "Signed and all signatures are valid" or similar. This banner is not part of the PDF content — it's the viewer's UI.

4.2 The Layer Model: n0, n2, and n4

4.2.1 Adobe's Legacy Layer System

Adobe Acrobat uses a layer-based approach to compose signature appearances. This system was designed by Adobe for their own implementation and is not formally part of the ISO 32000 specification — but because Adobe Acrobat is the dominant viewer, understanding it matters.

The layers are:

- **n0** (Layer 0 — Background): Contains the background graphic and a static validity indicator. Adobe Acrobat uses this to show the blue "valid" checkmark or the red "invalid" X. This layer is generated and updated by Acrobat itself — it is not typically set by the signing application.
- **n2** (Layer 2 — Content): Contains the signer's information — name, date, reason, location, and optionally a handwritten signature image or company logo. **This is the layer you create** when building a visual signature appearance.
- **n4** (Layer 4 — Text overlay): An optional text overlay, rarely used.

In practice, most signing applications (including PDFBox-based implementations) create a single appearance stream that serves as the entire appearance — they don't use the n0/n2/n4 layer decomposition. Adobe Acrobat, when it encounters a signature without layer-based appearances, creates its own composite appearance using the provided stream as the content layer and adding its own validity indicators.

The layer model is an implementation detail of Adobe Acrobat, not a spec requirement. When you create a signature appearance with PDFBox, you create a single Form XObject for the `/AP /N` entry. Acrobat will overlay its own validity graphics on top. Other viewers (LibreOffice, Foxit, browser-based viewers) each handle the appearance differently — some show your appearance as-is, some add their own decorations.

4.2.2 What This Means for Your Implementation

When building a signature appearance with PDFBox:

1. **Create a single Form XObject** with the signer's information
2. **Don't worry about n0/n2/n4 layers** — the single appearance works across all viewers
3. **Set reasonable fonts and sizes** — the appearance will be rendered in the space defined by the signature field's `/Rect`
4. **Include key information**: signer's name, signing date/time, reason (if applicable)
5. **Test in multiple viewers** — Acrobat, Foxit, LibreOffice, and a browser-based viewer — to ensure the appearance renders correctly in each

4.3 Creating Signature Appearances with PDFBox

4.3.1 Basic Text Appearance

The simplest appearance shows the signer's name and date:

```
private PDFormXObject createSignatureAppearance(PDDocument doc,
        PDRectangle rect, String signerName, Calendar signDate)
        throws IOException {

    PDFormXObject form = new PDFormXObject(doc.getDocument().createCOSStream());
    form.setResources(new PDResources());
    form.setBBox(new PDRectangle(rect.getWidth(), rect.getHeight()));

    PDFont font = new PDType1Font(Standard14Fonts.FontName.HELVETICA);
    form.getResources().put(COSName.getPDFName("F1"), font);

    float fontSize = 8;
    String dateStr = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss z")
            .format(signDate.getTime());

    // Build the content stream manually
    try (OutputStream os = form.getStream().createOutputStream()) {
        String content = String.join("\n",
            "BT",
            "/F1 " + fontSize + " Tf",
            "0 " + (rect.getHeight() - fontSize - 2) + " Td",
            "(" + escapeForPdf(signerName) + ") Tj",
            "0 -" + (fontSize + 2) + " Td",
            "(Date: " + escapeForPdf(dateStr) + ") Tj",
            "ET"
        );
        os.write(content.getBytes(StandardCharsets.US_ASCII));
    }

    return form;
}

private String escapeForPdf(String text) {
    return text.replace("\\", "\\\\")
               .replace("(", "\\(")
               .replace(")", "\\)");
}
```

In PDFBox 3.x, you **cannot** use `PDPageContentStream` to write to a Form XObject. `PDPageContentStream` is strictly for page content. For Form XObjects, you must write raw PDF operators to the stream's output stream, as shown above.

This is one of the breaking changes from PDFBox 2.x, where `PDPageContentStream` could be used with Form XObjects via a constructor overload that accepted a `PDFormXObject` parameter. That constructor no longer exists in 3.x.

4.3.2 Appearance with Signature Image

Many signing workflows include a handwritten signature image — either captured via a canvas on a web page or tablet, or uploaded as a PNG/JPEG file. Adding an image to the appearance requires embedding it as a PDF image XObject:

```
private PDFormXObject createAppearanceWithImage(PDDocument doc,
        PDRectangle rect, String signerName, Calendar signDate,
        byte[] signatureImagePng) throws IOException {
```

```
    PDFormXObject form = new PDFormXObject(doc.getDocument().createCOSStream());
    form.setResources(new PDResources());
    form.setBBox(new PDRectangle(rect.getWidth(), rect.getHeight()));

    // Add font
    PDFont font = new PDType1Font(Standard14Fonts.FontName.HELVETICA);
    form.getResources().put(COSName.getPDFName("F1"), font);

    // Add signature image
    PDImageXObject image = PDImageXObject.createFromByteArray(
            doc, signatureImagePng, "signature");
    form.getResources().put(COSName.getPDFName("Img0"), image);

    // Layout: image on the left, text on the right
    float imgWidth = rect.getWidth() * 0.4f;
    float imgHeight = rect.getHeight() - 4;
    float textX = imgWidth + 5;
    float fontSize = 7;
    String dateStr = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss z")
            .format(signDate.getTime());

    try (OutputStream os = form.getStream().createOutputStream()) {
        StringBuilder sb = new StringBuilder();
        // Draw image
        sb.append("q\n");  // save graphics state
        sb.append(imgWidth).append(" 0 0 ").append(imgHeight)
          .append(" 2 2 cm\n");  // transform matrix
        sb.append("/Img0 Do\n");  // draw image
        sb.append("Q\n");  // restore graphics state
        // Draw text
        sb.append("BT\n");
        sb.append("/F1 ").append(fontSize).append(" Tf\n");
        sb.append(textX).append(" ")
          .append(rect.getHeight() - fontSize - 2).append(" Td\n");
        sb.append("(Digitally signed by:) Tj\n");
        sb.append("0 -").append(fontSize + 2).append(" Td\n");
        sb.append("(").append(escapeForPdf(signerName)).append(") Tj\n");
        sb.append("0 -").append(fontSize + 2).append(" Td\n");
        sb.append("(Date: ").append(escapeForPdf(dateStr)).append(") Tj\n");
        sb.append("ET\n");
        os.write(sb.toString().getBytes(StandardCharsets.US_ASCII));
    }

    return form;
}
```

4.3.3 Attaching the Appearance to the Signature

After creating the appearance Form XObject, attach it to the signature field's widget annotation:

```
// Create signature field and widget annotation
PDSignature signature = new PDSignature();
// ... set signature properties ...
```

```
// Set the visual rectangle on page 1
PDPage page = doc.getPage(0);
PDRectangle rect = new PDRectangle(100, 600, 200, 50);

SignatureOptions options = new SignatureOptions();
options.setPreferredSignatureSize(65536);
options.setPage(0);  // page index (0-based)
options.setVisualSignature(
    createSignatureAppearance(doc, rect, "Test Signer", Calendar.getInstance()));

doc.addSignature(signature, signingInterface, options);
doc.saveIncremental(outputStream);
```

4.4 Appearance Rendering Across Viewers

4.4.1 How Different Viewers Handle Appearances

One of the most frustrating aspects of PDF signature appearances is that different viewers render them differently — even when the appearance stream is identical.

Table 4.1: Signature appearance rendering behavior across major viewers

Viewer	Behavior
Adobe Acrobat	Overlays its own validity indicator (blue checkmark / red X) on layer n0. Your appearance stream is rendered on layer n2. Acrobat may resize your content to fit its layer model. If `/NeedAppearances` is true in the AcroForm, Acrobat may regenerate appearances from scratch.
Foxit Reader	Renders the appearance stream mostly as provided. Adds a small validity icon in the corner. Generally the most faithful rendering of custom appearances.
LibreOffice Draw	Renders the appearance stream as-is. Does not overlay validity indicators on the appearance itself — shows validation status in a separate panel. Good for testing that your raw appearance looks correct.
Chrome (built-in viewer)	Renders appearance streams but does not validate signatures. No validity indicators. The appearance is what you see — useful for checking visual layout.
Firefox (pdf.js)	Renders appearance streams. Does not validate signatures (though experimental validation support has been discussed). Like Chrome, useful for layout testing.
Okular (KDE)	Renders appearances and provides signature validation. May show validity status differently from other viewers.

Always test your signature appearances in at least three viewers — Adobe Acrobat, a non-Adobe desktop viewer (Foxit or LibreOffice), and a browser. Appearances that look

perfect in Acrobat may render differently in other viewers due to:

- Different font fallback behavior (if the font isn't embedded)
- Different handling of the coordinate system and bounding box
- Different overlay/decoration behavior
- Different treatment of transparency and blending

4.4.2 The `/NeedAppearances` Problem

The AcroForm dictionary has a boolean flag `/NeedAppearances` that tells viewers whether to regenerate form field appearances from scratch. If this flag is `true`, some viewers (notably Adobe Acrobat) will ignore the existing appearance streams and generate their own based on the field's value.

For signature fields, this is usually not what you want. If `/NeedAppearances` is true and Acrobat regenerates the signature's appearance, your carefully crafted visual (with the signer's image, custom layout, etc.) gets replaced with Acrobat's default rendering.

Always set `/NeedAppearances` to `false` (or remove the key entirely — the default is `false`) before signing:

```
PDAcroForm acroForm = doc.getDocumentCatalog().getAcroForm();
if (acroForm != null) {
    acroForm.getCOSObject().setBoolean(COSName.NEED_APPEARANCES, false);
}
```

4.5 Positioning: Getting the Rectangle Right

4.5.1 The Coordinate System

PDF uses a **bottom-left origin** coordinate system where (0, 0) is at the lower-left corner of the page. The y-axis goes *up*, not down. This is the opposite of most screen coordinate systems and is a common source of positioning errors.

The signature field's `/Rect` array defines the annotation rectangle as `[x1 y1 x2 y2]`, where:

- `(x1, y1)` is the lower-left corner
- `(x2, y2)` is the upper-right corner

So a signature field at the bottom-right of an A4 page (595 x 842 points) might have:

```
/Rect [395 50 575 100]    % 180pt wide, 50pt tall, near bottom-right
```

A common mistake: specifying the rectangle with screen coordinates (origin at top-left, y going down). If you compute `y = pageHeight - desiredY`, make sure you do this for *both* y1 and y2, and that y1 < y2.

Debug tip: If your signature appearance shows up in the wrong position, check:

1. Is the page rotated? (A `/Rotate` key on the page dictionary changes the visible coordinate system)
2. Is the `MediaBox` the expected size? (Custom page sizes shift everything)
3. Did you flip y1 and y2? (The lower-left corner must have the smaller y value)

4.5.2 Handling Page Rotation

PDF pages can have a `/Rotate` key that specifies a rotation angle (0, 90, 180, or 270 degrees). This rotation affects how the page is *displayed* but does not change the coordinate system of the page content or annotations.

This means that on a 90-degree rotated page (landscape orientation achieved via rotation), the annotation rectangle coordinates are still in the unrotated coordinate space. To place a signature at the visual bottom-right of a rotated page, you need to account for the rotation.

```
PDPage page = doc.getPage(pageIndex);
int rotation = page.getRotation();
PDRectangle mediaBox = page.getMediaBox();

float sigWidth = 200;
float sigHeight = 50;
float margin = 30;

PDRectangle sigRect;
switch (rotation) {
    case 0:   // Normal portrait
        sigRect = new PDRectangle(
            mediaBox.getWidth() - sigWidth - margin,
            margin,
            sigWidth, sigHeight);
        break;
    case 90:  // Landscape (rotated clockwise)
        sigRect = new PDRectangle(
            margin,
            margin,
            sigHeight, sigWidth);  // swap width/height
        break;
    case 180: // Upside-down portrait
        sigRect = new PDRectangle(
            margin,
            mediaBox.getHeight() - sigHeight - margin,
            sigWidth, sigHeight);
        break;
    case 270: // Landscape (rotated counter-clockwise)
        sigRect = new PDRectangle(
            mediaBox.getWidth() - sigHeight - margin,
            mediaBox.getHeight() - sigWidth - margin,
            sigHeight, sigWidth);
        break;
    default:
        sigRect = new PDRectangle(margin, margin, sigWidth, sigHeight);
}
```

4.6 Security Implications of Visual Appearances

4.6.1 The Appearance/Content Disconnect

The visual appearance of a signature is stored in the PDF's object structure, which is covered by the byte range. But the appearance is not cryptographically bound to the *identity* of the

signer. The appearance can show anything — ”Signed by Alice” while the CMS is signed by Bob’s certificate.

A conforming viewer should display the certificate information from the CMS (not from the appearance) in its signature properties panel. But what the user *sees* on the page is the appearance — and most users trust the visual appearance more than the signature properties panel.

This is a deliberate design trade-off in PDF: the signer (or the signing application) controls what the appearance looks like, and the viewer overlays validity information. The validity indicators (checkmarks, status banners) come from the viewer’s own analysis of the CMS — they don’t come from the appearance stream.

4.6.2 Shadow Attacks and Visual Appearances

Shadow Attacks (Chapter 24) exploit the gap between what a signer sees and what the document contains. In the **Replace** variant, the attacker creates a form field with a specific visual appearance, then changes the field’s value (and thus its appearance) after the signer certifies the document. Because the certification allows form filling (P=2 or P=3), the change is permitted — but the visual content of the document has changed dramatically.

The defense is the same as for all Shadow Attacks: validate not just the cryptographic integrity but also the *semantic* integrity — what changed between revisions, and were those changes consistent with the displayed content?

For maximum security: When validating a signed PDF, don’t just check the signature’s validity. Extract the revision that was signed (by truncating the file at the byte range end — see Chapter 13) and compare its visual rendering to the current document. If they differ, the document has been modified since signing — even if the modifications were technically ”permitted” by the MDP level.

4.7 Common Appearance Problems

4.7.1 Problem: Blank Signature Rectangle

Symptom: The signature field shows as an empty box in the viewer.

Causes:

- No appearance stream set (the `/AP` dictionary is missing)
- `/NeedAppearances` is `true`, and the viewer regenerated an empty appearance
- The Form XObject has an empty stream (no drawing instructions)
- The bounding box (`/BBox`) is [0 0 0 0] or has zero dimensions

4.7.2 Problem: Text Appears Clipped or Off-Screen

Symptom: Only part of the signature text is visible, or the text appears outside the signature rectangle.

Causes:

- The text positioning (`Td` operators) exceeds the Form XObject’s `/BBox`

- The font size is too large for the available space
- The `/BBox` doesn't match the annotation's `/Rect` dimensions

Fix: Ensure the Form XObject's `/BBox` width and height match the annotation `/Rect` dimensions, and that all drawing instructions fall within the `/BBox`.

4.7.3 Problem: Font Not Rendering

Symptom: Text appears as squares, dots, or is completely invisible.

Causes:

- The font resource is not declared in the Form XObject's `/Resources`
- The font name in the content stream (`/F1`) doesn't match the resource key
- Non-ASCII characters used with a non-embedded Standard 14 font (Standard 14 fonts only support WinAnsiEncoding)

Fix: For text beyond basic Latin characters (accented letters, CJK, etc.), embed a TrueType or Type0 font. Standard 14 fonts (Helvetica, Times, Courier, etc.) only support a limited character set.

4.8 Key Takeaways

- Signature appearance is an **annotation appearance stream** — a Form XObject with drawing instructions
- The appearance is **visually independent** from the cryptographic signature — they can show different things
- **Invisible signatures** have zero-size rectangles or no appearance stream — they exist cryptographically but don't show on the page
- The **n0/n2/n4 layer model** is Adobe's implementation detail, not a spec requirement — create a single Form XObject for maximum compatibility
- **PDFBox 3.x** requires writing raw PDF operators to Form XObject streams — `PDPageContentStream` only works for pages
- **Always set `/NeedAppearances` to `false`** before signing to prevent viewers from regenerating your custom appearance
- **Test in multiple viewers** — Acrobat, Foxit, LibreOffice, and browsers all render appearances differently
- The **coordinate system** is bottom-left origin with y going up — account for page rotation when positioning
- Visual appearances are a **security surface** — Shadow Attacks exploit the disconnect between appearance and content

Chapter 5

Signature Validation

Signing a PDF is the easy part. Validating a signed PDF — correctly, securely, and in a way that matches what other validators produce — is where the real complexity lives. A signature can fail validation for dozens of reasons, and the error messages from different viewers are often vague or contradictory. Adobe says ”valid.” LibreOffice says ”signature cannot be verified.” Your custom validator throws a `CMSException`. Which one is right?

This chapter walks through the complete validation process, from the first byte range check to the final certificate chain verification. We cover what each validation step checks, what can go wrong at each step, how different viewers handle each step, and how to debug validation failures systematically.

5.1 The Five Steps of Signature Validation

Signature validation is not a single check — it is a pipeline of five independent verifications, each of which must pass. A failure at any step means the signature is invalid (or at least questionable), regardless of whether the other steps pass.

5.1.1 Step 1: Structural Integrity (Byte Range)

Question answered: ”Does the signature cover the expected bytes of the file?”

What to check:

1. The `/ByteRange` array has exactly 4 elements: `[offset1 length1 offset2 length2]`
2. `offset1` is 0 (signature covers from the start of the file)
3. `offset2 + length2` equals the file size (or revision size for multi-signature documents)
4. The gap `[length1, offset2)` contains only the `/Contents` hex string
5. No trailing unsigned bytes exist after the signed region (unless they are a subsequent legitimate revision)

Common failures:

- Trailing bytes after the signed region — possible Incremental Saving Attack (Chapter 24)
- ByteRange doesn’t start at 0 — broken signing implementation
- Gap contains unexpected data — corrupted or manually edited PDF

Chapter 13 covers this step in exhaustive detail.

5.1.2 Step 2: Hash Verification (Digest Match)

Question answered: ”Does the hash of the signed bytes match what the CMS claims was signed?”

What to check:

1. Read the digest algorithm from the CMS `SignerInfo.digestAlgorithm`
2. Compute the hash over the byte range segments: `hash = algo(segment1 || segment2)`
3. Extract the `messageDigest` attribute from the CMS `signedAttrs`
4. Compare the computed hash to the CMS-provided hash

Common failures:

- Hash algorithm mismatch — you computed SHA-256 but the CMS used SHA-384
- Byte range computation error — you read the wrong bytes
- Modified file — someone changed the file after signing

```
// Step 2: Compute and compare hash
String digestAlgOid = signerInfo.getDigestAlgOID();
String algName = mapOidToAlgorithm(digestAlgOid);
MessageDigest digest = MessageDigest.getInstance(algName);

int[] byteRange = signature.getByteRange();
digest.update(pdfBytes, byteRange[0], byteRange[1]);
digest.update(pdfBytes, byteRange[2], byteRange[3]);
byte[] computedHash = digest.digest();

Attribute digestAttr = signerInfo.getSignedAttributes()
        .get(CMSAttributes.messageDigest);
byte[] claimedHash = ASN1OctetString.getInstance(
        digestAttr.getAttrValues().getObjectAt(0)).getOctets();

if (!MessageDigest.isEqual(computedHash, claimedHash)) {
    log.error("Hash mismatch — document was modified after signing");
    return ValidationResult.HASH_MISMATCH;
}
```

5.1.3 Step 3: Cryptographic Verification (CMS Signature)

Question answered: ”Was the CMS `SignedData` actually created by the claimed signer’s private key?”

What to check:

1. Extract the signer’s certificate from the CMS `SignedData.certificates` collection
2. Verify that the `SignerInfo.signatureAlgorithm` matches the certificate’s key type (RSA cert with RSA signature, ECDSA cert with ECDSA signature)
3. Verify the signature over the `signedAttrs` using the signer’s public key
4. Verify that the signed attributes include the required fields: `contentType`, `messageDigest`, and (for PAdES) `ESS-signing-certificate-v2`

Common failures:

- Wrong certificate used for verification — the CMS contains multiple certificates, and you picked an intermediate instead of the signer
- Algorithm mismatch — rare, but can happen with misconfigured BouncyCastle providers
- Missing `ESS-signing-certificate-v2` — not a cryptographic failure, but a PAdES compliance failure

```
// Step 3: CMS cryptographic verification
CMSSignedData cmsData = new CMSSignedData(
        new CMSProcessableByteArray(signedBytes), signature.getContents());

SignerInformationStore signerStore = cmsData.getSignerInfos();
SignerInformation signerInfo = signerStore.getSigners().iterator().next();

// Find the signer's certificate
X509CertificateHolder signerCert = findSignerCertificate(
        cmsData.getCertificates(), signerInfo.getSID());

// Verify the CMS signature
boolean cmsValid = signerInfo.verify(
        new JcaSimpleSignerInfoVerifierBuilder()
                .setProvider("BC")
                .build(signerCert));

if (!cmsValid) {
    log.error("CMS signature verification failed - " +
              "signature was not created by the claimed certificate");
    return ValidationResult.CMS_INVALID;
}
```

5.1.4 Step 4: Certificate Chain Validation

Question answered: ”Is the signer’s certificate trustworthy? Was it valid at the time of signing?”

What to check:

1. Build the certificate chain from the signer’s certificate up to a trusted root
2. Verify each certificate’s signature (each cert is signed by its issuer)
3. Check validity periods — was each certificate valid at the signing time?
4. Check revocation status — was any certificate revoked at or before the signing time?
5. Check key usage — does the signer’s certificate allow digital signatures (`digitalSignature` or `nonRepudiation` key usage)?
6. Check basic constraints — are intermediate certificates properly marked as CA certificates?

Common failures:

- **Self-signed certificate**: The signer’s certificate is not issued by a recognized CA. This is technically valid (the CMS signature will verify) but the certificate is not trusted. Most viewers show ”signature is valid but the identity of the signer could not be verified.”
- **Expired certificate**: The certificate was valid when the signature was created, but has since expired. With timestamps, this is not a problem (the timestamp proves the

signature was created before expiration). Without timestamps, viewers may warn about an expired certificate.

- **Revoked certificate**: A certificate in the chain was revoked (compromised, or the CA issued it in error). Revocation checking requires OCSP or CRL lookup — which may fail if the revocation servers are unreachable.
- **Missing intermediate**: The CMS doesn't include all intermediate certificates, and the validator can't build the chain. This is the most common real-world certificate chain failure.
- **Not in trust store**: The root certificate is not in the validator's trust store. Different validators have different trust stores — Adobe uses AATL, operating systems use their own, and EU validators use the EUTL.

5.1.5 Step 5: Trust Evaluation

Question answered: "Do I trust the entity that signed this document?"

This is the most subjective step, and it's where validators diverge the most. Trust evaluation is not a purely technical question — it depends on context:

Adobe Acrobat trust model: Acrobat trusts certificates that chain to a root in the **Adobe Approved Trust List (AATL)**. If the certificate chains to an AATL root, Acrobat shows a green checkmark. If not — even if the certificate is perfectly valid, issued by a reputable CA, and chains to a root in the operating system's trust store — Acrobat shows a yellow warning: "At least one signature has problems."

This is one of the most frustrating aspects of PDF signature validation for developers. Your signature is cryptographically valid, the certificate is issued by a recognized CA, the certificate is not expired or revoked — but Acrobat shows a warning because the CA is not on Adobe's list. Getting on the AATL requires the CA to apply to Adobe and pay fees.

EU validation model: EU validators (like the DSS demonstration webapp, or validators used by EU government services) trust certificates that chain to a root in the **EU Trusted List (EUTL)**. The EUTL is the authoritative registry of Qualified Trust Service Providers. A signature is a "qualified electronic signature" — with the legal weight of a handwritten signature under eIDAS — only if the certificate chains to an EUTL-listed QTSP.

Operating system trust model: Some applications (like LibreOffice) use the operating system's certificate store for trust evaluation. This means the set of trusted roots varies by OS and by the user's system configuration.

Custom trust model: Your application can define its own trust store — a specific set of root certificates that you trust for your use case. This is common in enterprise environments where the signing certificates are issued by an internal CA.

When debugging trust failures, ask three questions:

1. What trust store is the validator using? (AATL, EUTL, OS store, custom)
2. Is the signer's root certificate in that trust store?
3. Are all intermediates available? (Either embedded in the CMS or fetchable via the certificate's Authority Information Access extension)

The answer to question 1 determines everything else. A signature can be "valid" in one validator and "untrusted" in another, purely because of trust store differences.

5.2 Validation Differences Across Viewers

One of the most confusing aspects of PDF signature validation is that different viewers produce different results for the same PDF. This is not a bug — it's a consequence of the five-step validation model, where each viewer makes different choices at each step.

Table 5.1: Validation behavior comparison across major PDF viewers

Check	Acrobat	Foxit	LibreOffice	PDFBox	EU DSS
Byte range trailing bytes	Warns	Warns	Partial	Manual	Warns
Hash verification	Yes	Yes	Yes	Manual	Yes
CMS signature	Yes	Yes	Yes	Manual	Yes
Certificate chain	AATL	Mixed	OS store	Manual	EUTL
Revocation (OCSP)	Yes	Yes	Limited	Manual	Yes
Revocation (CRL)	Yes	Yes	Yes	Manual	Yes
MDP enforcement	Full	Mostly	Partial	None	Full
Timestamp verification	Yes	Yes	Partial	Manual	Yes
PAdES compliance	Partial	Partial	No	Manual	Full

"Manual" for PDFBox means the library provides the primitives (signature extraction, byte range access, CMS parsing) but does not perform the validation automatically — you must write the validation logic yourself. This is by design: PDFBox is a PDF library, not a validation service.

EU DSS provides the most complete automated validation, including ETSI-compliant validation reports.

5.3 Debugging Validation Failures

5.3.1 The Systematic Approach

When a signature fails validation, resist the urge to guess. Instead, walk through the five steps systematically:

Step 1: Is the byte range intact?

```
# Check byte range with QPDF
qpdf --show-object=<sig-obj-number> document.pdf
# Look at /ByteRange — does offset2 + length2 == file size?
wc -c document.pdf
```

Step 2: Does the hash match?

```
// Compute the hash yourself and compare to the CMS messageDigest
// (Code in Chapter 13)
```

Step 3: Does the CMS verify?

```
# Extract /Contents, decode hex, verify with OpenSSL
openssl cms -inform DER -in signature.der -verify \
    -content signed-bytes.bin -noverify -out /dev/null
# -noverify skips certificate verification — tests only the CMS math
```

Step 4: Is the certificate chain valid?

```
# Extract the signer's certificate and verify the chain
openssl cms -inform DER -in signature.der -noout -cmsout -print | \
    grep -A 5 "cert:"
openssl verify -CAfile trusted-roots.pem signer-cert.pem
```

Step 5: Is the certificate trusted?

Check the signer's root against the relevant trust store (AATL list, EUTL, OS store).

5.3.2 Common Error Messages and Their Meaning

Table 5.2: PDF signature error messages decoded

Error Message (Viewer)	Step	What It Actually Means
"Signature is INVALID" (Acrobat)	2 or 3	Hash mismatch or CMS verification failure. The document was modified after signing, or the CMS is malformed.
"At least one signature has problems" (Acrobat)	5	The certificate does not chain to an AATL root. The signature may be cryptographically perfect — Acrobat just doesn't trust the CA.
"Document has been altered or corrupted since the Signature was applied" (Acrobat)	1 or MDP	Either the byte range doesn't cover the full file (trailing bytes), or modifications after signing exceed the MDP permissions.
"Signature is valid, but the identity of the signer could not be verified" (Acrobat)	4 or 5	CMS is valid, but the certificate chain is incomplete or the root is not trusted.
"The signature cannot be verified" (LibreOffice)	3, 4, or 5	Generic error — LibreOffice doesn't distinguish well between CMS failure and trust failure. Check the signature details panel.
`CMSException: message-digest attribute value does not match calculated value` (BouncyCastle)	2	Hash mismatch. The bytes you passed to the verifier don't match what was signed.
`CertPathValidatorException: validity check failed` (Java)	4	A certificate in the chain has expired.

Error Message (Viewer)	Step	What It Actually Means
`CertPathValidatorException: no valid CRL found` (Java)	4	Revocation checking failed because the CRL distribution point is unreachable or the CRL is not cached.

5.4 Timestamp Verification

5.4.1 Why Timestamps Matter for Validation

A digital signature proves that a specific certificate holder signed specific bytes. But it doesn't prove *when* the signing happened. Without a timestamp, you can only say "the signature was created sometime before now."

This matters because certificates expire and can be revoked. If a certificate was valid on March 1 but expired on March 31, and you validate the signature on April 15:

- **Without timestamp**: The certificate is expired. Some validators will flag the signature as questionable.
- **With timestamp**: The timestamp proves the signature was created on March 1, when the certificate was valid. The signature remains valid despite the certificate's expiration.

5.4.2 Verifying the Timestamp

A signature timestamp is embedded as an unsigned attribute (`id-aa-signatureTimeStampToken`, OID 1.2.840.113549.1.9.16.2.14) in the CMS `SignerInfo`:

```
// Extract and verify the signature timestamp
AttributeTable unsignedAttrs = signerInfo.getUnsignedAttributes();
if (unsignedAttrs != null) {
    Attribute tsAttr = unsignedAttrs.get(
            PKCSObjectIdentifiers.id_aa_signatureTimeStampToken);
    if (tsAttr != null) {
        CMSSignedData tsToken = new CMSSignedData(
                tsAttr.getAttrValues().getObjectAt(0).toASN1Primitive().getEncoded());

        // Verify the timestamp's CMS signature
        SignerInformation tsSigner = tsToken.getSignerInfos()
                .getSigners().iterator().next();
        X509CertificateHolder tsCert = findSignerCertificate(
                tsToken.getCertificates(), tsSigner.getSID());

        boolean tsValid = tsSigner.verify(
                new JcaSimpleSignerInfoVerifierBuilder()
                        .setProvider("BC").build(tsCert));

        if (tsValid) {
            // Extract the timestamp time
            TimeStampToken tst = new TimeStampToken(tsToken);
            Date signingTime = tst.getTimeStampInfo().getGenTime();
            log.info("Signature was timestamped at: {}", signingTime);
```

```
                // Verify that the timestamp covers the signature value
                byte[] tsDigest = tst.getTimeStampInfo()
                        .getMessageImprintDigest();
                // This digest should match the hash of the signature value
            }
        }
    }
```

5.4.3 Timestamp Chain of Trust

The timestamp itself is a CMS signature — signed by the TSA's certificate. This certificate has its own chain and trust requirements:

1. The TSA certificate must have the `id-kp-timeStamping` extended key usage (OID 1.3.6.1.5.5.7.3.8)
2. The TSA certificate chain must be valid at the time of verification
3. The TSA's root must be in your trust store

If the TSA's certificate has also expired by the time you validate (years later), you need **long-term validation (LTV)** — which is the subject of Chapter 18.

5.5 Building a Complete Validation Pipeline

5.5.1 Architecture

A production validation pipeline typically follows this structure:

```
Input: PDF file
  │
  ├─→ Step 1: Extract all signatures
  │      └─→ For each signature:
  │            ├─→ Step 2: Byte range check
  │            ├─→ Step 3: Hash verification
  │            ├─→ Step 4: CMS verification
  │            ├─→ Step 5: Certificate chain
  │            ├─→ Step 6: Revocation check
  │            ├─→ Step 7: Timestamp verification (if present)
  │            ├─→ Step 8: MDP compliance (if certification sig)
  │            └─→ Step 9: Trust evaluation
  │
  └─→ Output: Validation report
        ├─→ Per-signature results
        ├─→ Overall document status
        └─→ Detailed diagnostics
```

Chapter 31 provides a complete Java implementation of this pipeline.

5.5.2 When to Use EU DSS Instead

If you need to produce **ETSI-compliant validation reports** (ETSI TS 119 102-2) — for example, because you're building a validation service for a government or regulated industry — use

the EU DSS framework (Chapter 23) instead of building the pipeline yourself. EU DSS handles all validation steps, produces standards-compliant reports, and has been tested against the full range of PAdES profiles and edge cases.

Building your own pipeline is appropriate when:

- You need custom trust evaluation logic
- You need to validate signatures in a specific context (e.g., your own application's signing workflow)
- You want to understand every step of the validation process (which is the point of this book)
- You're building a specialized tool (forensic analysis, security research)

5.6 Key Takeaways

- Signature validation is a **five-step pipeline**: byte range hash CMS certificate chain trust
- Each step is independent — a failure at any step means the signature is invalid or questionable
- **Different viewers produce different results** because they use different trust stores and enforce different rules
- Adobe Acrobat uses **AATL** for trust; EU validators use **EUTL**; LibreOffice uses the **OS trust store**
- The most common real-world failure is **trust evaluation** — the signature is cryptographically perfect but the root CA isn't in the validator's trust store
- **Timestamps** prove when a signature was created — essential for handling expired certificates and building long-term validation
- **Debug systematically**: walk through the five steps in order, using QPDF, OpenSSL, and BouncyCastle to isolate the failure
- **EU DSS** provides the most complete automated validation with ETSI-compliant reports
- PDFBox provides the **primitives** for validation but does not automate the pipeline — you build it yourself

Chapter 6

PAdES Profiles (ETSI EN 319 142)

ISO 32000-2 defines *how* a digital signature is embedded in a PDF. PAdES defines *what* that signature must contain to be legally meaningful in Europe — and, increasingly, everywhere else.

PAdES — PDF Advanced Electronic Signatures — is a set of standards published by the **European Telecommunications Standards Institute (ETSI)** that constrain PDF signatures to meet specific compliance requirements. A PAdES signature is always a valid ISO 32000-2 signature, but the reverse is not true. PAdES adds mandatory signed attributes, requires specific SubFilter values, mandates timestamp handling, and defines four progressively more complete signature profiles.

Understanding PAdES is not optional if your signatures need to be validated by European government services, if your customers require eIDAS-compliant signatures, or if you want your signatures to remain verifiable for years or decades. Even if you're not operating in the EU, PAdES profiles have become the de facto standard for "properly structured" PDF signatures worldwide — Adobe Acrobat increasingly favors PAdES-structured signatures in its validation logic.

This chapter explains each PAdES profile, what it requires, how the profiles relate to each other, and how to produce each one.

6.1 The Four PAdES Baseline Profiles

PAdES defines four **baseline profiles**, each building on the previous one. They are formally named PAdES-B-B, PAdES-B-T, PAdES-B-LT, and PAdES-B-LTA. The naming follows the pattern: PAdES-B(aseline)-<level>.

Table 6.1: PAdES Baseline Profile Summary

Profile	Full Name	What It Adds
B-B	Baseline — Basic	The minimum PAdES signature. CMS `SignedData` with mandatory signed attributes including `ESS-signing-certificate-v2`. The signature itself — no timestamps, no revocation data.
B-T	Baseline — Timestamp	B-B + a **signature timestamp** from a trusted TSA. Proves when the signature was created. The timestamp is an unsigned CMS attribute (`id-aa-signatureTimeStampToken`).

Profile	Full Name	What It Adds
B-LT	Baseline — Long-Term	B-T + embedded **revocation data** (OCSP responses and/or CRLs) and the full certificate chain in the PDF's **DSS** (Document Security Store) dictionary. Everything needed to validate the signature offline, without contacting any external servers.
B-LTA	Baseline — Long-Term with Archive timestamp	B-LT + a **document timestamp** signature that covers the entire file (including the DSS dictionary). Enables re-timestamping decades later, creating an indefinite chain of trust that outlives any individual certificate.

6.1.1 The Progression: Why Four Levels?

Each profile addresses a specific failure mode:

- **B-B** fails when: the signing certificate expires (validators may warn), or when you need to prove *when* the signature was created (there's no timestamp)
- **B-T** fails when: the OCSP responder or CRL distribution point goes offline years later, and you can't check revocation status
- **B-LT** fails when: the timestamp's certificate, or the signing certificate's root CA, expires or is removed from trust stores decades later
- **B-LTA** handles all of these by providing a renewable chain of timestamps

In practice:

- **B-B** is sufficient for short-lived documents that will be validated within the signing certificate's validity period (typically 1-3 years)
- **B-T** is the minimum for any document with legal significance — you need to prove when it was signed
- **B-LT** is required for long-term archival (10+ years) where you can't assume external services will remain available
- **B-LTA** is the gold standard for documents that must be verifiable indefinitely — government records, notarized documents, regulatory filings

6.2 PAdES-B-B: The Baseline Signature

6.2.1 Requirements

A PAdES-B-B signature must include:

1. **SubFilter**: Must be `ETSI.CAdES.detached` (not `adbe.pkcs7.detached`)
2. **Mandatory signed attributes in CMS**:
   - `content-type` (OID 1.2.840.113549.1.9.3) — must be `id-data`
   - `message-digest` (OID 1.2.840.113549.1.9.4) — the hash of the signed bytes
   - `ESS-signing-certificate-v2` (OID 1.2.840.113549.1.9.16.2.47) — identifies the signing certificate by its hash, preventing certificate substitution attacks

3. **Signing certificate embedded in CMS**: The signer's certificate (and ideally the full chain up to but not including the root) must be embedded in the CMS `SignedData.certificates` collection
4. **Digest algorithm**: Must be SHA-256 or stronger (SHA-384, SHA-512). SHA-1 is **not** PAdES-compliant.

6.2.2 The ESS-signing-certificate-v2 Attribute

This attribute is the key difference between a basic ISO 32000-2 signature and a PAdES-B-B signature. It binds the CMS signature to a specific certificate by including a hash of the certificate in the signed attributes.

Why this matters: without `ESS-signing-certificate-v2`, an attacker could theoretically substitute a different certificate into the CMS `certificates` collection — one with the same public key but different identity information. The CMS signature would still verify (same key), but the identity would be wrong. The ESS attribute prevents this by cryptographically committing to a specific certificate.

```
// Creating the ESS-signing-certificate-v2 attribute with BouncyCastle
MessageDigest certDigest = MessageDigest.getInstance("SHA-256");
byte[] certHash = certDigest.digest(signingCertificate.getEncoded());

ESSCertIDv2 essCertId = new ESSCertIDv2(
        new AlgorithmIdentifier(NISTObjectIdentifiers.id_sha256),
        certHash);

SigningCertificateV2 signingCertV2 = new SigningCertificateV2(
        new ESSCertIDv2[] { essCertId });

// Add to signed attributes
CMSAttributeTableGenerator signedAttrGen = new DefaultSignedAttributeTableGenerator(
        new AttributeTable(new Hashtable<>() {{
            put(PKCSObjectIdentifiers.id_aa_signingCertificateV2,
                new Attribute(PKCSObjectIdentifiers.id_aa_signingCertificateV2,
                    new DERSet(signingCertV2)));
        }}));
```

A common mistake: using `ESS-signing-certificate` (v1, OID 1.2.840.113549.1.9.16.2.12) instead of `ESS-signing-certificate-v2` (OID 1.2.840.113549.1.9.16.2.47). The v1 attribute uses SHA-1 for the certificate hash, which is deprecated. PAdES requires v2, which uses SHA-256 by default.

iText 2.x (and old versions of some other libraries) produced v1 attributes. If you encounter a PDF with `ESS-signing-certificate` (v1), it's likely from a legacy signing implementation.

6.2.3 SubFilter: `ETSI.CAdES.detached` vs. `adbe.pkcs7.detached`

PAdES requires the SubFilter value `ETSI.CAdES.detached`. This is structurally identical to `adbe.pkcs7.detached` — both produce a detached CMS signature embedded in the `/Contents` hex string. The difference is semantic: `ETSI.CAdES.detached` signals that the signature follows the CAdES (CMS Advanced Electronic Signatures) profile, which includes the ESS-signing-certificate-v2 attribute and other CAdES requirements.

In practice, many validators accept both SubFilter values for PAdES validation. Adobe Acrobat treats `ETSI.CAdES.detached` and `adbe.pkcs7.detached` similarly. But for strict PAdES compliance — especially when dealing with EU government validators — use `ETSI.CAdES.detached`.

6.3 PAdES-B-T: Adding a Timestamp

6.3.1 What B-T Adds

PAdES-B-T is PAdES-B-B plus a **signature timestamp** — a timestamp token from a Trusted Timestamp Authority (TSA) that proves when the signature was created.

The timestamp is embedded as an **unsigned attribute** (`id-aa-signatureTimeStampToken`, OID 1.2.840.113549.1.9.16.2.14) in the CMS `SignerInfo`. It's unsigned because it's applied *after* the signature — you sign first, then request a timestamp over the signature value.

6.3.2 How Timestamping Works

The timestamp workflow:

1. **Sign the document** — produce the CMS `SignedData` with all signed attributes (B-B level)
2. **Hash the signature value** — compute SHA-256 over the CMS `SignerInfo.signature` bytes
3. **Send a timestamp request** — send the hash to a TSA using RFC 3161 (HTTP POST to the TSA's URL)
4. **Receive the timestamp response** — the TSA returns a `TimeStampToken` (which is itself a CMS `SignedData` signed by the TSA's certificate)
5. **Embed the timestamp** — add the `TimeStampToken` as an unsigned attribute in the CMS

```
// Requesting a timestamp from a TSA
TimeStampRequestGenerator tsReqGen = new TimeStampRequestGenerator();
tsReqGen.setCertReq(true);  // Request TSA's certificate in the response

byte[] signatureHash = MessageDigest.getInstance("SHA-256")
        .digest(signerInfo.getSignature());

TimeStampRequest tsReq = tsReqGen.generate(
        TSPAlgorithms.SHA256, signatureHash);

// Send to TSA via HTTP
byte[] tsReqBytes = tsReq.getEncoded();
HttpURLConnection conn = (HttpURLConnection)
        new URL(tsaUrl).openConnection();
conn.setDoOutput(true);
conn.setRequestMethod("POST");
conn.setRequestProperty("Content-Type", "application/timestamp-query");
conn.getOutputStream().write(tsReqBytes);

TimeStampResponse tsResp = new TimeStampResponse(conn.getInputStream());
tsResp.validate(tsReq);  // Verify the response matches our request

TimeStampToken tsToken = tsResp.getTimeStampToken();
```

6.3.3 Choosing a TSA

Choosing a TSA is a non-trivial decision. The TSA's certificate must be trusted by the validators that will check your signatures. Key considerations:

- **For Adobe Acrobat**: The TSA's root must be in the AATL, or the timestamp will show a warning
- **For EU compliance**: The TSA should be a Qualified Trust Service Provider (QTSP) listed in the EUTL
- **For availability**: The TSA must be reachable when you sign — if the TSA is down, your signing fails (or you produce a B-B signature without a timestamp)
- **For cost**: Some TSAs are free (FreeTSA, DigiCert's free tier), others charge per timestamp or per certificate

Chapter 18 covers timestamps in depth, including TSA selection, failover strategies, and the Document Security Store.

6.4 PAdES-B-LT: Long-Term Data

6.4.1 What B-LT Adds

PAdES-B-LT is PAdES-B-T plus **long-term validation data** embedded directly in the PDF. This data ensures that the signature can be validated years from now, even if the signing certificate's CA has gone out of business, the OCSP responder has been decommissioned, and the CRL distribution points return 404.

The long-term data is stored in the **DSS** (Document Security Store) dictionary, which is added to the document catalog:

```
<< /Type /Catalog
   /AcroForm << ... >>
   /DSS <<
     /Certs [30 0 R 31 0 R 32 0 R]       % Certificate chain as DER-encoded streams
     /OCSPs [33 0 R]                       % OCSP responses as DER-encoded streams
     /CRLs [34 0 R]                        % CRLs as DER-encoded streams
     /VRI <<
       /A1B2C3D4... <<                     % Hash-keyed VRI (Validation Related Info)
         /Cert [30 0 R 31 0 R]
         /OCSP [33 0 R]
         /TU (D:20260308120000Z)           % Time of validation
       >>
     >>
   >>
>>
```

6.4.2 The DSS Dictionary

Table 6.2: DSS Dictionary Keys (ISO 32000-2:2020 §12.8.4.3)

Key	Type	Req	Spec Ref
`/Certs`	array of streams	Optional	§12.8.4.3
DER-encoded certificates — the full chain for every signature and timestamp in the document. Include the signer's certificate, all intermediates, and optionally the root.			
`/OCSPs`	array of streams	Optional	§12.8.4.3
DER-encoded OCSP responses for certificate revocation checking. Each response covers a specific certificate at a specific point in time.			
`/CRLs`	array of streams	Optional	§12.8.4.3
DER-encoded Certificate Revocation Lists. Used when OCSP is not available for a certificate.			
`/VRI`	dictionary	Optional	§12.8.4.3
Validation Related Information — a dictionary keyed by uppercase hex SHA-1 hashes of signature values, mapping each signature to its specific validation data.			

6.4.3 Why B-LT Matters

Without B-LT, validating a signature requires contacting external servers:

- OCSP responder to check revocation status
- CRL distribution point to download the revocation list
- CA's AIA (Authority Information Access) URL to fetch intermediate certificates

These servers may be unreachable years from now. CAs shut down, merge, change URLs, or decommission infrastructure. With B-LT, everything needed for validation is embedded in the PDF itself — the document is self-contained.

6.5 PAdES-B-LTA: Archival Timestamps

6.5.1 What B-LTA Adds

PAdES-B-LTA is PAdES-B-LT plus a **document timestamp** — a timestamp signature that covers the entire PDF file, including the DSS dictionary.

This timestamp is not part of the original signature's CMS — it's a separate signature dictionary with `/SubFilter /ETSI.RFC3161`:

```
<< /Type /Sig
   /Filter /Adobe.PPKLite
   /SubFilter /ETSI.RFC3161
   /ByteRange [0 52000 84776 2100]
   /Contents <3082...>
>>
```

This timestamp signature covers the entire file (including the original signature, the DSS, and everything else) and proves that all of this data existed at the time the timestamp was created.

6.5.2 The Archival Chain

The power of B-LTA is that it creates a **renewable chain of trust**:

1. The original signature (signed by the signer's certificate, valid for 1-3 years)
2. The signature timestamp (signed by the TSA's certificate, valid for 10-20 years)
3. The archival timestamp (signed by the TSA's certificate, covers everything)
4. Years later: add another archival timestamp (covers the previous archival timestamp)
5. Repeat indefinitely

Each archival timestamp extends the provable lifetime of the signature. Even after the original signing certificate has expired, been revoked, and the CA has gone out of business — the chain of archival timestamps proves that the signature was valid at the time it was created.

This is the mechanism that national archives use to preserve the legal validity of digitally signed documents for decades.

6.5.3 Re-Timestamping

To extend a B-LTA signature's validity:

1. Fetch fresh OCSP responses / CRLs for the archival timestamp's certificate chain
2. Add them to the DSS dictionary
3. Apply a new archival timestamp (document timestamp) over the entire file

This can be done without the original signer's involvement — it's a maintenance operation performed by the document custodian.

Automated re-timestamping should be a scheduled process in any document management system that stores B-LTA signed documents. Before the current archival timestamp's TSA certificate expires (check the certificate's `notAfter` date), apply a new archival timestamp. This keeps the chain of trust alive.

6.6 Profile Comparison Matrix

Table 6.3: PAdES profile feature comparison

Feature	B-B	B-T	B-LT	B-LTA
CMS with ESS-signing-certificate-v2	Yes	Yes	Yes	Yes
Signature timestamp	No	Yes	Yes	Yes
Embedded revocation data (DSS)	No	No	Yes	Yes
Embedded certificate chain (DSS)	No	No	Yes	Yes
Document timestamp (archival)	No	No	No	Yes
Offline validation	No	Partial	Yes	Yes
Validity beyond cert expiry	No	Yes (with TS)	Yes	Yes (indefinite)

Feature	B-B	B-T	B-LT	B-LTA
EU/eIDAS suitability	Minimal	Short-term	Good	**Full compliance**
Typical use case	Internal docs	Contracts	Regulated docs	Archival

6.7 Migrating Between Profiles

One of the elegant properties of PAdES profiles is that you can **upgrade** a signature's profile without re-signing:

- **B-B B-T**: Add a timestamp (unsigned attribute to the CMS). Requires the original file and the CMS.
- **B-T B-LT**: Fetch revocation data and certificates, create the DSS dictionary, add as incremental update.
- **B-LT B-LTA**: Add a document timestamp covering the entire file.

This progression means you can sign with B-B today (fast, no TSA dependency) and upgrade to B-LTA later as a batch operation. EU DSS provides methods for exactly this:

```
// EU DSS: Extend a B-B signature to B-LTA
PAdESService service = new PAdESService(certificateVerifier);
service.setTspSource(tspSource);

DSSDocument extendedDoc = service.extendDocument(
    signedDocument,
    new PAdESSignatureParameters() {{
        setSignatureLevel(SignatureLevel.PAdES_BASELINE_LTA);
    }});
```

6.8 Key Takeaways

- PAdES constrains ISO 32000-2 signatures for legal and regulatory compliance — a PAdES signature is always a valid PDF signature, but not vice versa
- **Four profiles**: B-B (basic), B-T (timestamped), B-LT (long-term data), B-LTA (archival)
- **ESS-signing-certificate-v2** is the mandatory attribute that makes a signature PAdES-compliant — without it, you have a valid ISO 32000-2 signature but not PAdES
- **SubFilter `ETSI.CAdES.detached`** is required for strict PAdES compliance
- The **DSS dictionary** (B-LT and above) embeds certificates, OCSP responses, and CRLs directly in the PDF for offline validation
- **B-LTA** provides indefinite validity through renewable archival timestamps
- Profiles can be **upgraded** without re-signing — B-B B-T B-LT B-LTA as incremental updates
- For EU/eIDAS compliance, **B-LTA is the target** — anything less may not meet long-term archival requirements

Chapter 7

Timestamps & Long-Term Validation

A digital signature without a timestamp is like a contract without a date. You can prove *who* signed and *what* they signed, but not *when*. And in the world of digital signatures, ”when” is often the most important question — because certificates expire, keys get compromised, algorithms get broken, and the difference between ”signed before the certificate expired” and ”signed after the certificate expired” can be the difference between a legally binding document and an invalid one.

This chapter covers everything about timestamps and long-term validation (LTV): how RFC 3161 timestamps work, how they’re embedded in PDF signatures, how the Document Security Store (DSS) enables offline validation, how archival timestamps create indefinite validity, and how to build a complete LTV infrastructure.

7.1 RFC 3161: The Timestamp Protocol

7.1.1 What a Timestamp Proves

An RFC 3161 timestamp is a cryptographic proof that a specific piece of data existed at a specific point in time. The proof comes from a **Trusted Timestamp Authority (TSA)** — a server operated by a trusted third party that:

1. Receives a hash of the data to be timestamped
2. Combines the hash with the current time
3. Signs the combination with the TSA’s private key
4. Returns the signed timestamp token

The timestamp token is a CMS `SignedData` structure (yes, a timestamp is itself a CMS signature) containing a `TSTInfo` (Timestamp Token Info) structure with:

- The hash algorithm and hash value of the timestamped data
- The timestamp time (UTC)
- The TSA’s name
- A serial number (unique per timestamp)
- An accuracy indication (how precise the time is)

7.1.2 The Timestamp Request/Response Flow

```
Client                              TSA Server
  |                                     |
  | 1. Compute hash of data             |
  |                                     |
  | 2. POST TimeStampRequest ---------> |
```

```
|    (hash algorithm, hash value)     |
|                                     | 3. Verify request
|                                     | 4. Get current UTC time
|                                     | 5. Create TSTInfo
|                                     | 6. Sign with TSA key
| <--------- TimeStampResponse ------- |
|    (TimeStampToken = CMS SignedData)|
|                                     |
| 7. Verify response                  |
| 8. Embed in CMS/PDF                 |
```

The protocol uses HTTP POST with content type `application/timestamp-query` for the request and `application/timestamp-reply` for the response. The request and response are DER-encoded ASN.1 structures defined in RFC 3161.

7.1.3 Implementation with BouncyCastle

```java
/**
 * Requests a timestamp from a TSA for the given data.
 * Returns the TimeStampToken that can be embedded as an unsigned CMS attribute.
 */
public TimeStampToken requestTimestamp(byte[] dataToTimestamp, String tsaUrl)
        throws Exception {

    // 1. Hash the data
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    byte[] hash = digest.digest(dataToTimestamp);

    // 2. Create the timestamp request
    TimeStampRequestGenerator reqGen = new TimeStampRequestGenerator();
    reqGen.setCertReq(true);  // Include TSA's cert in the response
    TimeStampRequest tsReq = reqGen.generate(TSPAlgorithms.SHA256, hash);

    // 3. Send to TSA via HTTP
    byte[] reqBytes = tsReq.getEncoded();
    HttpURLConnection conn = (HttpURLConnection)
            new URL(tsaUrl).openConnection();
    conn.setDoOutput(true);
    conn.setRequestMethod("POST");
    conn.setRequestProperty("Content-Type", "application/timestamp-query");
    conn.setConnectTimeout(10000);
    conn.setReadTimeout(30000);

    try (OutputStream out = conn.getOutputStream()) {
        out.write(reqBytes);
    }

    // 4. Read the response
    byte[] respBytes;
    try (InputStream in = conn.getInputStream()) {
        respBytes = in.readAllBytes();
    }
```

```
        // 5. Parse and validate the response
        TimeStampResponse tsResp = new TimeStampResponse(respBytes);
        tsResp.validate(tsReq);  // Verify it matches our request

        if (tsResp.getStatus() != PKIStatus.GRANTED
                && tsResp.getStatus() != PKIStatus.GRANTED_WITH_MODS) {
            throw new TSPException("TSA rejected request: status=" +
                    tsResp.getStatus() + ", statusString=" +
                    tsResp.getStatusString());
        }

        TimeStampToken tsToken = tsResp.getTimeStampToken();
        log.info("Timestamp obtained: time={}, serial={}, TSA={}",
                tsToken.getTimeStampInfo().getGenTime(),
                tsToken.getTimeStampInfo().getSerialNumber(),
                tsToken.getTimeStampInfo().getTsa());

        return tsToken;
    }
```

7.1.4 TSA Selection Criteria

Choosing the right TSA is critical. A timestamp is only as trustworthy as the TSA that produced it.

Table 7.1: TSA selection criteria for different use cases

Criterion	Considerations
Trust store compatibility	For Adobe Acrobat: TSA's root must be in AATL. For EU compliance: TSA must be a QTSP in the EUTL. For general use: TSA's root should be widely trusted.
Availability & SLA	TSAs can go down. For production signing, use a TSA with a published SLA. Consider a fallback TSA if the primary is unreachable.
Cost	Free TSAs exist (FreeTSA.org, some DigiCert endpoints) but may not be suitable for production. Commercial TSAs charge per timestamp or per annual subscription. Costs range from free to several thousand euros per year.
Accuracy	TSAs report their time accuracy (e.g., ±1 second). For most use cases, sub-second accuracy is not critical.
Geographic location	Some regulations require the TSA to be operated within specific jurisdictions. EU regulations may require a TSA operated by an EU-based QTSP.
Response time	Network latency + TSA processing time. Free TSAs may be slower. For high-volume signing (thousands per minute), TSA response time matters.

Criterion	Considerations
Certificate lifetime	The TSA's signing certificate has an expiry date. Longer-lived TSA certificates mean your timestamps are verifiable for longer before re-timestamping is needed.

Never hardcode a single TSA URL without a fallback strategy. If the TSA is unreachable when your application needs to sign, the signing fails. At minimum, implement:

1. A configurable TSA URL (not hardcoded)
2. A reasonable timeout (10-30 seconds)
3. A retry with backoff (the TSA may be temporarily overloaded)
4. Optionally, a fallback TSA URL
5. A graceful degradation path: if no TSA is available, produce a B-B signature and log a warning for later B-T upgrade

7.2 Signature Timestamps vs. Document Timestamps

PDF supports two kinds of timestamps, and they serve different purposes:

7.2.1 Signature Timestamps

A **signature timestamp** is embedded *inside* the CMS `SignedData` as an unsigned attribute of the `SignerInfo`. It timestamps the *signature value* — proving when the signature was created.

- **Where it lives**: CMS `SignerInfo.unsignedAttrs.id-aa-signatureTimeStampToken`
- **What it timestamps**: The bytes of the `SignerInfo.signature` field
- **PAdES level**: Required for PAdES-B-T and above
- **Who creates it**: The signing application, immediately after computing the CMS signature

7.2.2 Document Timestamps

A **document timestamp** is a separate signature in the PDF — a `/Sig` dictionary with `/SubFilter /ETSI.RFC3161` — that timestamps the *entire document*.

- **Where it lives**: A separate signature dictionary in the PDF, applied as an incremental update
- **What it timestamps**: The entire PDF file (via byte range), including all previous signatures and the DSS
- **PAdES level**: Required for PAdES-B-LTA (the archival timestamp)
- **Who creates it**: The document management system, as a maintenance operation

The key difference: a signature timestamp proves when a *specific signature* was created. A document timestamp proves that the *entire document* (including all signatures and validation data) existed at a specific time.

```
// Signature timestamp (inside CMS)
/Sig <<
```

```
  /SubFilter /ETSI.CAdES.detached
  /Contents <CMS SignedData:
    SignerInfo:
      signedAttrs: [contentType, messageDigest, ESS-signing-certificate-v2]
      signature: <RSA/ECDSA signature bytes>
      unsignedAttrs: [
        signatureTimeStampToken: <TimeStampToken>  ← THIS
      ]
  >
>>

// Document timestamp (separate signature)
/Sig <<
  /SubFilter /ETSI.RFC3161             ← THIS
  /ByteRange [0 ... ... ...]          ← Covers entire file
  /Contents <TimeStampToken as CMS>
>>
```

7.3 The Document Security Store (DSS)

7.3.1 Purpose

The DSS is a dictionary in the document catalog that stores all the data needed to validate every signature in the document *offline* — without contacting any external servers.

Without a DSS, validating a signature requires:

- Fetching the signer's certificate chain from the CMS and/or AIA URLs
- Contacting OCSP responders to check revocation status
- Downloading CRLs from distribution points

All of these require network access and functioning servers. Years from now, these servers may not exist. The DSS solves this by embedding everything locally.

7.3.2 Building the DSS

Building a DSS requires collecting all the validation data and embedding it as an incremental update:

```
/**
 * Adds a DSS (Document Security Store) dictionary to the PDF,
 * embedding certificates, OCSP responses, and CRLs for all signatures.
 */
public void addDSS(PDDocument doc, List<ValidationData> validationData)
        throws IOException {

    COSDictionary dssDictionary = new COSDictionary();
    COSArray certsArray = new COSArray();
    COSArray ocspsArray = new COSArray();
    COSArray crlsArray = new COSArray();
```

```
        for (ValidationData vd : validationData) {
            // Embed certificates
            for (byte[] certBytes : vd.getCertificateChain()) {
                PDStream certStream = new PDStream(doc);
                try (OutputStream os = certStream.createOutputStream()) {
                    os.write(certBytes);
                }
                certsArray.add(certStream.getCOSObject());
            }

            // Embed OCSP responses
            for (byte[] ocspBytes : vd.getOcspResponses()) {
                PDStream ocspStream = new PDStream(doc);
                try (OutputStream os = ocspStream.createOutputStream()) {
                    os.write(ocspBytes);
                }
                ocspsArray.add(ocspStream.getCOSObject());
            }

            // Embed CRLs
            for (byte[] crlBytes : vd.getCrls()) {
                PDStream crlStream = new PDStream(doc);
                try (OutputStream os = crlStream.createOutputStream()) {
                    os.write(crlBytes);
                }
                crlsArray.add(crlStream.getCOSObject());
            }
        }

        if (certsArray.size() > 0) dssDictionary.setItem(COSName.getPDFName("Certs"), certsArray);
        if (ocspsArray.size() > 0) dssDictionary.setItem(COSName.getPDFName("OCSPs"), ocspsArray);
        if (crlsArray.size() > 0) dssDictionary.setItem(COSName.getPDFName("CRLs"), crlsArray);

        // Add DSS to document catalog
        doc.getDocumentCatalog().getCOSObject()
                .setItem(COSName.getPDFName("DSS"), dssDictionary);
    }
```

7.3.3 VRI: Validation Related Information

The DSS can optionally include a `/VRI` sub-dictionary that maps each signature to its specific validation data. The VRI keys are the uppercase hex SHA-1 hash of the signature value (the bytes in `/Contents`):

```
/DSS <<
  /Certs [...]
  /OCSPs [...]
  /VRI <<
    /A1B2C3D4E5F6... <<        % SHA-1 hash of signature 1's /Contents
      /Cert [30 0 R 31 0 R]    % Certs for this specific signature
      /OCSP [33 0 R]           % OCSP for this specific signature
      /TU (D:20260308120000Z)  % Time of validation
    >>
```

```
    /F6E5D4C3B2A1... <<        % SHA-1 hash of signature 2's /Contents
      /Cert [30 0 R 32 0 R]
      /OCSP [34 0 R]
      /TU (D:20260309140000Z)
    >>
  >>
>>
```

VRI is useful for multi-signature documents where each signature has different validation data. Without VRI, a validator must try all certificates and OCSP responses in the DSS against each signature to find the right match. With VRI, the mapping is explicit.

The VRI key is a SHA-1 hash — yes, SHA-1. This is a historical artifact from when the DSS was first specified. SHA-1 is used here only as a key derivation function (to generate a unique dictionary key from the signature value), not for cryptographic security. The security of the DSS does not depend on SHA-1 collision resistance.

7.4 Fetching Revocation Data

7.4.1 OCSP (Online Certificate Status Protocol)

OCSP is the preferred mechanism for checking certificate revocation. You send a request to the CA's OCSP responder with the certificate's serial number, and it responds with "good," "revoked," or "unknown."

```
/**
 * Fetches an OCSP response for the given certificate.
 * Returns the DER-encoded OCSP response bytes.
 */
public byte[] fetchOCSPResponse(X509Certificate cert,
        X509Certificate issuer) throws Exception {

    // 1. Find the OCSP responder URL from the certificate's AIA extension
    String ocspUrl = extractOcspUrl(cert);
    if (ocspUrl == null) {
        log.warn("No OCSP URL in certificate AIA for: {}",
                cert.getSubjectX500Principal());
        return null;
    }

    // 2. Build the OCSP request
    CertificateID certId = new CertificateID(
            new JcaDigestCalculatorProviderBuilder().build()
                    .get(CertificateID.HASH_SHA1),
            new JcaX509CertificateHolder(issuer),
            cert.getSerialNumber());

    OCSPReqBuilder reqBuilder = new OCSPReqBuilder();
    reqBuilder.addRequest(certId);
    OCSPReq ocspReq = reqBuilder.build();

    // 3. Send via HTTP
    HttpURLConnection conn = (HttpURLConnection)
```

```
            new URL(ocspUrl).openConnection();
    conn.setDoOutput(true);
    conn.setRequestMethod("POST");
    conn.setRequestProperty("Content-Type", "application/ocsp-request");
    conn.getOutputStream().write(ocspReq.getEncoded());

    byte[] respBytes = conn.getInputStream().readAllBytes();

    // 4. Validate the response
    OCSPResp ocspResp = new OCSPResp(respBytes);
    if (ocspResp.getStatus() != OCSPResp.SUCCESSFUL) {
        log.error("OCSP response status: {}", ocspResp.getStatus());
        return null;
    }

    BasicOCSPResp basicResp = (BasicOCSPResp) ocspResp.getResponseObject();
    SingleResp singleResp = basicResp.getResponses()[0];

    if (singleResp.getCertStatus() == CertificateStatus.GOOD) {
        log.info("OCSP: certificate is GOOD (produced: {})",
                basicResp.getProducedAt());
        return respBytes;
    } else {
        log.error("OCSP: certificate status is NOT GOOD: {}",
                singleResp.getCertStatus());
        return null;  // Certificate is revoked or unknown
    }
}
```

7.4.2 CRL (Certificate Revocation List)

CRLs are the fallback when OCSP is not available. A CRL is a signed list of all revoked certificates issued by a CA.

```
/**
 * Downloads a CRL from the certificate's CRL Distribution Point.
 * Returns the DER-encoded CRL bytes.
 */
public byte[] fetchCRL(X509Certificate cert) throws Exception {
    String crlUrl = extractCrlUrl(cert);
    if (crlUrl == null) {
        log.warn("No CRL URL in certificate for: {}",
                cert.getSubjectX500Principal());
        return null;
    }

    HttpURLConnection conn = (HttpURLConnection) new URL(crlUrl).openConnection();
    conn.setConnectTimeout(10000);
    conn.setReadTimeout(30000);

    byte[] crlBytes = conn.getInputStream().readAllBytes();

    // Validate the CRL
    CertificateFactory cf = CertificateFactory.getInstance("X.509");
    X509CRL crl = (X509CRL) cf.generateCRL(new ByteArrayInputStream(crlBytes));
```

```
        log.info("CRL downloaded: issuer={}, entries={}, nextUpdate={}",
                crl.getIssuerX500Principal(),
                crl.getRevokedCertificates() != null ?
                        crl.getRevokedCertificates().size() : 0,
                crl.getNextUpdate());

        return crlBytes;
    }
```

CRLs can be **very large** — some CAs have CRLs of 500 KB or more. Embedding a large CRL in the DSS significantly increases the PDF file size. Prefer OCSP when available — OCSP responses are typically 1-3 KB.

Also be aware of **delta CRLs** — some CAs publish only the changes since the last full CRL. If the certificate's CRL Distribution Point specifies a delta CRL, you may need both the full CRL and the delta CRL for complete validation.

7.5 The Complete B-LTA Workflow

Putting it all together, here's the end-to-end workflow for producing a PAdES-B-LTA signature:

```
1. Create PAdES-B-B signature
   └── CMS with ESS-signing-certificate-v2, SHA-256, SubFilter ETSI.CAdES.detached

2. Add signature timestamp → PAdES-B-T
   └── Request RFC 3161 timestamp over the signature value
   └── Embed as unsigned CMS attribute

3. Fetch validation data
   └── Build certificate chain (signer → intermediates → root)
   └── Fetch OCSP response for each certificate
   └── Fetch CRL as fallback where OCSP unavailable
   └── Fetch TSA's certificate chain and revocation data too

4. Create DSS dictionary → PAdES-B-LT
   └── Embed all certificates as streams
   └── Embed all OCSP responses as streams
   └── Embed all CRLs as streams
   └── Optionally create VRI entries
   └── Add DSS to document catalog as incremental update

5. Add document timestamp → PAdES-B-LTA
   └── Request RFC 3161 timestamp over the entire file (including DSS)
   └── Apply as separate /Sig dictionary with SubFilter ETSI.RFC3161
   └── Save as incremental update
```

Each step is an incremental update to the PDF. The final file contains the original document, the signature, the DSS, and the document timestamp — everything needed for indefinite validation.

7.6 Key Takeaways

- **RFC 3161 timestamps** are CMS-signed proofs that data existed at a specific time — issued by Trusted Timestamp Authorities
- **Signature timestamps** (in CMS unsigned attributes) prove when a specific signature was created
- **Document timestamps** (separate `/Sig` with SubFilter `ETSI.RFC3161`) prove when the entire document existed — used for archival
- The **DSS dictionary** embeds certificates, OCSP responses, and CRLs directly in the PDF for offline validation
- **VRI entries** map specific signatures to their validation data using SHA-1 hash keys
- **OCSP** is preferred over CRLs for revocation checking (smaller, more specific)
- **TSA selection** matters: trust store compatibility (AATL/EUTL), availability, cost, and response time
- **PAdES-B-LTA** is the complete package: signature + timestamp + validation data + archival timestamp
- **Re-timestamping** extends validity indefinitely — a scheduled maintenance operation for archival documents
- Always implement **TSA failover** — a TSA outage should not prevent signing

Chapter 8

Security, Incidents & Outlook

8.1 PDF Signature Attacks: A Complete Technical Analysis

> We found 21 out of 22 desktop viewer applications and 5 out of 7 online validation services to be vulnerable to at least one of our attacks.
>
> — Vladislav Mladenov et al. 1 Trillion Dollar Refund

For nearly two decades, the PDF signature ecosystem operated on an assumption that turned out to be wrong: that viewers actually verified what they claimed to verify. Between 2019 and 2021, a research team at Ruhr University Bochum systematically dismantled this assumption. They showed that an attacker could take a validly signed PDF, modify it in specific ways, and present the result to any major PDF viewer — Adobe Acrobat, Foxit Reader, Nitro, LibreOffice, Okular, dozens of others — and the viewer would display a green checkmark and the text ”Signature is valid.”

The signed content had been altered. The viewer said it hadn’t. Every viewer was wrong.

This chapter is a complete technical analysis of every major attack against PDF digital signatures — from the Bochum research program to earlier CVEs, from the theoretical mechanics to working code that demonstrates each attack class. The purpose is not to enable attacks but to make them impossible to misunderstand. If you build or maintain a PDF signature validation system, you need to understand exactly how these attacks work so that your code cannot be fooled by them. If you are a security researcher, this chapter gives you the foundation to find the next generation of PDF signature vulnerabilities — because the Bochum team found the obvious ones, but the attack surface is far from fully explored.

8.1.1 The Researchers

The PDF signature attacks of 2019–2021 were the work of the **Chair of Network and Data Security** (Lehrstuhl für Netz- und Datensicherheit) at Ruhr University Bochum, Germany, led by **Prof. Dr. Jörg Schwenk**.

The core research team:

- **Vladislav Mladenov** — PhD researcher, co-first author of the CCS 2019 paper, led the implementation and testing of the USF, ISA, and SWA attack variants
- **Christian Mainka** — post-doctoral researcher, co-first author of both the CCS 2019 and NDSS 2021 papers, designed the Shadow Attack taxonomy and coordinated responsible disclosure
- **Karsten Meyer zu Selhausen** — researcher and co-founder of **Hackmanit GmbH**, the security consultancy that spun out of the Bochum group; contributed to attack implementation and vendor coordination

- **Martin Grothe** — researcher, contributed to the CCS 2019 paper's systematic evaluation across 22 viewers
- **Simon Rohlmann** — researcher, co-author of the NDSS 2021 Shadow Attacks paper, developed the Hide-and-Replace variant
- **Prof. Dr. Jörg Schwenk** — senior author on both papers, provided the research direction and the formal security model for PDF signatures

The team maintained a coordinated disclosure website at **pdf-insecurity.org**, where they published proof-of-concept PDFs, detailed attack descriptions, and a viewer-by-viewer vulnerability matrix. Hackmanit GmbH later developed **PDF Detector**, a tool for checking PDFs against known attack patterns.

The two landmark papers:

1. *"1 Trillion Dollar Refund: How To Spoof PDF Signatures"* — presented at **ACM Conference on Computer and Communications Security (CCS)**, November 2019, London. This paper introduced three attack classes (USF, ISA, SWA) and tested them against 22 desktop viewers and 7 online validation services.
2. *"Shadow Attacks: Hiding and Replacing Content in Signed PDFs"* — presented at **Network and Distributed Systems Security Symposium (NDSS)**, February 2021 (virtual). This paper introduced a fundamentally different attack model where malicious content is prepared *before* the document is signed.

Both conferences are top-tier academic venues in computer security. The CCS 2019 paper won a **Distinguished Paper Award** — a recognition given to only a handful of papers at each conference.

8.1.2 Before Bochum: The State of PDF Signature Validation

To understand why the Bochum attacks worked, you need to understand what PDF viewers *were* doing when they claimed to "validate" a signature.

A correct signature validation must verify at least the following:

1. **Structural integrity**: The byte range specified in the `/ByteRange` array must cover the entire file except the gap reserved for the `/Contents` value. There must be no unaccounted bytes before, between, or after the byte ranges.
2. **Cryptographic verification**: The CMS `SignedData` structure in `/Contents` must contain a valid signature over the hash of the byte range data. The digest algorithm, signing algorithm, and signed attributes must all be consistent.
3. **Certificate chain validation**: The signer's certificate must chain to a trusted root. The chain must be valid at the time of signing (or at the time of a trusted timestamp). Revocation status must be checked.
4. **Modification detection**: If additional data has been appended to the file after the signature (an incremental update), the viewer must determine whether those changes are permissible under the signature's `/TransformMethod` (e.g., DocMDP permission level) or whether they indicate tampering.

What the Bochum team discovered was that most viewers implemented step 3 reasonably well (they checked certificate chains) but failed catastrophically at steps 1, 2, and 4. Some viewers didn't check the cryptographic signature value at all. Some didn't verify that the

byte range covered the entire file. Some allowed arbitrary changes after signing without any warning.

The result: a perfectly executed certificate chain validation was protecting… nothing. The viewers were carefully verifying *who* signed the document, while completely ignoring *whether the document had been changed since they signed it*.

8.1.3 Attack Class 1: Universal Signature Forgery (USF)

8.1.3.1 The Concept

Universal Signature Forgery is the simplest and most devastating of the three original attack classes. The premise is almost embarrassingly straightforward: what happens if the CMS signature value in `/Contents` is wrong?

A correct implementation should reject the signature immediately — if the cryptographic value doesn't verify, nothing else matters. What the Bochum team found was that some viewers never checked.

In the most extreme cases, the `/Contents` value could be empty (all zeros), filled with random bytes, or contain a valid CMS structure that signed completely different data — and the viewer would still display "Signature is valid" with a green checkmark.

8.1.3.2 How It Works

The attack requires no special tools and no knowledge of the signer's private key. The attacker takes a legitimately signed PDF, modifies the document content (changes text, replaces pages, adds content), and re-calculates any internal structures (cross-reference table, object offsets) needed to keep the file syntactically valid. The `/Contents` value — the CMS `SignedData` blob — is left unchanged, even though it now contains a signature over the *original* document bytes, not the modified ones.

In a correct implementation, the viewer would:

1. Read the `/ByteRange` to determine which bytes are signed
2. Hash those bytes using the digest algorithm specified in the CMS structure
3. Extract the signature value from the CMS `SignerInfo`
4. Verify the signature over the hash using the signer's public key
5. If verification fails reject the signature

Vulnerable viewers skipped step 4, step 3, or in some cases both. They parsed the CMS structure to extract the signer's certificate (for display purposes), checked that the certificate chained to a trust anchor, and then displayed the result — without ever verifying that the signature value was mathematically valid for the current document content.

8.1.3.3 The Variants

The Bochum team identified three USF variants with increasing severity:

USF Variant 1: Manipulated SignerInfo

The `SignerInfo.signature` value (the actual RSASSA-PKCS1-v1_5 or ECDSA bytes) is replaced with random data. A correct implementation rejects this immediately during RSA/ECDSA verification. Vulnerable viewers never reached that step.

USF Variant 2: Manipulated ContentInfo

The outer CMS `ContentInfo` wrapping is modified so the `SignedData` structure is structurally invalid — wrong OIDs, missing fields, or inconsistent version numbers. A correct implementation rejects this during CMS parsing. Vulnerable viewers used lenient parsing that silently ignored structural errors.

USF Variant 3: Manipulated `/Contents` Value

The entire `/Contents` hex string is replaced — either with zeros, with a truncated CMS structure, or with a CMS structure copied from a different document. This tests whether the viewer checks the relationship between the CMS content and the actual document bytes at all.

8.1.3.4 Demonstration: Detecting USF Vulnerability

The following code demonstrates how to detect whether a signed PDF's CMS signature actually validates cryptographically — the check that USF-vulnerable viewers failed to perform. This is the *defense*, not the attack: if you are building a validator, this is the minimum you must implement.

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;
import org.bouncycastle.cert.X509CertificateHolder;
import org.bouncycastle.cert.jcajce.JcaX509CertificateConverter;
import org.bouncycastle.cms.CMSProcessableByteArray;
import org.bouncycastle.cms.CMSSignedData;
import org.bouncycastle.cms.SignerInformation;
import org.bouncycastle.cms.jcajce.JcaSimpleSignerInfoVerifierBuilder;
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import org.bouncycastle.util.Store;

import java.io.FileInputStream;
import java.io.InputStream;
import java.security.Security;
import java.security.cert.X509Certificate;
import java.util.Collection;

/**
 * Demonstrates the cryptographic signature verification that USF-vulnerable
 * viewers failed to perform. This is the MINIMUM check for signature integrity.
 */
public class UsfDefenseCheck {

    static {
        Security.addProvider(new BouncyCastleProvider());
    }

    public static void verifySignature(byte[] pdfBytes) throws Exception {
        PDDocument doc = Loader.loadPDF(pdfBytes);
```

```
for (PDSignature sig : doc.getSignatureDictionaries()) {
    // Step 1: Extract the signed byte ranges
    byte[] signedContent = sig.getSignedContent(
            new ByteArrayInputStream(pdfBytes));

    // Step 2: Extract the CMS SignedData from /Contents
    byte[] cmsBytes = sig.getContents(
            new ByteArrayInputStream(pdfBytes));

    // Step 3: Parse the CMS structure
    CMSSignedData cmsSignedData = new CMSSignedData(
            new CMSProcessableByteArray(signedContent),
            cmsBytes);

    // Step 4: Verify each signer's cryptographic signature
    Store<X509CertificateHolder> certStore =
            cmsSignedData.getCertificates();

    for (SignerInformation signer :
            cmsSignedData.getSignerInfos().getSigners()) {

        Collection<X509CertificateHolder> matches =
                certStore.getMatches(signer.getSID());

        if (matches.isEmpty()) {
            System.err.println(
                    "FAIL: No certificate found for signer");
            continue;
        }

        X509CertificateHolder certHolder = matches.iterator().next();
        X509Certificate cert = new JcaX509CertificateConverter()
                .setProvider("BC")
                .getCertificate(certHolder);

        // THIS IS THE CHECK THAT USF-VULNERABLE VIEWERS SKIPPED
        boolean cryptoValid = signer.verify(
                new JcaSimpleSignerInfoVerifierBuilder()
                        .setProvider("BC")
                        .build(cert));

        if (!cryptoValid) {
            System.err.println(
                    "FAIL: Cryptographic signature verification "
                    + "FAILED for signer: " + cert.getSubjectX500Principal());
            System.err.println(
                    "This document may have been tampered with (USF attack)");
        } else {
            System.out.println(
                    "PASS: Cryptographic signature valid for: "
                    + cert.getSubjectX500Principal());
        }
    }
}
```

```
        doc.close();
    }
}
```

- Extracts the bytes covered by `/ByteRange` — this is the data that was supposedly signed
- Extracts the raw CMS `SignedData` blob from the `/Contents` hex string
- Creates a CMS structure with the signed content as the detached data — BouncyCastle will use this for digest comparison
- If no certificate matches the `SignerInfo.sid`, the CMS structure is malformed — reject immediately
- **This is the critical line.** `signer.verify()` performs the actual RSA/ECDSA verification: it computes the digest over the signed content, verifies the signed attributes, and checks the signature value against the signer's public key. USF-vulnerable viewers never called the equivalent of this method.

If your signature validation code does not include a step equivalent to `signer.verify()` — an actual cryptographic verification of the signature value against the document bytes — your code is vulnerable to Universal Signature Forgery. This is non-negotiable. It must happen before you display any "valid" indicator to the user.

8.1.3.5 Constructing a USF Attack (Proof of Concept)

To understand USF fully, here is how an attacker constructs one. This is *not* production exploit code — it is a minimal demonstration of the mechanics that every PDF signature validator must defend against.

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.PDPage;
import org.apache.pdfbox.pdmodel.PDPageContentStream;
import org.apache.pdfbox.pdmodel.font.PDType1Font;
import org.apache.pdfbox.pdmodel.font.Standard14Fonts;

import java.io.ByteArrayOutputStream;
import java.io.File;
import java.nio.file.Files;

/**
 * Demonstrates USF attack mechanics. Takes a signed PDF, modifies the visible
 * content, and writes the result. The CMS signature value is left unchanged --
 * it now signs the ORIGINAL content, not the modified content.
 *
 * A correct validator will reject this because the digest over the byte range
 * no longer matches the digest in the CMS SignerInfo.
 * A USF-vulnerable validator will show "Signature is valid."
 */
public class UsfAttackDemo {

    public static void main(String[] args) throws Exception {
        byte[] originalPdf = Files.readAllBytes(
                new File("signed-document.pdf").toPath());
```

```
        // Load the signed PDF
        PDDocument doc = Loader.loadPDF(originalPdf);

        // Add a new page with attacker-controlled content
        PDPage attackPage = new PDPage();
        doc.addPage(attackPage);

        PDPageContentStream cs = new PDPageContentStream(doc, attackPage);
        cs.beginText();
        cs.setFont(
                new PDType1Font(Standard14Fonts.FontName.HELVETICA_BOLD), 24);
        cs.newLineAtOffset(50, 700);
        cs.showText("APPROVED - Payment of $1,000,000 authorized");
        cs.endText();
        cs.close();

        // Save with full rewrite — this changes object offsets,
        // xref table, and file length. The /ByteRange in the original
        // signature now covers different bytes, but the /Contents
        // CMS value still contains the hash of the ORIGINAL bytes.
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        doc.save(baos);
        doc.close();

        Files.write(
                new File("usf-attack-result.pdf").toPath(), baos.toByteArray());

        // A correct validator: REJECTS (digest mismatch)
        // A USF-vulnerable validator: shows green checkmark
    }
}
```

- Modifying the document structure after it was signed — adding a page, changing text, anything
- The attacker can insert any content they want
- A full `save()` (not `saveIncremental()`) rewrites the entire file, changing byte offsets everywhere. The original `/ByteRange` and `/Contents` are preserved structurally, but the bytes they reference have changed. The CMS signature now signs data that no longer exists at those byte positions.

This code produces a PDF that any *correct* validator will reject. The point is that in 2019, most validators were not correct. The digest computed over the new byte range content will not match the digest inside the CMS `SignerInfo.digestAlgorithm` — the very first check in proper CMS verification detects this. USF worked because viewers displayed the signer's certificate information *without performing this check*.

8.1.4 Attack Class 2: Incremental Saving Attack (ISA)

8.1.4.1 The Concept

The Incremental Saving Attack is more subtle than USF and exploits a feature that is fundamental to how PDF signatures work: **incremental updates**.

When a PDF is signed, the signing process:

1. Appends the signature dictionary and any new objects to the end of the file
2. Appends a new cross-reference section pointing to the new/modified objects
3. Appends a new trailer
4. The `/ByteRange` in the signature covers everything from the beginning of the file through the end of this new revision — with a gap for the `/Contents` value

The critical insight is that PDF is designed to allow *additional* incremental updates after signing. This is intentional — you need to be able to add a second signature to an already-signed document, or fill in form fields on a signed form. ISO 32000-2:2020 §12.8.1 explicitly allows certain modifications after signing, depending on the signature's `/TransformMethod` and permission level.

The question is: does the viewer correctly distinguish between *permissible* incremental updates (adding another signature) and *impermissible* ones (changing the page content)?

The Bochum team found that most viewers did not.

8.1.4.2 How It Works

The attacker takes a legitimately signed PDF and appends a new incremental update — exactly the way PDFBox's `saveIncremental()` method does. But instead of adding a second signature or filling in a form field, the attacker appends objects that change the visible content: modified page content streams, replaced images, rewritten text, or entirely new pages.

The original signature remains intact — its byte range still covers the original revision, and its CMS signature is still mathematically valid over those bytes. The appended revision contains the modifications, complete with a new cross-reference section and trailer that redirect the viewer to the modified objects.

A correct validator must:

1. Verify the signature is cryptographically valid (steps from USF defense)
2. Determine the document's revision at the time of signing (the byte range endpoint)
3. Compare the current document state to the signed revision
4. If there are changes beyond the signed revision, evaluate whether those changes are permissible under the signature's DocMDP permissions
5. If impermissible changes are detected, display a clear warning: "The document has been modified after signing"

Vulnerable viewers either:

- Did not compare the current document state to the signed revision at all
- Compared but accepted all incremental updates as permissible
- Only checked whether the appended data was "syntactically valid PDF" rather than whether it changed the visible content

8.1.4.3 The Variants

ISA Variant 1: Content Replacement via Incremental Update

The attacker appends a new revision that redefines existing page content stream objects. The cross-reference in the new revision points to new objects at the end of the file that replace the original content. The viewer renders the new content; the signature covers the old content.

ISA Variant 2: Page Insertion/Deletion

The attacker appends a new revision that modifies the page tree — adding new pages, removing existing pages, or reordering them. The `/Pages` dictionary in the new revision points to a different set of page objects.

ISA Variant 3: Annotation and Form Field Manipulation

The attacker appends annotations or modifies form field values. This variant is particularly insidious because some annotation changes *are* legitimately allowed after signing (e.g., adding signature fields for subsequent signers). The boundary between ”allowed” and ”not allowed” is defined by the DocMDP permission level, and most viewers did not enforce it correctly.

8.1.4.4 Demonstration: ISA Attack Mechanics

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.cos.COSDictionary;
import org.apache.pdfbox.cos.COSName;
import org.apache.pdfbox.cos.COSArray;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.PDPage;
import org.apache.pdfbox.pdmodel.PDPageContentStream;
import org.apache.pdfbox.pdmodel.common.PDRectangle;
import org.apache.pdfbox.pdmodel.font.PDType1Font;
import org.apache.pdfbox.pdmodel.font.Standard14Fonts;

import java.io.ByteArrayOutputStream;
import java.io.File;
import java.nio.file.Files;

/**
 * Demonstrates ISA mechanics. Takes a signed PDF and appends an incremental
 * update that replaces the visible content of page 1.
 *
 * The original signature remains cryptographically valid over the original
 * byte range. The appended revision changes what the user sees.
 *
 * A correct validator detects the impermissible modification.
 * An ISA-vulnerable validator shows "Signature is valid" on the modified content.
 */
public class IsaAttackDemo {

    public static void main(String[] args) throws Exception {
        byte[] signedPdf = Files.readAllBytes(
                new File("signed-document.pdf").toPath());

        // Load for incremental save — preserves the original bytes
        PDDocument doc = Loader.loadPDF(signedPdf);

        // Get the first page and replace its content
```

```
        PDPage page = doc.getPage(0);

        // Create a new content stream that overwrites existing content
        PDPageContentStream cs = new PDPageContentStream(
                doc, page,
                PDPageContentStream.AppendMode.OVERWRITE,
                true);

        cs.beginText();
        cs.setFont(
                new PDType1Font(Standard14Fonts.FontName.HELVETICA_BOLD), 36);
        cs.newLineAtOffset(50, 700);
        cs.showText("CONTRACT VOIDED");
        cs.endText();
        cs.close();

        // Save incrementally — appends new objects + xref + trailer
        // The original signature's byte range is UNTOUCHED
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        doc.saveIncremental(baos);
        doc.close();

        Files.write(
                new File("isa-attack-result.pdf").toPath(), baos.toByteArray());

        // The file now looks like this:
        // [original PDF bytes ... signature ... %%EOF]  <-- signed revision
        // [new objects ... new xref ... new trailer ... %%EOF]  <-- attacker's appendage
        //
        // The signature covers only the first section.
        // The viewer renders the second section.
        // A correct validator: warns "Document modified after signing"
        // An ISA-vulnerable validator: shows green checkmark
    }
}
```

- `AppendMode.OVERWRITE` replaces the existing content stream. PDFBox tracks this as a modified object, which will be written in the incremental update
- The attacker-controlled content — can be anything
- `saveIncremental()` appends to the existing file bytes without modifying them. The original signature's `/ByteRange` and `/Contents` remain at their original byte positions. The new revision simply adds new objects and a new cross-reference section after the original `%%EOF`

8.1.4.5 Detecting ISA: Revision Comparison

The defense against ISA requires comparing the document as rendered from the signed revision to the document as rendered from the current (potentially modified) revision.

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;

import java.io.ByteArrayInputStream;
```

```
import java.util.Arrays;

/**
 * Demonstrates how to extract the signed revision of a PDF and compare it
 * to the full file — the core defense against ISA.
 */
public class IsaDefenseCheck {

    public static void checkForAppendedContent(byte[] pdfBytes)
            throws Exception {

        PDDocument doc = Loader.loadPDF(pdfBytes);

        for (PDSignature sig : doc.getSignatureDictionaries()) {
            int[] byteRange = sig.getByteRange();

            if (byteRange == null || byteRange.length != 4) {
                System.err.println("FAIL: Invalid or missing /ByteRange");
                continue;
            }

            // The signed region ends at byteRange[2] + byteRange[3]
            int signedEnd = byteRange[2] + byteRange[3];

            // Check: does the file extend beyond the signed region?
            if (pdfBytes.length > signedEnd) {
                int extraBytes = pdfBytes.length - signedEnd;
                System.err.println(
                        "WARNING: File contains " + extraBytes
                        + " bytes AFTER the signed byte range");
                System.err.println(
                        "This indicates an incremental update after signing.");
                System.err.println(
                        "These bytes are NOT covered by the signature.");
                System.err.println(
                        "Investigate whether the changes are permissible "
                        + "(e.g., additional signatures, form fills) "
                        + "or malicious (content replacement).");

                // Extract the signed revision for comparison
                byte[] signedRevision = Arrays.copyOfRange(
                        pdfBytes, 0, signedEnd);

                // You can now load signedRevision as a separate PDDocument
                // and compare its rendered pages to the full document
                // to determine exactly what changed
            } else if (pdfBytes.length == signedEnd) {
                System.out.println(
                        "PASS: File ends exactly at the signed byte range. "
                        + "No incremental updates after signing.");
            } else {
                System.err.println(
                        "FAIL: File is SHORTER than the signed byte range. "
                        + "File is truncated or corrupted.");
            }
```

```
        }

        doc.close();
    }
}
```

- `/ByteRange` is an array of four integers: `[offset1, length1, offset2, length2]`. The two ranges cover everything before and after the `/Contents` gap
- The signed content ends at the start of range 2 plus its length — this is the last byte the signature covers
- If the file extends beyond this point, there is appended content that the signature does not cover. This is the ISA detection point
- Extracting the signed revision as a standalone PDF allows you to render it separately and compare to the current document state

A naive ”does the file extend beyond the byte range?” check catches obvious ISA attacks, but it is not sufficient. A legitimate second signature also extends the file. The real check requires parsing the appended incremental update and determining *what* changed — was it another signature (allowed), a form fill (allowed under DocMDP P=3), or a content modification (never allowed)? This is the hard part of ISA defense, and it is where most viewers failed in 2019.

8.1.5 Attack Class 3: Signature Wrapping Attack (SWA)

8.1.5.1 The Concept

The Signature Wrapping Attack is the most technically sophisticated of the three original attack classes. Where USF exploits missing cryptographic verification and ISA exploits missing modification detection, SWA exploits the way viewers *locate* the signature within the file.

The core insight: the `/ByteRange` array tells the viewer which bytes are signed, but *nothing prevents an attacker from moving the signed bytes to a different location in the file* — as long as the byte values at the `/ByteRange` offsets still produce the same hash.

Actually, SWA is even simpler than that. The attacker doesn’t need the same hash. The attacker modifies the `/ByteRange` array itself to point to a *different* region of the file — one that contains attacker-controlled content — while keeping the original CMS signature that was computed over the original content.

Wait — if the `/ByteRange` is modified, wouldn’t the signature be invalid? Yes, *if the viewer reads the `/ByteRange` from the signed revision*. But many viewers read the `/ByteRange` from the *current* cross-reference table, which may have been modified by an incremental update. The attacker appends a new revision that:

1. Redefines the signature dictionary object to point to a new `/ByteRange`
2. Places attacker-controlled content at the byte positions specified by the new `/ByteRange`
3. Ensures the CMS signature (in the original `/Contents`) is structurally present

The viewer follows the new cross-reference to the redefined signature dictionary, reads the attacker’s `/ByteRange`, hashes the attacker’s content, and — if it doesn’t actually verify the hash against the CMS signature (shades of USF) — displays ”valid.”

In some variants, the viewer *does* check the hash, but the attacker has been clever about which bytes are at the new `/ByteRange` positions, potentially including a copy of the original signed content or exploiting hash-algorithm weaknesses.

8.1.5.2 How It Works

```
Original file layout:
  [header ... objects ... page content ... ]
  [signature dict: /ByteRange [0 840 3000 1200] /Contents <CMS...>]
  [xref ... trailer ... %%EOF]

  Bytes 0-839 and 3000-4199 are signed.
  Bytes 840-2999 are the /Contents gap (CMS signature value).
  The CMS signature is valid over hash(bytes[0..839] + bytes[3000..4199]).

Attacker appends an incremental update:
  [... %%EOF]                              <-- original file ends here
  [new object: modified signature dict     <-- attacker redefines the sig dict
      /ByteRange [0 840 5000 1200]         <-- points to NEW byte positions
      /Contents <original CMS bytes>]      <-- keeps the original CMS value
  [attacker content at byte offset 5000]   <-- whatever the attacker wants
  [new xref pointing to new sig dict]
  [new trailer ... %%EOF]
```

The viewer follows the new xref, finds the redefined signature dictionary, reads `/ByteRange [0 840 5000 1200]`, and hashes bytes at those positions. If it checks the CMS signature against this hash, it will fail (because the CMS signed the *original* byte range content). But if the viewer is also vulnerable to USF (skips the hash verification), the attack succeeds.

SWA is often combined with elements of ISA and USF. The Bochum team found that many real-world exploits used a combination of attack techniques.

8.1.5.3 Demonstration: Detecting SWA

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;

/**
 * Demonstrates SWA detection: verify that the /ByteRange covers the entire
 * file without gaps (except the /Contents gap) and without overlaps.
 */
public class SwaDefenseCheck {

    public static void checkByteRangeIntegrity(byte[] pdfBytes)
            throws Exception {

        PDDocument doc = Loader.loadPDF(pdfBytes);

        for (PDSignature sig : doc.getSignatureDictionaries()) {
            int[] byteRange = sig.getByteRange();
```

```
if (byteRange == null || byteRange.length != 4) {
    System.err.println("FAIL: Missing or malformed /ByteRange");
    continue;
}

int offset1 = byteRange[0]; // Should be 0
int length1 = byteRange[1];
int offset2 = byteRange[2];
int length2 = byteRange[3];
int signedEnd = offset2 + length2;

// Check 1: First range must start at byte 0
if (offset1 != 0) {
    System.err.println(
            "FAIL: /ByteRange does not start at byte 0. "
            + "Starts at: " + offset1);
}

// Check 2: The gap between ranges is the /Contents value
int gapStart = offset1 + length1;
int gapEnd = offset2;
int gapSize = gapEnd - gapStart;

if (gapSize <= 0) {
    System.err.println(
            "FAIL: /ByteRange gap is zero or negative. "
            + "Possible SWA indicator");
}

// Check 3: Verify /Contents hex string is at the gap position
// The gap should contain exactly the hex-encoded CMS signature
// surrounded by angle brackets: <hex...>
if (pdfBytes[gapStart] != '<') {
    System.err.println(
            "FAIL: Expected '<' at gap start (offset "
            + gapStart + "), found: 0x"
            + Integer.toHexString(
                    pdfBytes[gapStart] & 0xFF));
}

// Check 4: The signed region should cover the entire file
// (for a single-signature document with no subsequent updates)
if (signedEnd < pdfBytes.length) {
    System.err.println(
            "WARNING: /ByteRange ends at byte " + signedEnd
            + " but file is " + pdfBytes.length
            + " bytes. Unsigned trailing content detected");
}

// Check 5: Cross-validate that the /ByteRange in the file
// matches what we'd expect from the signed revision
// This requires parsing the original revision's xref
System.out.println(
        "ByteRange analysis: "
        + "range1=[" + offset1 + "," + (offset1 + length1) + ") "
```

```
                    + "gap=[" + gapStart + "," + gapEnd + ") "
                    + "range2=[" + offset2 + "," + signedEnd + ") "
                    + "fileSize=" + pdfBytes.length);
        }

        doc.close();
    }
}
```

- The first byte range must start at offset 0. If it doesn't, there are unsigned bytes at the beginning of the file — a clear SWA indicator
- The gap between the two ranges must be positive and should correspond to the `/Contents` hex string. A zero or negative gap indicates structural manipulation
- The gap should contain the hex-encoded CMS value. Checking that it starts with < and ends with > is a basic sanity check

SWA defense requires more than just checking `/ByteRange` arithmetic. A sophisticated attacker can construct a valid-looking byte range that passes arithmetic checks but points to carefully crafted content. The complete defense is: (a) verify `/ByteRange` covers the file correctly, (b) verify the CMS signature cryptographically (USF defense), and (c) verify that the `/ByteRange` values come from the signed revision's cross-reference, not from an appended revision.

8.1.6 Attack Class 4: Shadow Attacks (2021)

8.1.6.1 A Fundamentally Different Threat Model

The three attacks from CCS 2019 (USF, ISA, SWA) all shared a common assumption: the attacker modifies a PDF *after* it has been legitimately signed. The signed document is genuine; the attacker's manipulation comes afterward.

Shadow Attacks, published at NDSS 2021, inverted the model. The attacker prepares the PDF *before* it is signed, embedding hidden malicious content that the signer cannot see during the signing process. The signer reviews the document, sees nothing suspicious, and signs it. The signed PDF is then sent to the recipient, who opens it — and sees different content from what the signer saw.

The signature is cryptographically valid. The signer genuinely signed those exact bytes. The bytes just don't mean what the signer thought they meant.

This is not a cryptographic failure. It is a semantic failure: the gap between what a human *sees* when they review a document and what the *bytes* actually encode. And it is far harder to defend against than USF, ISA, or SWA.

8.1.6.2 Shadow Attack Variant 1: Hide

The attacker creates a PDF with two layers of content — one visible, one hidden. The hidden content is placed *behind* the visible content using PDF features like:

- **Overlapping objects**: A white rectangle drawn on top of text, hiding it. The text is in the content stream; the rectangle covers it. After signing, the attacker removes the rectangle via an incremental update, revealing the hidden text.

- **Optional Content Groups (OCG)**: PDF's layer mechanism. Content can be tagged with `/OC` (Optional Content) markers and made visible or invisible via the `/OCProperties` dictionary. The attacker sets a layer to invisible before signing, then sets it to visible after signing.
- **Referenced XObjects**: Content referenced as `Form XObject` but not actually rendered on any page. After signing, an incremental update adds the XObject to a page's content stream.

The key constraint: *all of the hidden content must be present in the file at the time of signing*, so that it falls within the signed byte range. The attacker only changes the *visibility* via an incremental update after signing — which some viewers allowed without warning.

8.1.6.3 Shadow Attack Variant 2: Replace

The attacker creates a PDF where the visible content can be *replaced* by different visible content without modifying any objects in the signed byte range. This exploits **interactive form features**:

- A text field with a default value ("Contract for $1,000") and a different actual value ("Contract for $1,000,000") — the default is displayed initially, but changing which value is rendered requires only an annotation flag change
- Font references that can be swapped — the same Unicode code points render as different glyphs depending on which font is active
- Images referenced by name from a page's Resources dictionary — changing which image a name resolves to requires only modifying the Resources dictionary

The Replace variant is particularly dangerous because it exploits features that are *designed* to change document appearance (form filling, annotation updates) — the viewer must distinguish between a legitimate form fill and an attacker swapping content.

8.1.6.4 Shadow Attack Variant 3: Hide-and-Replace

The most powerful variant combines both techniques. The attacker creates a PDF that:

1. Contains the content the signer will see (layer A)
2. Contains the content the recipient will see (layer B), hidden behind layer A
3. Is structured so that a single incremental update (changing a few bytes in an annotation dictionary or OCG configuration) switches from layer A to layer B

After signing, the attacker appends the minimal incremental update needed to switch layers. If the viewer allows annotation or OCG changes after signing (as many did, because these are often considered "permissible" changes), the attack succeeds.

8.1.6.5 Demonstration: Shadow Attack (Hide Variant) Preparation

```
import org.apache.pdfbox.Loader;
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.PDPage;
import org.apache.pdfbox.pdmodel.PDPageContentStream;
import org.apache.pdfbox.pdmodel.common.PDRectangle;
import org.apache.pdfbox.pdmodel.font.PDType1Font;
```

```
import org.apache.pdfbox.pdmodel.font.Standard14Fonts;

import java.io.ByteArrayOutputStream;
import java.io.File;
import java.nio.file.Files;

/**
 * Demonstrates the preparation phase of a Shadow Attack (Hide variant).
 * The attacker creates a PDF with hidden text behind a white overlay.
 *
 * BEFORE signing: Signer sees "Payment: $1,000"
 * AFTER signing + reveal: Recipient sees "Payment: $1,000,000"
 *
 * The hidden text ("Payment: $1,000,000") is present in the signed bytes.
 * The overlay (white rectangle) hides it. An incremental update that removes
 * the overlay reveals the hidden text — and the signature remains valid.
 */
public class ShadowAttackHideDemo {

    public static void main(String[] args) throws Exception {
        PDDocument doc = new PDDocument();
        PDPage page = new PDPage(PDRectangle.A4);
        doc.addPage(page);

        PDType1Font font = new PDType1Font(
                Standard14Fonts.FontName.HELVETICA);
        PDType1Font fontBold = new PDType1Font(
                Standard14Fonts.FontName.HELVETICA_BOLD);

        PDPageContentStream cs = new PDPageContentStream(doc, page);

        // Layer 1: The hidden content (written FIRST, so it's behind)
        // This text will be present in the signed bytes
        cs.beginText();
        cs.setFont(fontBold, 16);
        cs.newLineAtOffset(50, 700);
        cs.showText("Payment: $1,000,000");
        cs.endText();

        // Layer 2: The overlay that hides it
        // A white rectangle drawn ON TOP of the hidden text
        cs.setNonStrokingColor(1.0f, 1.0f, 1.0f); // White
        cs.addRect(45, 695, 250, 25); // Covers the text above
        cs.fill();

        // Layer 3: The visible content (what the signer sees)
        cs.setNonStrokingColor(0.0f, 0.0f, 0.0f); // Black
        cs.beginText();
        cs.setFont(fontBold, 16);
        cs.newLineAtOffset(50, 700);
        cs.showText("Payment: $1,000");
        cs.endText();

        // Legitimate-looking document body
        cs.beginText();
```

```
        cs.setFont(font, 12);
        cs.newLineAtOffset(50, 650);
        cs.showText("This agreement confirms the payment amount above.");
        cs.endText();

        cs.close();

        // Save — this PDF is now ready to be sent to the signer
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        doc.save(baos);
        doc.close();

        Files.write(
                new File("shadow-prepared.pdf").toPath(), baos.toByteArray());

        // The signer opens this PDF, sees "Payment: $1,000", and signs it.
        // The signed byte range includes ALL content: both the hidden
        // "$1,000,000" text and the white rectangle that covers it.
        //
        // After signing, the attacker creates an incremental update that
        // removes or makes transparent the white rectangle, revealing
        // "Payment: $1,000,000" — and the signature validates.
    }
}
```

- The hidden text is written to the content stream first. In PDF rendering order, later operators paint *over* earlier ones on the same area
- A white filled rectangle covers the hidden text completely. The signer sees only white space at this position
- The visible text is drawn on top of the white rectangle. This is what the signer reads when reviewing the document before signing

Shadow Attacks are fundamentally harder to prevent than USF, ISA, or SWA because the malicious content is *part of the signed data.* The signature is cryptographically valid over the exact bytes the signer intended to sign. The defense must go beyond byte-level verification into *semantic* analysis: does the document contain hidden content? Are there overlapping objects? Are there unused Form XObjects? Are there Optional Content Groups with suspicious visibility settings? This is an open research problem with no complete solution.

8.1.6.6 The Scope of Shadow Attack Vulnerability

The Bochum team tested 29 PDF viewers against the Shadow Attack variants. The results were comprehensive:

Table 8.1: Shadow Attack test results across PDF viewers (NDSS 2021)

Viewer	Hide	Replace	Hide-and-Replace
Adobe Acrobat Reader DC	Vulnerable	Vulnerable	Vulnerable
Foxit Reader	Vulnerable	Vulnerable	Vulnerable

Viewer	Hide	Replace	Hide-and-Replace
LibreOffice Draw	Vulnerable	Partially vulnerable	Vulnerable
Nitro Pro	Vulnerable	Vulnerable	Vulnerable
PDF-XChange	Vulnerable	Not vulnerable	Vulnerable
Okular	Vulnerable	Vulnerable	Vulnerable
Evince	Vulnerable	Vulnerable	Vulnerable
Chrome PDF Viewer	No signature validation	—	—
macOS Preview	Vulnerable	Vulnerable	Vulnerable

This table is a representative subset. The full results covering 29 viewers are available in the NDSS 2021 paper and at pdf-insecurity.org. Chrome's built-in PDF viewer does not validate signatures at all (it displays them but does not indicate validity), so Shadow Attacks are not applicable in the traditional sense — but the user also receives no warning that the document has been signed and potentially tampered with.

8.1.7 Pre-Bochum Vulnerabilities: Earlier PDF Signature Issues

The Bochum research was the most systematic analysis of PDF signature security, but it was not the first time vulnerabilities were discovered. Several earlier issues laid the groundwork — and reveal that the *category* of bug (failing to verify what you claim to verify) has been a persistent pattern in PDF signature implementations.

8.1.7.1 CVE-2018-16042 — Adobe Acrobat Signature Bypass (2018)

In 2018, Adobe patched **CVE-2018-16042** in Acrobat DC and Acrobat Reader DC (versions before 2019.008.20081). The vulnerability allowed a crafted PDF to display a valid signature indicator even when the document had been modified.

The technical details were not fully disclosed (Adobe's advisory was characteristically brief), but the vulnerability class — signature validation bypass via document manipulation — is the same category as the ISA attacks that Bochum would systematize a year later. This CVE was one of several signals that the PDF signature validation code in major viewers had not been subject to rigorous adversarial testing.

Affected versions: Acrobat DC/Reader DC before 2019.008.20081
Impact: Signature validation bypass — modified documents displayed as validly signed
Fix: Updated signature validation logic in the January 2019 security update

8.1.7.2 CVE-2018-4993 — Adobe Acrobat NTLM Credential Theft (2018)

While not strictly a signature bypass, **CVE-2018-4993** demonstrated a related risk: a crafted PDF could trigger NTLM authentication requests to an attacker-controlled server when

opened in Adobe Acrobat. Combined with a forged signature (via any of the bypass techniques), this creates a compound attack: the victim opens a ”validly signed” PDF, and their Windows credentials are silently exfiltrated.

Affected versions: Acrobat DC/Reader DC before 2018.011.20040
Impact: NTLM hash theft via SMB relay when opening a crafted PDF
Fix: Restricted external resource loading in the May 2018 security update

8.1.7.3 SHAttered and the SHA-1 Deprecation (2017)

In February 2017, researchers from Google and CWI Amsterdam published **SHAttered** — the first practical collision attack against SHA-1. They produced two different PDF documents with the same SHA-1 hash, proving that SHA-1 could no longer be relied upon for collision resistance.

For PDF signatures, this had immediate implications. The `adbe.pkcs7.sha1` SubFilter — which embeds a SHA-1 hash of the entire document as the CMS content data — was now theoretically vulnerable to a collision attack: an attacker could prepare two documents with the same SHA-1 hash, have the victim sign one, and then substitute the other.

The practical impact was limited (the collision required approximately 2^{63} computations and the two colliding documents had to be carefully constructed), but the theoretical break was sufficient to trigger industry-wide deprecation:

- ISO 32000-2:2020 deprecated `adbe.pkcs7.sha1` entirely
- Adobe Acrobat began showing warnings for SHA-1 signed documents
- The CA/Browser Forum required SHA-256 as the minimum for all new certificates
- PAdES profiles already mandated SHA-256 as the minimum digest algorithm

Table 8.2: SHA-1 timeline for PDF signatures

Year	Event
2005	Theoretical weakness in SHA-1 published (Wang et al.)
2011	NIST deprecated SHA-1 for digital signatures in federal use
2013	Microsoft announced SHA-1 certificate sunset for 2016
2017	SHAttered: first practical SHA-1 collision (Google/CWI). Two PDFs with identical SHA-1 hashes publicly demonstrated
2017	ISO 32000-2 deprecated `adbe.pkcs7.sha1` SubFilter
2020	SHA-1 collisions reduced to 2^{63} operations (Leurent and Peyrin, ”SHA-1 is a Shambles”), making chosen-prefix collisions feasible

If your code still generates signatures using SHA-1 (either via `adbe.pkcs7.sha1` or using SHA-1 as the `digestAlgorithm` in the CMS `SignerInfo`), stop immediately. SHA-1 is broken. Use SHA-256 or SHA-384. There is no legitimate reason to use SHA-1 for any new signature created after 2017.

8.1.7.4 The Comodo Registration Authority Breach (2011)

In March 2011 — the same month as DigiNotar, though discovered slightly earlier — an attacker compromised a **registration authority (RA)** affiliated with Comodo, one of the world's largest certificate authorities at the time.

The attacker used the compromised RA account to issue nine fraudulent SSL certificates for high-profile domains: `login.live.com`, `mail.google.com`, `www.google.com`, `login.yahoo.com`, `login.skype.com`, `addons.mozilla.org`, and three others. The certificates were issued by Comodo's legitimate intermediate CA, meaning they chained correctly to a trusted root.

The breach was discovered quickly because the attacker tested one of the certificates against a live server, and certificate pinning in Google Chrome detected the mismatch. Without Chrome's certificate pinning, the fraudulent certificates could have been used for undetectable man-in-the-middle attacks.

For PDF signatures: The Comodo breach demonstrated that even without compromising a root CA's private key, an attacker could obtain legitimate-looking certificates through a *registration authority* compromise — a weaker link in the chain. A document signed with one of these fraudulent certificates would have validated correctly in any PDF viewer that trusted Comodo's root. The signer would appear to be "Google" or "Microsoft" or "Yahoo" — entities that obviously did not sign the document.

The incident led to:

- Mandatory two-factor authentication for all RA accounts at major CAs
- Domain control validation (DCV) requirements tightened by the CA/Browser Forum
- Increased scrutiny of the RA model, eventually leading several CAs to eliminate external RAs entirely

8.1.7.5 The Symantec Distrust (2017–2018)

Between 2015 and 2017, Google's Chrome security team and the CA/Browser Forum discovered that **Symantec** (which at that time owned the Thawte, GeoTrust, VeriSign, and RapidSSL certificate brands) had mis-issued thousands of certificates — including certificates for domains whose owners had not authorized them.

The problems were systemic:

- Symantec had delegated certificate issuance to third-party partners (RAs) without adequate oversight
- Test certificates for real domains were issued to unauthorized parties
- Audit logs were incomplete or missing
- Symantec initially denied the scope of the problem, then acknowledged it in stages

Google announced a graduated distrust of all Symantec-issued certificates:

- Chrome 66 (April 2018): distrusted certificates issued before June 1, 2016
- Chrome 70 (October 2018): distrusted all remaining Symantec certificates

Mozilla Firefox and Apple Safari followed with their own distrust timelines. Symantec sold its CA business to DigiCert, which re-issued certificates for affected customers under DigiCert's infrastructure.

For PDF signatures: Symantec was a major CA in the AATL (Adobe Approved Trust List). Documents signed with Symantec-issued certificates before the distrust cutoff dates would validate correctly when signed, but might show warnings or failures in PDF viewers that updated their trust stores to reflect the browser distrust. This created a time-dependent validation problem: a PDF signed with a Symantec certificate in 2015 was ”valid” when signed, ”suspect” in 2018, and — depending on the viewer’s trust configuration — might show as ”invalid” or ”valid with warnings” today.

This is exactly the problem that PAdES-B-LTA (long-term validation with archival timestamps) was designed to solve: by embedding the full certificate chain and revocation data at the time of signing, and then protecting it with periodic archival timestamps, the signature’s validity at the time of signing can be verified even after the issuing CA is no longer trusted.

8.1.8 Exploits in the Wild

A natural question: were any of these attacks exploited before or after the Bochum disclosures?

The honest answer is: **we don’t know for certain**.

PDF signatures are used in financial contracts, government documents, legal filings, tax returns, insurance claims, and medical records. If an attacker successfully forged a signature on a financial contract, the victim might not discover it until a dispute arises — and even then, the forensic analysis might not identify the specific attack technique.

What we do know:

8.1.8.1 Indicators of Real-World Exploitation

1. **The ”1 Trillion Dollar Refund” title was not hyperbole.** The paper’s title referenced the fact that tax authorities in multiple countries accept signed PDFs as official submissions. The German tax system (ELSTER), for example, processes millions of electronically signed documents per year. If USF or ISA could forge signatures on tax submissions, the financial exposure was enormous. The Bochum team coordinated with CERT-Bund (Germany’s national CERT) before publication.
2. **Adobe’s response was unusually fast.** Adobe issued CVE-2020-9592 and CVE-2020-9596 and released patches within weeks of the responsible disclosure. For a company that typically operates on monthly security update cycles, the accelerated timeline suggests Adobe assessed the risk as immediate and severe — consistent with either active exploitation or intelligence indicating exploitation was imminent.
3. **The EU Digital Signature Service (DSS) was vulnerable.** The EU’s own reference implementation for signature validation — the tool member states use to validate eIDAS-compliant signatures — was vulnerable to ISA attacks. If the EU’s own validator could be fooled, signatures submitted to government services across 27 member states could potentially have been forged without detection.
4. **Commercial signing services had no detection capability.** Before the Bochum research, no commercial PDF signing or validation service tested for these attack classes. There were no automated tools to detect ISA, SWA, or Shadow Attack patterns in submitted PDFs. Any exploitation that occurred before 2019 would have been invisible to the existing security infrastructure.

8.1.8.2 Post-Disclosure Exploitation Risk

After the papers were published and patches released, the risk shifted:

- **Unpatched viewers remain vulnerable.** Not all users update their PDF viewers. Enterprise environments, government agencies with slow update cycles, and users in regions with limited internet access may still be running vulnerable versions of Acrobat, Foxit, or other viewers.
- **The attack techniques are public.** The Bochum team published detailed descriptions and proof-of-concept PDFs. Any attacker with basic PDF knowledge can reproduce the attacks.
- **Shadow Attacks remain partially unresolved.** While most viewers now detect the Hide and Replace variants, the fundamental problem — that PDF semantics allow content to be visually hidden — is inherent to the format. New variants of Shadow Attacks are likely still discoverable.

8.1.9 The Patch Response: What Changed

8.1.9.1 Adobe's Response

Adobe's patches for CVE-2020-9592 and CVE-2020-9596 (released in the May 2020 Acrobat security update, version 2020.006.20042) implemented:

1. **Cryptographic signature verification** — closing the USF vulnerability by ensuring the CMS signature value is always verified against the document hash
2. **Byte range coverage validation** — checking that the `/ByteRange` covers the entire file from offset 0 to the file's end (minus the `/Contents` gap), closing the SWA vulnerability
3. **Incremental update analysis** — warning users when an incremental update modifies content beyond what the signature's permission level allows, closing the ISA vulnerability
4. **Shadow content detection** — partial detection of Hidden content patterns (overlapping objects, OCGs with suspicious visibility settings)

Acrobat's signature panel now displays more granular information:

- "Signed and all signatures are valid" — no modifications detected
- "Signed but the document has been modified since signing" — incremental updates detected, with details about what changed
- "At least one signature has problems" — cryptographic or structural verification failed
- "The document has been altered or corrupted since the signature was applied" — clear indication of tampering

8.1.9.2 Foxit's Response

Foxit Reader patched its signature validation in version 10.1 (released October 2020), implementing similar checks: CMS verification, byte range validation, and incremental update analysis. Foxit was notably slower than Adobe in responding — roughly five months after Adobe's patch.

8.1.9.3 LibreOffice's Response

LibreOffice (which uses the NSS library for cryptographic operations) patched its PDF signature validation in version 7.0. The fix was more complex because LibreOffice's PDF import/export pipeline differs significantly from dedicated PDF viewers — it converts PDF to an internal format for editing, which complicates byte-level signature verification.

8.1.9.4 The Broader Industry

By mid-2021, most actively maintained PDF viewers had released patches addressing the three original attack classes (USF, ISA, SWA). Shadow Attack mitigations were slower to arrive and remain incomplete in some viewers as of 2026.

The Bochum research also influenced the standards bodies:

- **ISO TC 171** (the committee responsible for ISO 32000-2:2020) incorporated the attack findings into the errata and clarifications for the second edition of PDF 2.0
- **ETSI** updated the PAdES standard (EN 319 142-1 V1.2.1, published 2022) to include explicit requirements for byte range coverage verification and incremental update validation
- **The EU Digital Signature Service (DSS)** — the open-source reference validator maintained by the European Commission — was patched and now includes specific test cases for each attack class

8.1.10 Building a Robust Validator: Lessons from the Attacks

Every attack in this chapter exploits a specific gap between what a viewer *should* check and what it *actually* checks. Here is the complete checklist, informed by every vulnerability discussed above.

Table 8.3: PDF Signature Validation Checklist (post-Bochum)

#	Check	Defends Against
1	Parse the CMS `SignedData` structure and verify it is well-formed (correct OIDs, consistent version numbers, all required fields present)	USF Variant 2
2	Verify the `SignerInfo.signature` value cryptographically against the signer's public key and the computed digest	USF Variant 1
3	Verify the message digest in the signed attributes matches the hash of the byte range content	USF Variant 3, SWA
4	Verify `/ByteRange` starts at offset 0	SWA
5	Verify `/ByteRange` gap contains the `/Contents` hex string	SWA

#	Check	Defends Against
6	Verify `/ByteRange` covers the entire file (for the signed revision) — no unaccounted bytes before, between, or after the ranges	SWA, ISA
7	If the file extends beyond the signed byte range, parse the incremental update and determine what changed	ISA
8	Verify that any changes in the incremental update are permissible under the signature's DocMDP permission level (P=1: no changes; P=2: form fills + signatures; P=3: form fills + signatures + annotations)	ISA Variant 3
9	Verify the certificate chain from the signer's certificate to a trusted root, checking revocation (OCSP/CRL) and validity dates	Certificate forgery, CA breaches
10	If a timestamp is present, verify the TSA certificate chain independently and check that the timestamp covers the correct data	Backdated signatures
11	Check for hidden content patterns: overlapping objects, Optional Content Groups with invisible layers, unreferenced Form XObjects, form fields with discrepant default/actual values	Shadow Attacks (Hide, Replace)
12	Ensure the digest algorithm is SHA-256 or stronger — reject SHA-1	SHAttered, hash collision attacks

Checks 1–6 are non-negotiable — any validator that skips them is vulnerable to at least one of USF, ISA, or SWA. Checks 7–10 are necessary for real-world security but require more complex implementation. Checks 11–12 are defense-in-depth against attacks that remain partially unresolved.

No checklist is complete. The Bochum team found these attack classes by thinking adversarially about the gap between what the spec *requires* and what implementations *do*. There will be more attacks. The defense is not just implementing these twelve checks — it is building a culture of adversarial testing into your validation code.

8.1.11 A Note on Responsible Disclosure

The Bochum team followed responsible disclosure for both papers. They notified affected vendors months before publication and provided proof-of-concept PDFs for testing.

For the CCS 2019 paper, the disclosure timeline was approximately:

- **March–April 2019**: Initial vendor notifications
- **May–August 2019**: Back-and-forth with vendors; some requested extended timelines; some did not respond

- **November 2019**: Paper presented at CCS; public disclosure
- **May 2020**: Adobe patches released (CVE-2020-9592, CVE-2020-9596)
- **October 2020**: Foxit patches released

The six-month gap between public disclosure and Adobe's patch — and the eleven-month gap before Foxit's patch — is notable. The Bochum team publicly criticized the slow response, particularly from viewers that took more than six months to patch after detailed proof-of-concept material was shared privately.

For the NDSS 2021 paper (Shadow Attacks), the disclosure process was similar but vendors responded faster, having already been through the CCS 2019 cycle.

The research materials — including proof-of-concept PDFs, the vulnerability matrix, and detailed attack descriptions — remain available at **pdf-insecurity.org**. The Hackmanit GmbH **PDF Detector** tool (available as a web service and standalone library) can check PDFs for known attack patterns.

8.1.12 Summary

PDF signature security before 2019 was, to put it bluntly, an illusion. The cryptographic primitives were sound — RSA, ECDSA, SHA-256, CMS are all well-designed and well-analyzed. The failure was entirely in how PDF viewers *used* those primitives. They verified certificates meticulously and signatures not at all.

The Bochum research (CCS 2019, NDSS 2021) exposed four attack classes:

- **USF**: The viewer doesn't check the cryptographic signature value — forge anything
- **ISA**: The viewer doesn't check for modifications after signing — append anything
- **SWA**: The viewer doesn't validate the byte range — redirect it to anything
- **Shadow Attacks**: The viewer doesn't analyze what the signed content *means* — hide anything

Every major PDF viewer was vulnerable. The patches took months to years. Some Shadow Attack variants remain partially unresolved.

Earlier events — the DigiNotar breach (2011), the Comodo RA compromise (2011), the Symantec distrust (2017–2018), and the SHAttered SHA-1 collision (2017) — had already weakened the trust infrastructure underneath PDF signatures. The Bochum attacks showed that even if you trusted the infrastructure, the validators themselves couldn't be trusted.

The lesson for implementers is clear: **verify everything, trust nothing, and test adversarially**. The validation checklist in this chapter is the minimum. The next attack class is already being researched.

Chapter 9

Code, Tools & Implementation

Chapter 10

The Java PDF Library Landscape

Before we write a single line of signing code, we need to talk about the tools we're using — where they came from, why they exist, and what baggage they carry. If you've ever wondered why there are three different "iText" libraries on Maven Central, why PDFBox's API changed so dramatically between version 2 and version 3, or why a library from the year 2000 with known bugs is still running in Fortune 500 production systems — this chapter is for you.

The Java PDF ecosystem is not just a collection of libraries. It is a story of open-source idealism, licensing betrayal (depending on who you ask), corporate acquisitions, community forks, and the kind of technical debt that accumulates when enterprises build critical infrastructure on a dependency and then the rules change. Understanding this history is not optional background reading — it directly affects the code you write, the signatures you produce, and the compatibility problems you will encounter.

10.1 The iText Saga: From PhD Project to Corporate Licensing Drama

10.1.1 Bruno Lowagie and the Birth of iText (2000–2009)

The story of Java PDF processing begins, for practical purposes, with Bruno Lowagie. In 2000, Lowagie — a Belgian mathematician working at a printing company — needed to generate PDF documents from Java. The options at the time were essentially nonexistent. Adobe had published the PDF specification, but the only tools that could create PDFs were Adobe's own products (Acrobat, Distiller) and a handful of expensive commercial libraries. There was nothing usable in the open-source Java world.

So Lowagie wrote his own library. He called it **iText**, released it under the **Mozilla Public License (MPL)** and later the **GNU Lesser General Public License (LGPL)**, and published it on SourceForge — the GitHub of its era. The LGPL license was critically important: it allowed companies to use iText in proprietary software without open-sourcing their own code, as long as they didn't modify the iText library itself. This is the license model that made iText ubiquitous.

The timing was perfect. Java was exploding in enterprise environments. J2EE application servers were the dominant architecture. Companies needed to generate invoices, reports, contracts, and forms as PDFs — millions of them per day, in automated workflows. iText was free, it worked, and its API was straightforward:

```
// iText 2.x — the classic API that a generation of Java developers learned
Document document = new Document();
PdfWriter.getInstance(document, new FileOutputStream("output.pdf"));
document.open();
document.add(new Paragraph("Hello, PDF"));
document.close();
```

This simplicity was revolutionary. You didn't need to understand PDF internals. You didn't need to manipulate dictionaries or worry about cross-reference tables. You told iText what you wanted on the page, and it produced a valid PDF. The library handled fonts, images, tables, forms, encryption, and — crucially for our purposes — **digital signatures**.

By the mid-2000s, iText was everywhere. It was embedded in JBoss, Alfresco, Jasper Reports, BIRT, and hundreds of other Java projects. Large banks, insurance companies, government agencies, and healthcare organizations had built their document generation pipelines on iText. The Maven coordinates `com.lowagie:itext` were among the most downloaded artifacts in the Java ecosystem.

10.1.2 The LGPL-to-AGPL Switch: The Day the Rules Changed (2009)

On December 1, 2009, Bruno Lowagie changed the license of iText from **LGPL** to **AGPL** (GNU Affero General Public License), starting with version **5.0.0**.

This was, depending on your perspective, either a reasonable business decision or an act of betrayal against the community that had made iText successful. Both perspectives have merit, and understanding the controversy matters because its consequences are still playing out in production systems today.

What AGPL means in practice: The AGPL is like the GPL, but with one additional requirement: if you use an AGPL-licensed library in software that users interact with over a network (which includes web applications, SaaS products, and API services), you must release the *entire source code* of your application under the AGPL. Not just your modifications to iText — your entire application.

For the thousands of companies using iText in their proprietary Java backends — generating invoices, processing insurance claims, signing contracts — this was devastating. They had exactly three options:

1. **Open-source their entire application** under AGPL — unthinkable for most enterprises
2. **Buy a commercial license** from iText Group NV (the company Lowagie founded to monetize iText) — expensive, typically €5,000–€50,000+ per year depending on deployment scale
3. **Stay on iText 2.1.7** — the last LGPL-licensed version, released in 2009 — forever

Most chose option 3.

Why Lowagie made the switch: Lowagie's stated reasons were straightforward: he had spent nearly a decade maintaining a complex library for free, while companies worth billions used it to generate revenue without contributing back. He wanted to build a business. The AGPL created a forcing function: use iText for free in open-source projects, or pay for a commercial license. This is a perfectly legitimate business model — MongoDB, Elasticsearch, and Redis would later make similar moves (though with different licenses).

Why the community was angry: The anger wasn't about the business model itself — it was about the trust violation. Enterprise architects had chosen iText *because* of its LGPL license. They had written it into procurement documents, gotten legal approval, and built systems that depended on it. The license change felt like a bait-and-switch: attract an enormous user base with a permissive license, then change the terms once everyone is locked in.

The fact that existing versions remained LGPL was cold comfort. iText 2.1.7 was already showing its age in 2009. By 2012, it had known bugs in font handling, incomplete PDF 1.7 support, and no support for the newer CMS structures that PAdES was beginning to require. But upgrading to iText 5.x meant either paying or going AGPL.

If you encounter a PDF signed by a Java application that uses `com.lowagie.text.pdf.PdfSignatureAppeara` in its imports — that's iText 2.x. The class was renamed to `com.itextpdf.text.pdf.PdfSignatureAppearanc` in iText 5.x (note the package change from `com.lowagie` to `com.itextpdf`). The package name tells you which era of iText produced the signature.

10.1.3 The CMS Structure Problem: Why Old iText Signatures Look Different

Here's where the licensing drama directly impacts signature compatibility.

iText 2.x (the LGPL version that much of the world is still using) produces CMS `SignedData` structures that are technically valid per RFC 5652 — Cryptographic Message Syntax (CMS), but differ from what modern validators expect in several ways:

Encapsulated vs. Detached content: Early iText versions could produce CMS signatures with *encapsulated* content — meaning the signed hash was embedded inside the `SignedData` structure's `encapContentInfo` field rather than being referenced externally via the PDF byte range. This was not wrong per the CMS spec, but it created problems:

- Modern PAdES validators expect detached signatures (SubFilter `adbe.pkcs7.detached` or `ETSI.CAdES.detached`), where the signed content is the byte range in the PDF itself — not a copy inside the CMS
- Some validators would see encapsulated content and assume the signature was standalone (not bound to the PDF), failing validation even though the cryptographic math was correct
- The double-hash problem: if the signed bytes in the CMS didn't match what a byte-range-aware validator computed from the PDF file, the signature appeared invalid

Missing signed attributes: iText 2.x did not always include the `ESS-signing-certificate-v2` attribute that PAdES requires. Without this attribute, the signature is valid per ISO 32000-2 but *not* PAdES-compliant — which means it fails validation in any system that enforces PAdES profiles (including EU government validation services and Adobe Acrobat in its stricter validation modes).

SHA-1 default: iText 2.1.7 defaulted to SHA-1 for the message digest. SHA-1 was already theoretically weakened by 2009 (Wang and Yu's collision attacks were published in 2005), and it was definitively broken in 2017 when the SHAttered attack produced actual colliding files. PDFs signed with iText 2.x before SHA-256 became the norm carry SHA-1 digests, which modern validators increasingly flag or reject.

If your organization has a corpus of PDFs signed between 2005 and 2015 by Java applications, there is a significant chance they were signed with iText 2.x using SHA-1 and potentially using encapsulated (not detached) CMS content. These signatures may fail validation in modern tools.

To verify: extract the CMS SignedData from the `/Contents` entry and check:

1. Is `encapContentInfo.eContentType` set with actual content? (encapsulated — old style)
2. Is the digest algorithm SHA-1 (`OID 1.3.14.3.2.26`)? (deprecated)

3. Is the `ESS-signing-certificate-v2` attribute present in `signedAttrs`? (missing = not PAdES-compliant)

Chapter 31 provides code to perform all three checks.

10.1.4 iText 5.x, 7.x, and 8.x: The Commercial Evolution

After the AGPL switch, iText evolved rapidly — but behind a paywall for commercial use.

iText 5.x (2009–2016): Maintained the original API design but added PDF 2.0 features, improved CMS handling, and better PAdES support. The package name changed from `com.lowagie` to `com.itextpdf`. If you see both package prefixes in the same project's classpath — and this happens more often than you'd think — you're looking at a project that tried to use both the old LGPL version and the new AGPL version simultaneously, which is a recipe for classloading nightmares.

iText 7.x (2016–2023): A complete rewrite with a modular architecture. The API was entirely different from iText 5.x — migration was not trivial. iText 7 introduced separate modules for core PDF operations, forms, signing, and other features. The signing module (`itext7-sign`) included comprehensive PAdES support, including B-LTA profiles with embedded timestamps and revocation data. The commercial license became the primary offering, with AGPL remaining for open-source projects.

iText 8.x (2023–present): The current generation, continuing the modular architecture with added support for PDF 2.0 features, improved accessibility tagging, and enhanced signing capabilities. iText Group NV (now iText Software, headquartered in Ghent, Belgium, with offices in the US and Singapore) is a substantial commercial entity with paying enterprise customers worldwide.

The commercial iText is, by all accounts, a high-quality library. Its signing implementation is mature, well-tested, and PAdES-compliant. But at commercial license prices that can reach tens of thousands of euros per year for enterprise deployments, it is not an option for every project — which is why the open-source alternatives exist.

10.1.5 The IBM Connection

IBM's involvement with iText is a lesser-known but important part of the story. In the early 2000s, IBM was one of the largest users of iText — embedding it in several of their enterprise products, including **IBM Content Manager**, **IBM Filenet**, and various WebSphere-based document processing systems.

IBM contributed patches and improvements back to the project during the LGPL era, which was entirely within the LGPL's collaborative spirit. However, when the license changed to AGPL, IBM — like most large enterprises — was unwilling to either open-source their proprietary products or pay iText's commercial licensing fees for a dependency that had previously been free.

This created an awkward situation: IBM's products contained iText 2.x code, their customers expected continued PDF functionality, and the upgrade path required either legal procurement (commercial iText license) or engineering investment (migrate to a different library). IBM's response, like most enterprises in this position, was a combination of staying on old

versions and gradually migrating new development to Apache PDFBox — which was, by then, becoming mature enough for production use.

The IBM story is representative of what happened across the industry: hundreds of enterprises had iText deeply embedded in their systems, and the license change created a slow-motion migration that, in many cases, is still incomplete in 2026.

10.2 OpenPDF: The Community Fork That Refuses to Die

10.2.1 The Fork (2016)

When it became clear that iText 5.x was permanently AGPL for commercial use, and that iText 2.1.7 was accruing unfixed bugs and missing modern features, a group of developers forked iText 2.1.7 under the name **OpenPDF** (initially called **LibrePDF**).

The fork was hosted on GitHub under the `LibrePDF/OpenPDF` repository, and it was explicitly positioned as the LGPL-licensed continuation of the original iText codebase. The initial stated goals were modest: fix bugs, add basic modern features (SHA-256 support, better font handling), and keep the library usable for projects that needed LGPL licensing.

10.2.2 What OpenPDF Is (and Isn't)

OpenPDF today (version 2.0.x, as of 2026) is a capable PDF generation library that maintains API compatibility with iText 2.x while adding features that the original 2.1.7 never had:

- **SHA-256/384/512 message digests** for signing (critical — SHA-1 is no longer acceptable)
- **AES-256 encryption** support
- **Improved font handling**, including better CJK support and TrueType subsetting
- **Bug fixes** for dozens of issues that were never patched in iText 2.1.7

However, OpenPDF also carries significant limitations:

- It is built on the iText 2.x architecture, which was designed for PDF 1.7 and earlier
- **No PDF 2.0 support** — cross-reference streams, tagged PDF improvements, and other 2.0 features are absent
- **Limited PAdES support** — basic PAdES-B-B signatures are possible, but B-LT and B-LTA profiles (which require DSS dictionary construction and archival timestamps) are not natively supported
- **No built-in LTV (Long-Term Validation)** — you'll need to construct the DSS dictionary yourself
- The contributor community is small — a handful of maintainers, not the large team that iText Group NV employs
- **Documentation is sparse** — the iText 2.x book (Lowagie's "iText in Action") is the closest thing to comprehensive documentation, and it describes an API from 2008

OpenPDF lives at `com.github.librepdf:openpdf` on Maven Central. Do not confuse it with `com.lowagie:itext` (the original iText 2.1.7) or `com.itextpdf:itextpdf` (the AGPL iText 5.x). All three exist on Maven Central. All three have different licenses, different capabilities, and different bugs. Getting the wrong one in your dependency tree is a common source of confusion — and potential legal liability.

10.2.3 Should You Use OpenPDF?

For **PDF generation** (creating invoices, reports, forms) where you need an LGPL-licensed library and don't require PDF 2.0 features: OpenPDF is a reasonable choice.

For **digital signatures**: OpenPDF can produce basic CMS signatures, but for PAdES compliance, long-term validation, and the kind of robust signing infrastructure this book teaches, **PDFBox is the better choice**. PDFBox's signing API is more actively maintained, better tested against the Bochum attack scenarios, and has a larger community contributing security fixes.

That said — if you have an existing codebase built on iText 2.x that you cannot easily migrate, OpenPDF is the path of least resistance for getting off the unmaintained 2.1.7 version while keeping your existing API calls intact.

10.3 Apache PDFBox: The Open-Source Standard

10.3.1 Origin Story: Ben Litchfield and the Apache Incubation

Apache PDFBox started as **PDFBox** — a standalone project created by **Ben Litchfield** in 2002. Litchfield, working as a Java developer, needed to *read* existing PDF documents — not just create them (which was iText's primary strength at the time). He needed to extract text, parse structures, and manipulate the low-level PDF object model. iText was designed as a PDF *writer*; PDFBox was designed as a PDF *reader* that could also write.

This fundamental design difference — reader-first vs. writer-first — is still visible in the APIs today:

- **iText/OpenPDF** excels at high-level document construction: "add a paragraph here, put a table there, generate the PDF"
- **PDFBox** excels at low-level document manipulation: "give me the COS dictionary for object 42, let me modify the annotation rectangle, show me the raw font encoding"

For digital signature work, PDFBox's reader-first architecture is a significant advantage. Signing a PDF requires understanding the existing document structure, computing byte ranges, parsing existing signatures, and manipulating specific dictionaries — all tasks where low-level access is essential.

In 2008, PDFBox entered the **Apache Software Foundation** incubator process, and in 2009 it graduated as a top-level Apache project under the **Apache License 2.0** — one of the most permissive open-source licenses available. Unlike iText's LGPL□AGPL journey, PDFBox has been Apache-licensed since inception and will remain so. This licensing certainty is one of the reasons enterprises migrated to PDFBox.

10.3.2 PDFBox 2.x: The Workhorse (2016–2023)

PDFBox 2.0 was the version that brought PDFBox into mainstream production use. It featured:

- A complete COS (Carousel Object System) model for low-level PDF manipulation
- Text extraction with position information
- PDF rendering (via `PDFRenderer`)

- AcroForm support (creating, filling, and flattening forms)
- Digital signature support via `PDDocument.addSignature()`
- Font handling (Type1, TrueType, CID fonts, subsetting)

The signing API in PDFBox 2.x was functional but had rough edges. The `SignatureInterface` required you to provide a `sign(InputStream)` method that received the bytes to sign and returned the CMS `SignedData` as a byte array. The actual CMS construction was your responsibility — typically using BouncyCastle (we'll discuss this later). PDFBox handled the PDF-specific parts: creating the signature dictionary, computing the byte range, reserving space for `/Contents`, and performing the incremental save.

```
// PDFBox 2.x signing pattern
PDDocument doc = PDDocument.load(inputFile);
PDSignature signature = new PDSignature();
signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
signature.setSubFilter(PDSignature.SUBFILTER_ADBE_PKCS7_DETACHED);
signature.setName("Signer Name");
signature.setSignDate(Calendar.getInstance());

doc.addSignature(signature, new SignatureInterface() {
    @Override
    public byte[] sign(InputStream content) throws IOException {
        // Your BouncyCastle CMS construction goes here
        return createCMSSignature(content, certificate, privateKey);
    }
});

doc.saveIncremental(outputStream);
doc.close();
```

This separation of concerns — PDFBox handles the PDF, you handle the cryptography — is elegant in theory but demanding in practice. Getting the CMS structure right for PAdES compliance requires deep knowledge of signed attributes, certificate embedding, and timestamp handling. Chapters 29 and 30 provide the complete implementation.

10.3.3 PDFBox 3.x: The Breaking Change (2024)

PDFBox 3.0 was released in 2024 as a major version bump with **breaking API changes**. The migration was significant enough to catch many projects off guard:

PDFBox 3.x is **not** a drop-in replacement for 2.x. If you're upgrading, budget significant development time. The changes are pervasive and affect every area of the API.

The most impactful changes for signing:

Loading documents:

```
// PDFBox 2.x
PDDocument doc = PDDocument.load(new File("document.pdf"));
PDDocument doc = PDDocument.load(byteArray);

// PDFBox 3.x — PDDocument.load() is GONE
PDDocument doc = Loader.loadPDF(new File("document.pdf"));
PDDocument doc = Loader.loadPDF(byteArray);
```

Font construction:

```
// PDFBox 2.x
PDType1Font font = PDType1Font.HELVETICA;

// PDFBox 3.x — static font constants are GONE
PDType1Font font = new PDType1Font(Standard14Fonts.FontName.HELVETICA);
```

AcroForm API:

```
// PDFBox 2.x
boolean needsAppearances = acroForm.isNeedAppearances();

// PDFBox 3.x — method renamed
boolean needsAppearances = acroForm.getNeedAppearances();
```

Font encoding access:

```
// PDFBox 2.x
Encoding encoding = font.getEncoding(); // Worked for most font types

// PDFBox 3.x — getEncoding() removed from PDFont interface
// Must access encoding through COS dictionary directly
COSDictionary fontDict = font.getCOSObject();
COSBase encoding = fontDict.getDictionaryObject(COSName.ENCODING);
```

Content streams:

```
// PDFBox 2.x — could write to form XObjects with PDPageContentStream
PDPageContentStream cs = new PDPageContentStream(doc, formXObject, ...);

// PDFBox 3.x — PDPageContentStream is for pages ONLY
// For form XObjects, write raw PDF operators via the stream's output
OutputStream os = formXObject.getStream().createOutputStream();
```

The signing API itself (`PDDocument.addSignature()`, `SignatureInterface`, `PDSignature`) remained structurally similar, but the surrounding changes — especially in document loading, font handling, and content stream creation — mean that any signing code that also creates visual signature appearances needs substantial rework.

Migration strategy: Don't try to upgrade PDFBox in-place. Create a new branch, update the dependency, let the compiler tell you every break, and fix them systematically. The PDFBox team maintains a migration guide that covers the major changes, but the edge cases (especially around font encoding and content stream APIs) will require reading the 3.x Javadoc carefully.

10.3.4 The PDFBox Signing Infrastructure: What's Under the Hood

PDFBox's signing implementation is one of the most impressive pieces of open-source PDF engineering. It's worth understanding the high-level architecture, because when something goes wrong during signing — and it will — you need to know where to look.

The signing process in PDFBox involves these internal steps:

1. **Signature dictionary creation**: PDFBox creates the `/Sig` dictionary with the properties you set (`/Filter`, `/SubFilter`, `/Name`, `/M`, etc.)

2. **Space estimation**: PDFBox needs to know how much space to reserve for the `/Contents` hex string *before* the signature is computed — because the signature covers the bytes outside the `/Contents` gap, and the file must be written before the signature can be calculated. The default is 16KB (32,768 hex characters), but PAdES-B-LTA signatures with embedded timestamps and revocation data can easily exceed this.
3. **Byte range computation**: PDFBox calculates the `/ByteRange` array — the two regions of the file that the signature covers (everything before and after the `/Contents` value)
4. **Incremental save**: The modified PDF is written as an incremental update — appended to the original file, preserving previous revisions
5. **Signing callback**: PDFBox passes the byte range data (as an `InputStream`) to your `SignatureInterface.sign()` implementation
6. **Signature injection**: The returned CMS bytes are hex-encoded and written into the reserved `/Contents` space

If the CMS signature is larger than the reserved space, the signing fails. If it's smaller (which is normal), the remaining space is filled with zeros — the zero-padding that you see in the hex dump of any signed PDF's `/Contents` entry.

This architecture means that **PDFBox does not construct the CMS signature itself**. It provides the bytes to sign, and you provide the signature. This is both a strength (maximum flexibility — you can use any CMS library, any algorithm, any certificate) and a burden (you must get the CMS right yourself). That's why the next chapter is dedicated entirely to BouncyCastle CMS construction.

10.4 The Legion of the Bouncy Castle

10.4.1 Why BouncyCastle Exists

Java has a built-in cryptographic provider — the `SunJSSE` and `SunJCE` providers that ship with every JDK. So why does almost every Java PDF signing project use BouncyCastle instead?

Three reasons:

1. **CMS support**: The JDK's built-in providers do not include a CMS (`SignedData`) implementation. They provide the raw cryptographic primitives (RSA, ECDSA, SHA-256, etc.), but not the higher-level CMS container format that PDF signatures require. BouncyCastle's `bcpkix` module provides `CMSSignedDataGenerator`, `CMSSignedData`, and all the ASN.1 infrastructure needed to construct and parse CMS structures.
2. **Algorithm breadth**: BouncyCastle supports algorithms that the JDK providers don't — including EdDSA (Ed25519, Ed448), GOST, SM2, and various other national or emerging algorithms. While PDF signatures currently use RSA and ECDSA almost exclusively, the ability to work with less common algorithms is valuable for government and specialized applications.
3. **ASN.1 library**: BouncyCastle's ASN.1 library (`bcutil` and `bcprov`) is the de facto standard for Java ASN.1 encoding/decoding. When you need to parse a certificate, construct a signed attribute, or decode an OCSP response, BouncyCastle's ASN.1 classes are what you use.

10.4.2 The BouncyCastle Ecosystem

BouncyCastle is not a single JAR — it's a family of related libraries:

Table 10.1: BouncyCastle modules relevant to PDF signing

Module	Maven Artifact	Purpose
Provider	`bcprov-jdk18on`	Core cryptographic provider: symmetric/asymmetric ciphers, hashes, key generation, random number generation. Register via `Security.addProvider(new BouncyCastleProvider())`
PKIX/CMS/EAC	`bcpkix-jdk18on`	CMS `SignedData` construction/parsing, PKCS#10 certificate requests, OCSP, timestamp protocol (RFC 3161), certificate path validation. **This is the module you need for PDF signing.**
Utilities	`bcutil-jdk18on`	PEM reader/writer, ASN.1 dump utilities, base64 encoding. Pulled in as a transitive dependency.

You must explicitly register the BouncyCastle provider before using it. If you forget this step, you'll get cryptic `NoSuchAlgorithmException` errors at runtime.

```
import org.bouncycastle.jce.provider.BouncyCastleProvider;
import java.security.Security;

// Do this once at application startup
if (Security.getProvider("BC") == null) {
    Security.addProvider(new BouncyCastleProvider());
}
```

In Spring Boot applications, put this in a `@Configuration` class with a `@PostConstruct` method. In standalone applications, put it in your `main()` method. Forgetting this registration is the #1 cause of "it works in my IDE but fails in production" signing issues.

10.4.3 BouncyCastle Version History and the `-jdk18on` Suffix

BouncyCastle's artifact naming has changed over the years, which causes dependency confusion:

- `bcprov-jdk15` — for JDK 1.5 (ancient, do not use)
- `bcprov-jdk15on` — for JDK 1.5 and later (the long-lived artifact ID)
- `bcprov-jdk18on` — for JDK 1.8 (Java 8) and later (current, use this)

The `jdk18on` variants (introduced with BouncyCastle 1.71) dropped support for Java versions before 8 and added Java module system (`module-info.java`) support. If you're on Java 11 or later (and for PDF signing with PDFBox 3.x, you should be on Java 17+), use the `jdk18on` artifacts exclusively.

Do not mix `jdk15on` and `jdk18on` artifacts in the same classpath. BouncyCastle uses a provider registration model, and having two versions of the provider registered simultaneously

causes unpredictable failures — the wrong version may handle a given algorithm, producing incompatible ASN.1 structures.

10.5 EU DSS: The European Commission's Signing Framework

10.5.1 What EU DSS Is

There is one more library that deserves its own section — not because it replaces PDFBox or BouncyCastle, but because it *builds on top of them* and adds a layer that would take months to implement yourself.

EU DSS (Digital Signature Services) is an open-source Java framework developed and maintained by **Nowina Solutions** under contract from the **European Commission**. It is the reference implementation for creating and validating digital signatures that comply with the European eIDAS regulation — which means it implements the full PAdES, CAdES, XAdES, and JAdES signature profiles, including the complex long-term validation (LTV) workflows that individual libraries like PDFBox cannot do alone.

The project lives at `github.com/esig/dss` and is released under the **LGPL 2.1** license. The Maven coordinates follow the pattern `eu.europa.ec.joinup.sd-dss:dss-*`.

10.5.2 Why EU DSS Matters for PDF Signatures

If your application needs to produce **PAdES-B-LTA** signatures — the gold standard for long-term archival signatures that remain verifiable for decades — you have two choices:

1. **Build it yourself**: Use PDFBox for the PDF layer, BouncyCastle for CMS construction, write your own timestamp client (RFC 3161), implement OCSP and CRL fetching, construct the DSS (Document Security Store) dictionary, embed the VRI (Validation Related Information), and manage archival timestamp chains. This is what Chapters 29–31 of this book teach. It works. It is also hundreds of hours of development and testing.
2. **Use EU DSS**: Import the `dss-pades` module, configure a `PAdESSignatureParameters` object, provide a `SignatureTokenConnection` (which wraps your certificate and private key), and let the framework handle everything — CMS construction, signed attribute inclusion, timestamp requests, revocation data embedding, DSS dictionary creation, and incremental save.

Here's what a PAdES-B-LTA signature looks like with EU DSS:

```
// EU DSS — PAdES-B-LTA signing (simplified)
PAdESSignatureParameters params = new PAdESSignatureParameters();
params.setSignatureLevel(SignatureLevel.PAdES_BASELINE_LTA);
params.setDigestAlgorithm(DigestAlgorithm.SHA256);
params.setSigningCertificate(new CertificateToken(certificate));
params.setCertificateChain(Arrays.asList(intermediateCerts));

// TSA (Timestamp Authority) for the signature timestamp
OnlineTSPSource tspSource = new OnlineTSPSource("http://tsa.example.com/timestamp");
params.setContentTimestampParameters(new TimestampParameters(DigestAlgorithm.SHA256));

PAdESService service = new PAdESService(commonCertificateVerifier);
service.setTspSource(tspSource);
```

```
// Two-step signing: digest, then sign
ToBeSigned dataToSign = service.getDataToSign(documentToSign, params);
SignatureValue signatureValue = signingToken.sign(dataToSign, params.getDigestAlgorithm(), privateKey);
DSSDocument signedDocument = service.signDocument(documentToSign, params, signatureValue);
```

Compare this to implementing the same B-LTA profile manually with PDFBox + BouncyCastle — the EU DSS version handles the timestamp embedding, OCSP/CRL fetching, DSS dictionary construction, and VRI population automatically.

10.5.3 EU DSS Architecture

EU DSS is modular. You don't need to import the entire framework — pick the modules relevant to your signature format:

Table 10.2: EU DSS modules relevant to PDF signing

Module	Purpose
`dss-pades`	PAdES signature creation and extension (B-B B-T B-LT B-LTA). Uses PDFBox internally for PDF manipulation.
`dss-cades`	CAdES signature creation. The PAdES module delegates CMS construction to this.
`dss-document`	Core document model (`DSSDocument`, `InMemoryDocument`, `FileDocument`) and service interfaces.
`dss-service`	Online data loaders — HTTP clients for TSA, OCSP, and CRL fetching.
`dss-tsl-validation`	EU Trusted List (EUTL) parsing and validation. Essential for determining whether a signing certificate is qualified under eIDAS.
`dss-validation-report`	Generates detailed validation reports following ETSI TS 119 102-2 — the same validation report format that EU government validation services produce.
`dss-token`	`SignatureTokenConnection` implementations: PKCS#12 file, PKCS#11 (HSM), JKS keystore, MS-CAPI (Windows certificate store).

10.5.4 EU DSS vs. Rolling Your Own

The honest comparison:

Advantages of EU DSS:

- **Compliance out of the box**: EU DSS is the European Commission's reference implementation. If you use it correctly, your signatures will pass validation on the EU's official validation service (DSS demonstration webapp) and on member-state validation services. This is not a guarantee you get from hand-rolling with PDFBox + BouncyCastle.
- **LTV complexity hidden**: Long-term validation requires fetching OCSP responses, downloading CRLs, embedding them in the PDF's DSS dictionary, creating VRI entries,

and adding archival timestamps. EU DSS does all of this. Implementing it yourself is a significant effort.
- **Trusted List integration**: The `dss-tsl-validation` module can load and parse the EU Trusted List, letting you programmatically determine whether a signing certificate is qualified. Building this yourself means implementing ETSI TS 119 612 XML parsing and certificate matching — nontrivial.
- **Validation reports**: EU DSS generates ETSI-compliant validation reports that are accepted by auditors and compliance officers.

Advantages of rolling your own (PDFBox + BouncyCastle):

- **Full control**: You understand every byte. When something fails, you know exactly where to look because you wrote every layer.
- **No framework weight**: EU DSS pulls in a substantial dependency tree. If you only need PAdES-B-B (baseline signature without timestamps or LTV), you can do it with PDFBox + BouncyCastle in about 200 lines of code — EU DSS adds a megabytes of framework for something achievable with direct library calls.
- **Learning**: Building it yourself teaches you the spec. Using EU DSS teaches you the framework's API. For a book like this one, understanding comes from building.
- **Customization**: EU DSS makes assumptions about how signatures should be constructed. If your use case deviates (unusual SubFilter values, custom signed attributes, non-standard timestamp configurations), you may fight the framework rather than benefit from it.

Our recommendation: Learn to build signatures with PDFBox + BouncyCastle first (Chapters 29–31). Then, when you need PAdES-B-LTA or EU Trusted List integration in production, use EU DSS — you'll understand what it's doing under the hood, and you'll be able to debug it when it behaves unexpectedly.

Chapter 23 provides a complete walkthrough of EU DSS for PAdES signing and validation.

10.5.5 EU DSS Version History

EU DSS follows a rapid release cycle, with major versions roughly annually:

- **DSS 5.x** (2017–2022): The long-lived stable series. Most documentation and blog posts you'll find online reference 5.x APIs.
- **DSS 6.x** (2023–present): The current generation, with Java 17+ as the minimum, updated dependencies (including PDFBox 3.x for the PAdES module), and improved validation reporting. The API changed significantly between 5.x and 6.x — if you're following tutorials, make sure they match your version.

EU DSS 6.x uses PDFBox 3.x internally. If your project also uses PDFBox directly (for example, for document manipulation before signing), you must ensure version alignment. Having DSS pull in PDFBox 3.0.0 while your code uses PDFBox 3.0.1 can cause subtle `NoSuchMethodError` exceptions at runtime. Use Maven's `<dependencyManagement>` to lock the PDFBox version across your entire project.

10.6 Analysis and Debugging Tools

A signing implementation is only as good as your ability to inspect what it produces. The following tools are indispensable for PDF signature development.

10.6.1 RUPS: The PDF Object Explorer

RUPS (Reading and Updating PDF Syntax) is a Java Swing application that lets you browse the internal object structure of a PDF file. It was originally developed by Bruno Lowagie as a companion to iText, and it shows every COS object, dictionary key, stream, and cross-reference entry in the file.

For signature work, RUPS is invaluable because it lets you:

- Inspect the `/Sig` dictionary and all its keys (`/ByteRange`, `/Contents`, `/SubFilter`, `/Filter`)
- Navigate the field tree to see how signature fields are structured
- View the `/AcroForm` dictionary and its `/SigFlags`
- Examine `/Lock` and `/SV` (Seed Value) dictionaries
- Browse incremental updates to see what changed between revisions
- Inspect the raw byte content of the `/Contents` hex string

RUPS is available on GitHub at `LibrePDF/OpenPDF` (it's bundled with the OpenPDF project). It's also available as a standalone download. The interface is spartan — it's a developer tool, not a user-facing product — but once you learn to navigate the tree view, it becomes the fastest way to understand what's actually inside a PDF.

When debugging a signature failure, the first thing you should do is open the PDF in RUPS and check:

1. Does the `/Sig` dictionary exist? (Navigate: Root AcroForm Fields [signature field] V)
2. Is the `/ByteRange` array plausible? (Four integers, the first should be 0, the last should be close to the file size)
3. Is `/Contents` non-empty? (Should be a long hex string, not all zeros)
4. What is the `/SubFilter`? (Should be `adbe.pkcs7.detached` or `ETSI.CAdES.detached`)

If any of these are wrong, you've found your problem before even looking at the CMS.

10.6.2 veraPDF: The Compliance Validator

veraPDF is the industry-standard PDF/A validation tool, developed by the Open Preservation Foundation and the PDF Association. While its primary mission is PDF/A compliance checking, it includes PDF analysis capabilities that are useful for signature work:

- **Policy-based validation** — you can define custom validation profiles that check signature-specific requirements
- **Feature extraction** — enumerate all signatures, certificates, and cryptographic properties in a PDF
- **Structural analysis** — verify that the PDF structure conforms to the specification

veraPDF is available as a CLI tool, a Java library (which you can embed in your validation pipeline), and a GUI application. It's the validator that national archives and government agencies use to verify PDF/A compliance — if your signed PDFs also need to be PDF/A compliant (as many government document workflows require), veraPDF is essential.

10.6.3 QPDF: The Swiss Army Knife

QPDF is a C++ command-line tool (created by Jay Berkenbilt) that can inspect, transform, and repair PDF files at the structural level. It's not Java, but every Java PDF developer should have it installed.

Key capabilities for signature work:

- `qpdf --show-xref` — displays the cross-reference table, showing where every object lives in the file. Critical for understanding byte range issues.
- `qpdf --show-object=N` — displays a specific object by number. Faster than navigating a GUI when you know the object number from an error message.
- `qpdf --check` — validates the PDF's structural integrity. If this reports problems, your signing code has a bug.
- `qpdf --qdf` — produces a "QDF" (QPDF data format) output where the PDF is restructured for human readability, with objects in order and comments showing object numbers. Invaluable for understanding what incremental saves changed.
- `qpdf --linearize` — linearizes the PDF for fast web viewing. **Do not linearize after signing** — it destroys the signature by reorganizing the file.

Several QPDF operations will destroy existing signatures:

- `--linearize` — reorganizes the entire file
- `--normalize-content=y` — reprocesses content streams
- `--object-streams=generate` — restructures object storage
- Any operation that produces a non-incremental output

Always use `--copy-encryption=source.pdf` and check whether the operation preserves incremental structure before running it on a signed PDF.

10.6.4 MuTool (from MuPDF)

MuTool is the command-line companion to MuPDF (Artifex's open-source PDF renderer). Its `mutool show` command provides quick access to PDF objects:

```
mutool show document.pdf trailer          # Show trailer dictionary
mutool show document.pdf xref             # Show cross-reference table
mutool show document.pdf 42 0             # Show object 42, generation 0
mutool show document.pdf outline          # Show document outline/bookmarks
```

MuTool is lightweight and fast — useful for quick inspections when you don't want to launch a GUI tool.

10.6.5 OpenSSL: CMS Verification Outside the PDF Context

For verifying the CMS signature independently of the PDF wrapper, OpenSSL is the standard tool:

```
# Extract the raw signature bytes from /Contents (after hex-decoding)
# and save as signature.der

# Verify the CMS structure
openssl cms -inform DER -in signature.der -noout -cmsout -print

# Verify the signature against the signed data
openssl cms -inform DER -in signature.der -verify \
    -content signed-data.bin \
    -CAfile ca-bundle.pem \
    -out /dev/null
```

This separates the CMS verification from the PDF byte-range logic — if OpenSSL says the CMS is valid but your PDF validator says the signature is invalid, the problem is in the byte range handling, not the cryptography.

10.6.6 ASN.1 Dump Tools

When debugging CMS issues, being able to read the raw ASN.1 structure is essential. Several tools help:

- **`openssl asn1parse`** — parses DER-encoded ASN.1, showing the tag-length-value tree
- **BouncyCastle's `ASN1Dump`** — `org.bouncycastle.asn1.util.ASN1Dump.dumpAsString(obj)` — prints any ASN.1 object as a readable tree from Java code
- **`dumpasn1`** (Peter Gutmann) — the original ASN.1 dumper, still the most detailed output
- **Web-based tools** — several websites parse ASN.1/DER online; useful for quick checks but do not upload production signatures to third-party websites

10.7 Choosing the Right Library for Your Project

After all this history, here is the practical guidance:

10.7.1 Decision Matrix

Table 10.3: Library selection for PDF signature projects

Requirement	PDFBox + BC	EU DSS	OpenPDF	iText 7/8 (AGPL)	iText 7/8 (Commercial)
License	Apache 2.0	LGPL 2.1	LGPL 2.1	AGPL 3.0	Proprietary
Cost	Free	Free	Free	Free (if AGPL-compatible)	€€€€
PAdES-B-B	Yes (manual CMS)	Yes (built-in)	Basic	Yes (built-in)	Yes (built-in)
PAdES-B-LTA	Yes (manual DSS)	Yes (built-in)	No	Yes (built-in)	Yes (built-in)

Requirement	PDFBox + BC	EU DSS	OpenPDF	iText 7/8 (AGPL)	iText 7/8 (Commercial)
EUTL / eIDAS validation	No (manual)	Yes (built-in)	No	Partial	Yes
PDF 2.0	Yes	Yes (via PDFBox)	No	Yes	Yes
Community size	Large (Apache)	Medium (EU-backed)	Small	N/A (AGPL)	Vendor-supported
Signing API maturity	High	Very high	Moderate	High	High
Low-level PDF access	Excellent	Limited (framework)	Limited	Good	Good
Our recommendation	**Learn + custom**	**Production PAdES**	Legacy migration	Open-source projects	Budget permits

10.7.2 Our Recommendation

For the projects this book targets — production signing services, validation pipelines, signature analysis tools — **Apache PDFBox 3.x with BouncyCastle** is the default recommendation:

1. **Apache 2.0 license**: No surprises, no AGPL obligations, no license fees. This will not change.
2. **Low-level access**: Signing requires understanding and manipulating the raw PDF structure. PDFBox's COS model gives you direct access. iText and OpenPDF abstract this away, which is convenient until you need to debug a byte-range computation or inspect a malformed signature dictionary.
3. **Active maintenance**: PDFBox has a dedicated Apache community, regular releases, and responsive handling of security issues. The Bochum attacks were patched quickly.
4. **Separation of concerns**: PDFBox handles the PDF, BouncyCastle handles the crypto. You understand both layers because you built both layers. When something breaks, you know exactly where to look.

The trade-off is that PDFBox makes you build more yourself — especially the CMS layer. That's what Chapters 29–31 are for.

The best way to learn is to build it yourself. Using a library that does everything for you (like commercial iText) gets you to production faster, but leaves you helpless when something goes wrong. With PDFBox + BouncyCastle, you understand every byte of the signature because you put it there. When a customer reports that their signature "shows invalid in Adobe but valid in Foxit," you'll know exactly where to look — because you wrote the code that constructed the CMS, computed the byte range, and injected the signature.

10.8 The Ecosystem at a Glance

To close this chapter, here is a summary of every tool and library we've discussed, organized by function:

Table 10.4: The Java PDF signature ecosystem

Function	Tool/Library	License	Notes
PDF manipulation (read/write)	Apache PDFBox 3.x	Apache 2.0	Default choice for this book. Reader-first architecture.
PDF manipulation (read/write)	OpenPDF 2.x	LGPL 2.1	Community fork of iText 2.x. Good for legacy migration.
PDF manipulation (read/write)	iText 7/8	AGPL / Commercial	Feature-rich but expensive or license-restrictive.
CMS / Cryptography	BouncyCastle 1.80+	MIT (Bouncy Castle License)	De facto standard for Java CMS. Register provider explicitly.
PAdES signing framework	EU DSS 6.x	LGPL 2.1	European Commission reference implementation. PAdES-B-B through B-LTA, validation reports, EU Trusted List integration.
PDF structure inspection	RUPS	LGPL	GUI object browser. First tool to reach for.
PDF/A compliance	veraPDF	Open source (MPL 2.0)	Industry-standard validator. Also useful for general PDF analysis.
PDF structural tools	QPDF	Apache 2.0	CLI Swiss army knife. `--show-xref`, `--qdf`, `--check`.
PDF rendering/inspection	MuTool (MuPDF)	AGPL	Lightweight CLI. Quick object inspection.
CMS verification	OpenSSL	Apache 2.0	CMS verification independent of PDF context.
ASN.1 inspection	BouncyCastle `ASN1Dump`	MIT	Java-based ASN.1 tree dump.
ASN.1 inspection	`dumpasn1` (Gutmann)	Public domain	The classic ASN.1 dumper.

Every tool in this table will appear at some point in the remaining chapters. Install them now — when you're debugging a signature failure at 2 AM, you don't want to be downloading tools.

10.9 Key Takeaways

- **iText's license change** (LGPL AGPL in 2009) split the Java PDF world in two. The consequences — old versions in production, community forks, enterprise migrations — are still visible in 2026.
- **OpenPDF** is the LGPL continuation of iText 2.x. Useful for legacy migration but limited for modern PAdES signing.
- **PDFBox** + **BouncyCastle** is the recommended stack for new signing projects. Apache-licensed, actively maintained, gives you low-level control.
- **PDFBox 3.x** broke API compatibility with 2.x. Budget migration time. Key changes: `Loader.loadPDF()`, new font constructors, content stream API changes.
- **BouncyCastle** provides the CMS layer that PDFBox doesn't. Always register the provider explicitly. Use `jdk18on` artifacts.
- **RUPS, QPDF, veraPDF, OpenSSL** — know your tools. PDF signature debugging is 50% knowing what to look at and 50% having the right tool to look at it.
- **The library you choose is a 10-year decision.** Choose based on license stability, community health, and technical fit — not just today's feature checklist.